

Vysoká škola ekonomická v Praze
Fakulta informatiky a statistiky



Návrh a testování prostředí pro AI agenty v softwarovém vývoji

BAKALÁŘSKÁ PRÁCE

Studijní program: Aplikovaná informatika

Autor: Thanh An Nguyen

Vedoucí práce: Ing. Jiří Korčák

Praha, květen 2026

Prohlášení

Prohlašuji, že jsem bakalářskou práci zpracoval samostatně a že jsem uvedl všechny použité prameny a literaturu, ze kterých jsem čerpal.

Potvrzuji, že při přípravě předložené práce **byly použity** nástroje umělé inteligence. Konkrétně byly použity těmito způsoby:

- úprava a rozšíření textových částí,
- vyhledávání a shrnutí literatury,
- generování částí zdrojových kódů.

Každé použití je dokumentováno v příloze ??.

Potvrzuji, že:

- má práce je originální a byla zpracována v souladu s etickými standardy, zejména s Etickým kodexem Vysoké školy ekonomické v Praze; že jsou v práci citovány všechny použité informační zdroje a že generovaný obsah byl mnou ověřen;
- přijímám plnou odpovědnost za celý obsah závěrečné práce.

V Praze dne 24. dubna 2026

Thanh An Nguyen

Poděkování

Poděkování.

Abstrakt

AI coding agenti dokáží generovat funkční kód, ale bez vhodných instrukcí nedodržují softwarově-inženýrské praktiky. Existující benchmarky měří pouze výsledek (vyřešil agent úkol?), nikoli kvalitu procesu ani kódu. Formalizovaný postup pro návrh a vyhodnocování instrukcí dosud chybí. Práce si klade tři cíle: (1) navrhnout sadu metrik měřících proces, kvalitu a efektivitu práce agenta; (2) demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami; (3) ablacemi prozkoumat, které složky instrukční sady přispívají k měřenému chování agenta a které jsou redundantní. Sadu metrik a postup ověřujeme na případové studii systému upomínek faktur. Agent (model MiniMax-M2.5 prostřednictvím nástroje OpenCode) prošel pěti pilotními iteracemi a čtyřmi ablačními běhy. Chování bylo měřeno 19 metrikami ve třech kategoriích: procesní (P), produktové (Q) a efektivita (E). Metriky zachytily, že iterativní úpravy instrukcí zlepšily dodržování pracovního postupu, ale ne monotónně. Nejlepší pilotní běh dosáhl plného splnění procesních metrik P1–P5 a zvýšil počet projitých referenčních testů z 39/42 na 41/42. Analýza odhalila vzorec vývoje instrukcí: obecné pravidlo → konkrétní příkaz → verifikační krok. Ablace ukázaly, že verifikační kroky nejsou redundantní; jejich odebrání zhoršilo funkční korektnost i automatizované metriky kvality kódu. Sekce kódových konvencí byla pro automatizované metriky převážně redundantní, ale ovlivňovala designovou kvalitu hodnocenou LLM-as-judge. Výsledky současně ukázaly, že procesní compliance je výrazně citlivá na nedeterminismus modelu. Přenositelným přínosem jsou sada metrik a iterativní postup; konkrétní naměřené hodnoty jsou vázány na podmínky případové studie.

Klíčová slova

AI coding agent, AGENTS.md, metriky kvality software, iterativní návrh, ablační studie

Abstract

AI coding agents can generate functional code, but without proper instructions they fail to follow software engineering practices. Existing benchmarks measure only the outcome (did the agent solve the task?), not process quality or code quality. A formalized process for designing and evaluating instructions is still lacking. This thesis pursues three goals: (1) design a metric suite measuring agent process, quality, and efficiency; (2) demonstrate an iterative instruction design process driven by these metrics; (3) use ablation to explore which instruction components contribute to the observed behavior and which are redundant. The metric suite and

process are evaluated on a single case study of a dunning system. The agent (MiniMax-M2.5 model via the OpenCode tool) underwent five pilot iterations and four ablation runs. Behavior was measured using 19 metrics in three categories: process (P), product (Q), and efficiency (E). The metrics showed that iterative instruction changes improved workflow adherence, but not monotonically. The best pilot run achieved full compliance on process metrics P1–P5 and increased reference-test performance from 39/42 to 41/42. Analysis revealed an instruction evolution pattern: general rule → specific command → verification step. Ablation showed that verification steps are not redundant: removing them degraded both functional correctness and automated code-quality metrics. The code-convention section proved largely redundant for automated metrics, but it still affected design quality as assessed by an LLM-as-judge. The results also showed that process compliance is highly sensitive to model non-determinism. The transferable contributions are the metric suite and the iterative process; the specific measured values are bound to the case study conditions.

Keywords

AI coding agent, AGENTS.md, software quality metrics, iterative design, ablation study

Obsah

Seznam obrázků

Seznam tabulek

Seznam použitých zkratek

Úvod

AI coding agenti dokáží autonomně implementovat funkcionalitu, psát testy, spravovat verzovací historii a komunikovat prostřednictvím issues a pull requestů. S rostoucí autonomií těchto nástrojů roste i potřeba systematicky hodnotit kvalitu jejich práce. Současné benchmarky hodnotí izolované aspekty práce agenta, především funkční korektnost. Holistický pohled, který by zachytil proces, kvalitu kódu a efektivitu společně, však chybí. Přitom i funkčně správné řešení může být v praxi odmítnuto kvůli nedostatkům v procesu nebo udržitelnosti. Praktik, který vidí jen pass/fail výsledek, tak může přecenit řešení, které by ve skutečném vývoji neprošlo code review nebo by se obtížně rozvíjelo.

Kvalitu práce agenta výrazně ovlivňují instrukce, které dostane. V praxi se etablují instrukční soubory jako `AGENTS.md` nebo `CLAUDE.md`, které definují pracovní postup, konvence a omezení projektu. Tyto soubory se stávají standardním rozhraním mezi člověkem a agentem nejen v softwarovém vývoji, ale i v univerzálních agentních nástrojích. Jak instrukce systematicky navrhovat a jak měřit, zda agent dodržuje požadované praktiky, zůstává otevřenou otázkou.

Tato práce navrhuje sadu metrik pokrývajících proces, kvalitu kódu a efektivitu práce agenta. Na případové studii necháváme agenta opakovaně implementovat systém upomínek faktur ze specifikace, výsledky měříme navrženými metrikami a na jejich základě upravujeme instrukce mezi iteracemi. Ablacemi dále zkoumáme, které složky instrukcí přispívají k měřenému chování a které jsou redundantní.

Práce nejprve vymezuje problém a cíle, poté shrnuje teoretická východiska z oblasti softwarového inženýrství, AI agentů a měření kvality. Na jejich základě navrhuje metodiku, kterou ověřuje na případové studii. Kapitola Vyhodnocení a diskuse interpretuje výsledky ve vztahu k cílům a porovnává je s existujícím výzkumem.

[RAW]

Audit trail (úvod, předchozí verze v2): 4 odstavce: kontext + gap (benchmarky jen pass/fail), instrukce jako problém, co práce dělá, průvodce po kapitolách. Zdroje tvrzení: benchmarky jen pass/fail potvrzeno z kap02 sekce 2.3.1 (SWE-bench, SWE-EVO, FeatureBench, ACE-Bench). “Pass $\neq$ merge” z METR 2026, citováno v kap02. Cíle odpovídají kap01 řádky 97–109.

Audit trail (úvod, předchozí verze v1): 3 odstavce: kontext (agenti + instrukce), co práce dělá (metriky + ablace), průvodce po kapitolách (výčet kap 1–5). Průvodce nahrazen stručným popisem logiky návaznosti.

1. Vymezení problému a cílů práce

1.1 Motivace

AI coding agenti, tedy autonomní systémy schopné samostatně psát kód, spravovat verzovací historii a interagovat s vývojovým prostředím, se v posledních letech staly rozšířeným nástrojem každodenního vývoje softwaru. Slibují výrazné zrychlení práce: implementace, která vývojáři zabere hodiny, je teoreticky proveditelná v řádu minut. Empirická data ale ukazují, že tento příslib se nenaplnuje automaticky. Randomizovaná studie METR (METR, 2025) na 246 úlohách doložila, že vývojáři s AI nástroji byli ve skutečnosti o 19 % pomalejší.

Současné benchmarky jako SWE-bench (Jimenez et al., 2024) hodnotí izolované aspekty práce agenta, především funkční korektnost. Tři empirické studie ale shodně ukazují, že samotný pass/fail výsledek pro praktickou použitelnost nestačí. METR (METR, 2026)¹ doložila, že polovina patchů projitých automatickými testy by nebyla přijata při code review. Li et al. (B. Li et al., 2026) na 1 210 sloučených pull requestech zjistili kvalitativní nedostatky i v přijatém kódu. Ehsani et al. (Ehsani et al., 2026) pak z 33 tisíc agentních pull requestů identifikovali, že vedle funkčních chyb tvoří významný podíl odmítnutí i procesní selhání. Každá studie pojmenovává jinou stranu téhož problému: pass/fail neměří, zda by řešení obstálo v code review, zda je kód udržovatelný a zda vznikl auditovatelným postupem. Holistický pohled, který by tyto dimenze zachytil společně, dosud chybí.

Výsledky agenta nejsou dány jen schopnostmi modelu samotného; lze je ovlivnit dvěma způsoby: trénováním modelu nebo instrukcemi v kontextovém okně. S rostoucí velikostí kontextových oken a zkracujícími se cykly mezi generacemi modelů se instrukce stávají praktičtější alternativou: jsou dostupné komukoliv, iterativně upravitelné a přenositelné mezi modely. Fine-tuning naproti tomu vyžaduje trénovací data, výpočetní kapacitu a investici vázanou na konkrétní verzi modelu. Shin et al. (Shin et al., 2025) ukazují, že iterativní zpřesňování instrukcí dosahuje srovnatelných výsledků s fine-tuned modely na úlohách s kódem.

Empirie o účinnosti instrukcí je ale smíšená. Lulla et al. (Lulla et al., 2026) zjistili, že přítomnost instrukčního souboru je spojena se zkrácením doby běhu o 28,6 %, zatímco Gloaguen et al. (Gloaguen et al., 2025) upozorňují, že generické instrukce přinášejí marginální zlepšení při vyšších nákladech. Které složky instrukcí k měřenému chování skutečně přispívají a které jen zabírají kontextové okno, dosud nebylo izolováno. Jak instrukce systematicky navrhovat a jak měřit, zda agent dodržuje požadované praktiky, zůstává proto otevřenou otázkou.

[RAW]

Audit trail (motivace, v8 → v9, 2026-04-22): Prohozeny předposlední a poslední odstavec.

¹Jiná studie téže organizace než (METR, 2025); zde analýza SWE-bench pull requestů, ne randomizovaný experiment.

Předtím končil odstavec o smíšené empirii (“otevřená otázka”), pak technický odst. o fine-tuningu; to oslabovalo přechod do cílů. Nově: nejdřív zdůvodnění proč instrukce (lze ovlivnit chování, fine-tuning vs instrukce, Shin srovnatelné), pak smíšená empirie a open question, který přímo vede do sekce Cíle. Úvodní věta předposledního odstavce nově otevřena větou “Výsledky agenta nejsou dány jen schopnostmi modelu samotného; lze je ovlivnit ...” aby zarámovala proč o způsobech ovlivnění vůbec mluvíme.

Audit trail (motivace, v7 → v8, 2026-04-22): Rozšířen 1. odstavec. Předchozí verze (“AI agenti se používají k vývoji softwaru, ale... Randomizovaná studie METR...”) byla příliš stručná a přecházela rovnou k METR bez kontextu. Nově: definice AI coding agentů → rozšířenost → příslib zrychlení → ale empirie → METR. Zachován věcný styl bez populárně-naučné hyperboly (“revoluční technologie” apod.).

Audit trail (motivace, v6 → v7, 2026-04-22): Sloučeny odst. 2 (Ehsani + Shin) a odst. 3 (SWE-bench + METR + Li) do jednoho gap-focused odstavce. Důvod: METR, Li i Ehsani dokládají tutéž tezi (pass/fail nestačí), jen z jiné strany (review, kvalita, proces). Roztržení mezi dva odstavce dělalo argument fragmentovaný. Shin přesunut do odst. 4 (instrukce vs fine-tuning), kde už byl citován; v odst. 2 se opakoval. Odst. 3 nově: instrukce jako řídicí nástroj + Lulla/Gloaguen + open question. Z 5 odstavců na 4.

Audit trail (motivace, předchozí verze DRAFT v4): 3 odstavce: (1) METR + Ehsani + Shin v jednom, (2) benchmarky jen SWE-bench, (3) gap instrukcí bez citací. Změny v v5: odst. 1 rozdělen na kontext (METR) a problém (Ehsani + Shin); benchmarky doplněny o METR pass≠merge a Li merge≠kvalita; gap instrukcí doplněn o Lulla a Gloaguen (protichůdné výsledky).

Audit trail (motivace, předchozí verze DRAFT v3): 4 odstavce: (1) METR + agent chybí, (2) benchmarky měří jen výsledek, (3) ICL vs fine-tuning, Shin, chybí postup, (4) co práce dělá. Změny v v4: přesunuto ICL/fine-tuning do rozsahu (1.3); instrukce jako klíčový nástroj sloučeny do odst. 1; odst. 4 smazán (duplicitní s cíli a úvodem); přidána motivace pro ablaci (odst. 3).

[RAW]

Předchozí verze motivace (audit trail):

Velké jazykové modely (LLM) dnes umí generovat kód, ale to neznamená, že umí vyvíjet software. Randomizovaná studie METR (METR, 2025) na 16 zkušených vývojářích a 246 úlohách ukázala, že s AI nástroji byli vývojáři o 19 % pomalejší, přestože sami odhadovali zrychlení o 24 %. Když projekt roste, bývá těžší jej rozšiřovat, jak pro člověka, tak pro LLM. Vývojáři přicházejí o kontext a hlubokou znalost kódové základny, zatímco agenti jsou limitováni omezeným context window. Otázka tedy není, co modely umí, ale jak je zasadit do vývojového procesu tak, aby produkovaly kvalitní výstup.

Ukazuje se, že odpověď leží spíš v instrukcích než v samotných modelech. Breunig (Breunig,

2025) zjistil, že změna promptu mění chování agenta výrazněji než výměna modelu. Lulla et al. (Lulla et al., 2026) dokládají, že přítomnost instrukčního souboru (AGENTS.md) je spojena se zkrácením mediánové doby běhu o 28,6 % a snížením výstupních tokenů o 16,6 %. Scaffolding, tedy struktura instrukcí a procesních pravidel, která agenta provází vývojem, má na výsledek zásadní vliv. Ale nikdo systematicky nezkoumal, *které* složky tohoto scaffoldingu jsou pro dodržování softwarově-inženýrských praktik nezbytné a které jsou zbytečné.

Tato práce zkoumá právě to: jak musí být strukturovány instrukce pro AI coding agenta, aby dodržoval specifikací řízený vývoj, test-driven development a granulární správu verzí; a co z těch instrukcí je skutečně kritické.

Původní poznámky:

Studie společnosti METR.ORG ukazuje, že LLM zkušené vývojáře spíše zpomaluje. S rychlým vývojem schopností modelů se situace pravděpodobně mění. Ale to neznamená, že je LLM samo o sobě dostatečné k vypracování dlouhotrvajících úkolů. Není to problém pouze LLM, když projekt roste, bývá těžší jej rozšiřovat jak pro člověka, tak i pro LLM. AI programování tenhle problém ještě více prohlubuje. Vývojáři přicházejí o kontext a hlubokou znalost kódové základny (codebase), zatímco velké jazykové modely (LLM) jsou limitovány omezenou pamětí (context window). Jak nastavit harness/scaffolding tak, aby v tom mohli fungovat agenti a lidé to stále měli pod kontrolou?

[RAW]

Review poznámky (2026-03-21):

(a) **Motivace nemotivuje cíl 3 (ablace):** Motivace vysvětluje proč potřebujeme metriky (cíle 1) a iterativní postup (cíle 2), ale neříká proč potřebujeme zjišťovat které složky instrukcí jsou nezbytné. Chybí věta typu: “Zůstává otevřená otázka, které složky instrukcí jsou pro dodržování praktik skutečně nezbytné a které jsou redundantní.” Starší verze motivace (audit trail, řádek 39) to obsahuje (“nikdo systematicky nezkoumal, *které* složky”), ale aktuální DRAFT ne.

(b) **Řádek 27, silný claim bez citace:** “pro systematické navrhování a vyhodnocování instrukcí obdobný postup dosud nevznikl”; buď citovat systematic review ukazující gap, nebo zeslabit na “je méně formalizován” s odkazem na kap02.

(c) **Řádek 56, “exit kritéria” bez vysvětlení:** Čtenář narazí na pojem poprvé v cíli 2, ale definice je až v kap03. Buď přidat závorku “(měřitelné prahy pro úspěšnost běhu, viz sekce ??)”, nebo přeformulovat na “stanovených požadavků”.

(d) **Terminologie “evaluační systém” vs “sada metrik”:** Řádek 29 říká “evaluační systém”, cíle 1 říká “sada metrik”. Jsou to totéž? Pokud evaluační systém = sada metrik + postup + exit kritéria, definovat explicitně. Pokud totéž, sjednotit.

TODO (2026-03-26): po přeformulaci cílů (DRAFT v6, PoC framing):

- **Motivace nemotivuje cíl 3:** přidat větu o tom, že zůstává otevřená otázka které složky instrukcí jsou nezbytné (viz bod (a) výše).
- **Rozsah:** přidat větu o feasibility demonstration; práce ukazuje že přístup funguje na jednom případě, ne že platí obecně. Sladit s PoC framingem cílů.
- **Rozsah ř. 152:** “evaluační systém” sjednotit s cíli (cíl 1 říká “sadu metrik”, cíl 2 říká “iterativní postup”). Buď definovat “evaluační systém = metriky + postup” nebo nahradit konkrétními pojmy.
- **Cíl 2:** “exit kritéria” zmizela z cíle (teď říká “měřitelné zlepšení”), takže bod (c) výše je vyřešen.

1.2 Cíle práce

1. Na základě analýzy existujících standardů kvality softwaru a současných benchmarků pro AI agenty navrhnout sadu metrik pokrývající proces, kvalitu kódu a efektivitu, a tím zachytit dimenze, které stávající benchmarky neměří.
2. Na případové studii demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami a vyhodnotit, zda a jak vede k měřitelným změnám v dodržování vývojových praktik.
3. Z instrukcí vytvořených v cíli 2 prozkoumat ablacemi, které složky přispívají k měřenému chování agenta a které jsou redundantní.

[RAW]

Audit trail: předchozí verze cílů (DRAFT v5):

1. Navrhnout sadu metrik která měří proces a kvalitu práce AI agenta, ne jen výsledek.
2. Iterativním postupem navrhnout instrukce které dovedou agenta k dodržování stanovených exit kritérií.
3. Ablacemi identifikovat, které složky instrukcí přispívají k měřenému chování agenta a které jsou redundantní.

Audit trail: předchozí verze cílů (DRAFT v4):

1. Navrhnout sadu metrik která měří proces a kvalitu práce AI agenta, ne jen výsledek.
2. Na případové studii demonstrovat iterativní postup pro návrh instrukcí s využitím těchto metrik.
3. Popsat pozorované tendence v chování agenta a vliv jednotlivých složek instrukcí na tyto tendence.

Poznámky ze schůzky s vedoucím (2026-03-16): Cíle práce = 3 fáze: (1) research gap: procesní metriky, (2) pilotní fáze: docílit exit kritérií, (3) komparativní fáze: ablace/substituce. Research question potřebuje přesná kritéria a kvantifikaci. **Pozn.:** substituce se nakonec realizovala implicitně v pilotní fázi (r1→r3), komparativní fáze obsahuje jen ablace.

Předchozí verze cílů (audit trail v3):

1. Navrhnout instrukce které dovedou AI coding agenta k dodržování strukturovaného vývojového postupu na případové studii (systém upomínek faktur).
2. Navrhnout sadu metrik která měří proces a kvalitu práce agenta, ne jen výsledek.
3. Navrhnout a demonstrovat iterativní postup pro systematický návrh a vyhodnocování instrukcí.

Předchozí verze cílů (audit trail v2):

1. Popsat, jak se řízení softwarových projektů mění v kontextu AI agentů, a zmapovat co víme o vlivu instrukcí na jejich chování.
2. Iterativně navrhnout instrukce, které dovedou AI coding agenta k dodržování vývojového postupu na case study projektu (systém upomínek faktur).
3. Zjistit, které složky těchto instrukcí jsou pro dodržování vývojového postupu nezbytné, které jsou zbytečné, a jaký kontext agent potřebuje k tomu, aby vytvářel a využíval projektové artefakty.

Původní verze:

1. Popsat jak se řízení SWE projektů mění v kontextu agentních systémů (teoretický rámec)
2. Navrhnout a implementovat experimentální prostředí (case study: systém upomínek faktur)
3. Prozkoumat vliv různých nastavení scaffoldingu na schopnost agenta provést kvalitní práci
4. Identifikovat jaký kontext je pro agenty klíčový a jak instrukce/procesy ovlivňují schopnost agenta tento kontext vytvářet a využívat

1.3 Rozsah práce

Cíle ověřujeme na případové studii systému upomínek faktur. Existující studie hodnotí agenty na velkém počtu krátkých úloh; naše práce volí opačný přístup: jeden projekt do hloubky, s opakovanými běhy a podrobnou analýzou každého z nich.

Práce se nezabývá trénováním modelů, protože cílem je zkoumat vliv instrukcí, ne schopnosti konkrétního modelu. Nesrovnává různé modely ani programovací jazyky, protože experimentální design se snaží držet ostatní podmínky co nejstabilnější a měnit především instrukce. Neporovnává agenta s lidským vývojářem, protože měří zlepšení mezi iteracemi, ne absolutní výkon. Navržená sada metrik a iterativní postup jsou koncipovány jako přenositelný rámec; konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii a slouží jako feasibility demonstrace, nikoli jako obecně platný výsledek pro všechny modely a projekty. Zdůvodnění volby projektu a podrobnosti experimentálního designu popisuje kapitola ??.

[RAW]

Audit trail (rozsah, předchozí verze DRAFT v3): 3 odstavce bez kontextu originálu. Změny v v4: vráceno blíže k originálu (v1). Ponechány: proč případová studie, dunning systém stručně, co neděláme, přenositelnost. Odstraněno oproti v1: “evaluační systém” → “sada metrik a iterativní postup”; kontrolované podmínky zkráceny (details v kap03). ICL/fine-tuning přesunuto do motivace (odst. 5).

Audit trail (rozsah, předchozí verze DRAFT v2): 3 odstavce: (1) instrukce ne fine-tuning (stručně), (2) jedna případová studie, feasibility, (3) co neděláme.

Audit trail (rozsah, předchozí verze DRAFT v1): 4 odstavce: (1) proč případová studie (nelze teoreticky), (2) kontrolované podmínky (vše stejné, jen instrukce), (3) dunning systém (deterministický, testovatelný), (4) co neděláme + přenositelnost.

[RAW]

Poznámky k propojení:

- Řízení vyžaduje holistický pohled (vidět celek, ne jen část) - proto celý SDLC, ne jedna fáze
- Billing Reminder Engine jako case study: malý projekt, ale reálné nuance (state machine, business days, edge cases)
- Deterministická logika (stejný vstup = stejný výstup) + jasná pravidla správnosti = objektivně testovatelné
- Hraniční případy (víkendy, svátky, grace periods) = místa kde lze testovat kvalitu agentní práce

TODO: Vysvětlit termíny:

- state machine - ?
- edge cases / hraniční případy - ?
- SDLC - ?

Scope SDLC (k diskuzi):

- Primární plán: celý SDLC včetně deployment/maintenance
- Fallback: zúžit na implementation + testing (hlavní doména coding agents)
- Poznámka: i impl + testing má feedback loop (implementace → testy → chyba → úprava) = stále systémový pohled

2. Teoretická východiska

Tato kapitola poskytuje teoretický základ pro metrický rámec a experimentální postup práce. První sekce se věnuje kvalitě software z pohledu softwarového inženýrství: proč je její zajištění obtížné, jaké dimenze má, jakými praktikami se zajišťuje a jak se měří. Druhá sekce zavádí AI coding agenty: co jsou, jak se řídí instrukcemi a jak se jejich výstupy hodnotí.

[RAW]

Audit trail (signpost kapitoly): Původní signpost odkazoval na čtyři celky; po restrukturalizaci (2026-04-08) kapitola má dvě hlavní sekce (2.1 kvalita, 2.2 agenti).

2.1 Kvalita software a její měření

Softwarové inženýrství je disciplína, která přistupuje k vývoji software systematicky: od specifikace přes implementaci po údržbu (IEEE Computer Society, 2024). Na rozdíl od programování, které se soustředí na algoritmy a implementaci, softwarové inženýrství řeší mimo jiné i to, jak zajistit kvalitu výsledku (Sommerville, 2016). To zahrnuje řízení požadavků, návrh architektury, správu konfigurací, zajištění kvality i řízení projektu. Kvalita se přitom v této disciplíně posuzuje na více entitách: nejen podle vlastností výsledného software produktu, ale i podle procesů, jimiž tento produkt vzniká, a podle zdrojů, které k tomu byly spotřebovány. Tato sekce proto nejprve zavádí tři entity měření podle Fentona a Biemana a shrnuje, jak je literatura pokrývá (sekce ??). Poté popisuje dimenze kvality produktu, procesu a zdrojů podrobněji (sekce ??) a nakonec shrnuje praktiky, kterými se kvalita v tradičním vývoji zajišťuje (sekce ??).

2.1.1 Měření kvality v softwarovém inženýrství

Měření kvality v softwarovém inženýrství se neomezuje na vlastnosti výsledného artefaktu. Fenton a Bieman (Fenton & Bieman, 2014) rozlišují tři kategorie entit, na kterých lze kvalitu měřit: *produkt*, tedy výstupy vývoje (implementace, testy, dokumentace); *proces*, tedy aktivity a postupy, kterými produkt vzniká; a *zdroje*, tedy vstupy nutné k jeho vzniku (čas, úsilí, lidé, výpočetní prostředky). Každá entita nese jiný typ evidence o kvalitě a celkové hodnocení je obvykle skládá dohromady.

Literatura pokrývá tyto tři entity asymetricky. Produktová stránka je nejzralejší. Standardizovaný rámec ISO/IEC 25010 (ISO/IEC, 2023) rozkládá kvalitu produktu do osmi charakteristik a navazuje na dřívější modely, například na McCallův (**mccall1977**); detailněji se jim věnuje sekce ?? . Procesní stránka má zavedené rámce v podobě modelů procesní zralosti: CMM/CMMI (**paulk1993cmm**) a rodina norem ISO/IEC 330xx (**iso33020**) popisují, jak hodnotit capability organizačního procesu. Tyto rámce ale pracují v horizontu auditu zralosti

napříč projekty a roky, nikoli v horizontu pozorování jednoho experimentálního běhu. Procesní metriky pro konkrétní experiment se proto obvykle definují ad hoc. Zdrojová stránka bývá zachycena zejména přes proxy nákladů, úsilí a času (Fenton & Bieman, 2014).

Tato asymetrie se v kapitole ?? stává motivací pro operacionalizaci jednotlivých dimenzí v kontextu coding agentů: pro produkt lze navázat na zavedené charakteristiky, pro proces a zdroje je třeba zvolit konkrétnější podobu než jakou nabízejí obecné rámce.

[RAW]

Audit trail (2.1.1, holistic restructure v2 2026-04-24): Sekce znovu přepsána ve druhé holistické iteraci. Po první verzi vyšlo najevo, že otevírat ISO/IEC 25010 v 2.1.1 dává čtenáři produktový mental model ještě předtím, než se zavedou tři entity měření. Nová 2.1.1 proto zavádí Fenton & Bieman rámec (produkt / proces / zdroje) a explicitně pojmenovává asymetrii pokrytí v literatuře: produkt má zralý standard (ISO 25010, dopředný odkaz na novou 2.1.2), proces má rámce zralosti (CMMI/ISO 33020) jen pro organizační audit a zdroje jsou typicky proxy nákladů. Tím vzniká explicitní motivace pro operacionalizaci v kap03. ISO 25010 charakteristiky se přesouvají do nové 2.1.2, která dimenze rozebírá podrobněji ve třech odstavcích (produkt, proces, zdroje). Paragraf o Brooksově složitosti odstraněn z 2.1.1; nepatří k měření kvality, ale k praktikám, a v case study se na něj již neopíráme.

Audit trail (2.1.1, holistic restructure 2026-04-24): Sekce přepsána po strukturální analýze celé kap02 z prvních principů (nezávisle na anchor handoffu). Klíčové změny: (a) McCallův model zredukován na zmínku v paragrafu o ISO/IEC 25010; samostatný odstavec odstraněn, protože McCall se v práci dál nepoužívá. (b) Definiční věty pro funkční vhodnost a udržitelnost integrovány do paragrafu o ISO 25010 a doplněny o operacionalizaci pro kontext BP (“v této práci to znamená, zda agent. . .”). (c) Odstavec o tom, jak se dvě dimenze v dalších sekcích měří, odstraněn; šlo o předčasný výhled na 2.1.2. (d) Brooks zkrácen; esenciální/akcidentální složitost popsána slovy, nikoli jen termínem. Celkově 3 odstavce místo 7; obsah redukován na to, co zbytek práce reálně potřebuje jako oporu pro Q metriky.

Audit trail (2.1.1, bridge cleanup 2026-04-22): Doplněny tři chybějící logické mosty v finálním textu: od více stakeholderů k modelům kvality, od vybraných charakteristik k jejich odlišnému typu evidence a od modelů kvality ke složitosti jako důvodu, proč je dosažení kvality obtížné.

Audit trail (2.1.1, DRAFT v9 — sloučení): Sloučeny sekce 2.1.1 (Složitost) a 2.1.2 (Dimenze kvality) do jedné sekce “Kvalita software”. Nový tok: definice kvality (IEEE) → modely (McCall, ISO 25010) → dvě klíčové dimenze → složitost jako výzva (Brooks, Lehman) → bridge na praktiky. Složitost již není rámována jako příčina kvality, ale jako jeden z aspektů, které dosažení kvality komplikují.

Předchozí stav (2 oddělené sekce): 2.1.1 Složitost software (DRAFT v8): Brooks esenciální/akcidentální, Lehman zákony, příklady z case study, důsledky. 2.1.2 Dimenze kvality

(DRAFT v1): McCall, ISO 25010, funkční vhodnost vs udržitelnost.

Audit trail (2.1.1, DRAFT v8): Přepis sekce. SWE definice přesunuta do signpost sekce 2.1. Sommerville fáze a Boehm spektrum odstraněny (přesun do 2.1.3 nebo signpost). Složitost rozepsána: příklady esenciální/akcidentální (systém upomínek), Lehman dva konkrétní zákony (kontinuální změna, rostoucí složitost), důsledky (interakce, chyby se propagují).

Audit trail (2.1.2, DRAFT v1): McCall 1977 citován přes Miguel et al. 2014 (survey). ISO/IEC 25010 citován přímo (standard). Dvě klíčové dimenze (funkční vhodnost, udržitelnost) vybrány jako relevantní; ostatní zmíněny ale ne rozvedeny.

Předchozí verze 2.1.1: DRAFT v7: SWE definice + Brooks/Lehman stručně + Sommerville/Boehm proces + 3 paragraphs (procesní, funkční, strukturální). DRAFT v6–v1: viz předchozí audit trail.

Předchozí RAW 2.1.2: TODO: Napsat o ISO 25010, modelech kvality.

2.1.2 Praktiky zajišťování kvality

Předchozí sekce vymezila, jaké vlastnosti charakterizují kvalitní software. Samotné vymezení těchto charakteristik však ještě neříká, jak jich při vývoji dosáhnout. Praktiky zajišťování kvality přitom v softwarovém inženýrství tvoří širší spektrum činností od práce s požadavky a specifikací přes organizaci změn a jejich ověřování až po průběžnou péči o strukturu kódu. Tato sekce se z tohoto širšího rámce soustředí na tři skupiny praktik, které se přímo promítají do hodnocení agentních běhů a výsledného kódu: organizaci práce se změnami, testování a postupy podporující udržitelnost kódu.

Procesní praktiky.

Procesní praktiky určují, jak změny vznikají, kontrolují se a začleňují do projektu. Jejich význam spočívá v tom, že zvyšují auditovatelnost a spolehlivost vývoje.

Základem těchto praktik je správa verzí, protože umožňuje sledovat historii změn a bezpečně se vrátit k předchozím stavům kódu. Pro kvalitu vývoje však není rozhodující jen samotný nástroj, ale způsob, jakým se používá. Práce v oddělených větvích (*branches*) izoluje rozpracované změny od stabilního kódu; Shihab et al. (Shihab et al., 2012) empiricky ukázali, že nesoulad mezi strukturou větví a organizační strukturou je spojen s vyšší mírou defektů po vydání. Granulární commity, kde každá změna odpovídá jednomu logickému kroku, usnadňují zpětnou analýzu chyb a code review (Humble & Farley, 2010). Průběžná integrace (*continuous integration*) tyto praktiky doplňuje automatickým ověřováním, že sloučení změn nenarušilo funkčnost (Humble & Farley, 2010).

Další procesní praktikou je code review, tedy kontrola kódu jiným vývojářem před jeho začleněním. Historicky probíhala formou strukturovaných inspekcí, dnes převládá méně formální podoba integrovaná do pull requestů, označovaná jako *modern code review*. Jde o průběžné posuzování menších změn jako běžné součásti integrace, nikoli o samostatnou formální inspekci celé implementace. Bacchelli a Bird (Bacchelli & Bird, 2013) empiricky zjistili, že moderní code review přispívá nejen k odhalování defektů, ale především ke zlepšování kvality kódu, sdílení znalostí a nalézání alternativních řešení. McIntosh et al. (McIntosh et al., 2016) dále prokázali, že kód, který neprošel review, vykazuje měřitelně vyšší míru defektů po vydání.

Vedle správy verzí a code review je pro kvalitu vývoje důležitá také *traceability*, tedy schopnost dohledat vazby mezi požadavkem a artefakty, které s ním souvisejí (Gotel & Finkelstein, 1994). Nejde jen o uchování historie kódu, ale o propojení zadání změny, její implementace, review a ověření. V praxi to znamená vazby mezi issue, commity a větvemi, pull requestem a testy. Díky nim lze zpětně pochopit, proč byla změna provedena, co přesně měnila a jak bylo ověřeno, že odpovídá zadání.

Testování.

Testování ověřuje, zda software skutečně plní požadavky, které na něj klademe. Jeho význam spočívá v tom, že poskytuje systematickou kontrolu chování výsledného řešení a zachycuje chyby dříve, než se projeví v provozu.

Testování se provádí na různých úrovních: *unit testy* ověřují jednotlivé funkce nebo moduly izolovaně, *integrační testy* ověřují spolupráci komponent a *akceptační testy* ověřují splnění požadavků z pohledu zadavatele. Každá úroveň zachycuje jiný typ chyb.

Z hlediska pořadí lze testování organizovat různými způsoby. Nejběžnější je *test-after* přístup, kdy vývojář napíše implementaci a následně k ní vytvoří testy. Přístup *test-driven development* (TDD) pořadí obrací: nejprve se napíše test definující očekávané chování, teprve pak implementace, která test splní (Beck, 2000). Tím se test stává specifikací požadavku, nikoli zpětnou kontrolou již napsaného kódu. Meta-analýza 27 studií Rafiqua a Mišiče (Rafique & Mišić, 2013) ukazuje, že TDD má pozitivní efekt na kvalitu kódu měřenou hustotou defektů. Efekt je výraznější v průmyslových projektech než v akademických experimentech, což naznačuje, že přínos roste se složitostí řešeného problému.

Praktiky podporující udržitelnost.

Praktiky podporující udržitelnost mají zajistit, aby kód zůstal srozumitelný a bezpečně modifikovatelný i při dalších změnách. Jejich význam spočívá v tom, že omezují růst složitosti a snižují riziko, že každá další úprava bude dražší a náchylnější k chybám.

V praxi se udržitelnost opírá o vlastnosti jako čitelnost, srozumitelné pojmenování, oddělení

zodpovědností, omezení složitosti nebo dodržování konvencí jazyka (McConnell, 2004). Proto ji podporují průběžný refaktoring, důsledná práce s rozhraními a odpovědnostmi modulů i automatizované kontroly kódu. SWEBOK zároveň zdůrazňuje, že její úroveň neposilují jen vlastnosti samotného kódu, ale i systematické vývojové procesy, techniky a nástroje (IEEE Computer Society, 2024). Některé z těchto vlastností lze ověřovat deterministicky, jiné však vyžadují i expertní posouzení; právě k těmto způsobům hodnocení se vrací následující sekce.

[RAW]

Audit trail (2.1.1, DRAFT v7): Rozšíření paragraphs o širí přístupů (teorie mapuje landscape, výběr v kap03):

- Procesní: přidána CI, zmínka o historických inspekcích vs. moderní code review
- Funkční: přidány test-after, TDD, BDD jako tři přístupy; odstavec o zdroji očekávaných hodnot (spec vs. pozorování)
- Strukturální: přidáno spektrum od deterministických nástrojů (statická analýza, typová kontrola, McCabe) po expertní posouzení; McCabe vrácen sem z 2.1.2
- Brooks esenciální/akcidentální terminologie vrácena (stručně)

Audit trail (předchozí verze): DRAFT v6: tři aspekty kvality, bridging věty, “mimo jiné”. DRAFT v5: paragraphs odvozené z metrik (ne z aspektů). DRAFT v4 a starší: viz předchozí audit trail.

2.1.3 Měření kvality software

Předchozí sekce identifikovala praktiky, kterými se kvalita software zajišťuje. Samotná existence praktik však neznamená, že se dodržují, ani že jejich výstupy jsou kvalitní. Kvalitu je proto třeba měřit.

Fenton a Bieman (Fenton & Bieman, 2014) rozlišují tři kategorie měřitelných entit v softwarovém inženýrství: *proces* (aktivity během vývoje), *produkt* (výstupy vývoje, například implementace, testy nebo dokumentace) a *zdroje* (lidé, nástroje, výpočetní kapacita). Toto členění je důležité, protože různé entity poskytují různý typ evidence. Nejde přitom o alternativu k modelu ISO/IEC 25010 z předchozí sekce, ale o jiný řez problémem: ISO/IEC 25010 vymezuje, *které* charakteristiky kvality produktu sledovat, zatímco Fenton a Bieman rozlišují, *na jaké entitě* kvalitu měříme.

Produktové měření.

Produktové měření se ptá, zda výsledné artefakty splňují požadované vlastnosti. Navazujeme zde na dvě dimenze vybrané v sekci ?? (funkční vhodnost a udržitelnost) a popisujeme, jakými způsoby se měří. Funkční vhodnost znamená shodu chování implementace se specifikací. Testování proto není měřenou vlastností samo o sobě, ale způsobem, jak tuto shodu

zjišťovat: test porovnává pozorovatelné chování programu s očekávaným chováním odvozeným z požadavků nebo akceptačních kritérií.

Testovací sada je zároveň samostatný produktový artefakt. U ní nestačí vědět jen to, zda testy prošly, protože průchod testů vypovídá primárně o testované implementaci, nikoli o síle samotných testů. Strukturální coverage (line, branch) ukazuje, jak velkou část kódu testy vykonaly, ale sama o sobě neříká, zda by testy zachytily skutečnou chybu (Papadakis et al., 2019). *Mutation testing* proto coverage doplňuje o druhý pohled: do programu systematicky zavádí drobné změny (*mutanty*) a sleduje, kolik z nich testy odhalí. Papadakis et al. (Papadakis et al., 2019) jej proto chápou jako měření síly testovací sady.

Udržovatelnost se na rozdíl od funkční vhodnosti neprojevuje primárně v jednom běhu programu, ale ve vlastnostech zdrojového kódu: zda je srozumitelný, lokalizovatelný při změně a bezpečně modifikovatelný. Část těchto vlastností lze hodnotit statickou analýzou, tedy zkoumáním kódu bez jeho spuštění. *Cyklomatická složitost* (McCabe, 1976) vyjadřuje počet lineárně nezávislých cest grafem řízení toku; vyšší hodnoty souvisejí s vyšší chybovostí a nižší testovatelností a v praxi se často používá práh 10 na funkci (McConnell, 2004). Linting a typová kontrola podobně zachycují porušení konvencí, nekonzistence v rozhraních a chyby v práci s datovými typy. Statická analýza je však pouze proxy pro vybrané atributy udržovatelnosti; vhodnost dekompozice, pojmenování nebo přiměřenost návrhových rozhodnutí přesahují možnosti čistě deterministických nástrojů.

Procesní měření.

Produktové měření samo nestačí, protože dvě řešení mohou mít podobný výsledek, a přesto vzniknout odlišně kontrolovatelným postupem. Procesní měření proto sleduje, zda praktiky popsané v sekci ?? skutečně proběhly a zda jejich artefakty (commity, issues, pull requesty, záznamy z CI) nesou požadovanou strukturu a kvalitu. Tento typ evidence neodpovídá na otázku, zda je výsledný produkt správný, ale zda vznikl způsobem, který je sledovatelný, přezkoumatelný a opakovatelný.

Měření zdrojů.

Třetí kategorií jsou metriky zdrojů, tedy nákladů spojených se vznikem výsledku. V tradičním softwarovém inženýrství sem patří zejména čas, lidské úsilí, použití nástrojů a výpočetních prostředků (Fenton & Bieman, 2014). U AI coding agentů tato rovina nabývá konkrétní podoby v čase běhu, spotřebovaných tokenech, použití nástrojů a stabilitě session. Dvě řešení mohou mít srovnatelnou produktovou kvalitu, ale lišit se náklady, za které byla vytvořena. Proto představují zdrojové metriky samostatnou rovinu hodnocení.

Z tohoto rozlišení plyne pro tuto práci základní rámec: hodnocení práce agenta nemůže stát pouze na tom, zda výsledný kód projde testy. Je třeba sledovat výstup, postup jeho vzniku

i náklady, za které byl vytvořen. Následující sekce proto přechází od měření kvality software obecně ke specifikům AI coding agentů a jejich hodnocení.

[RAW]

Audit trail (2.1.3, přepis 2026-04-24): Sekce znovu zpřesněna po strukturální kontrole. Hlavní změna: produktové měření již není rámováno jako “testování”, ale jako hodnocení výstupních artefaktů vůči požadovaným vlastnostem. Doplněno, že funkční vhodnost znamená shodu implementace se specifikací a testování je způsob zjištění této shody. Testovací sada je nově explicitně označena jako samostatný produktový artefakt, aby bylo později srozumitelné oddělení kvality implementace a kvality testů v kapitole ???. Procesní a zdrojové měření bylo zesíleno tak, aby sekce nepůsobila jako katalog testovacích nástrojů, ale jako rámec produkt/proces/zdroje.

Audit trail (2.1.3, finální přepis v2 2026-04-21): Sekce znovu přepsána po revizi argumentu. Předchozí verze byla příliš blízko katalogu nástrojů a míchala dvě roviny: ISO charakteristiky kvality a Fentonovu klasifikaci měřených entit. Nový tok je: praktiky → proč nestačí bez měření → ISO říká co na produktu sledovat, Fenton na čem měřit → produktové měření → procesní měření → zdrojové měření → bridge na agenty. Záměrně ponecháno stručné; benchmarky, LLM-as-judge a test oracle problem zůstávají až v sekci ???.

Audit trail (2.1.3, finální přepis v1 2026-04-21): Sekce přepsána z DRAFT do finálního textu. Nový tok: praktiky → proč nestačí bez měření → Fentonova trojice (proces, produkt, zdroje) → produktové měření (testování, mutation testing, statická analýza jako proxy) → meze statické analýzy → procesní a zdrojové měření → bridge na hodnocení agentů. LLM-as-judge a test oracle problem ponechány v sekci ???, aby tato sekce zůstala na úrovni obecného měření kvality software.

Audit trail (2.1.2, DRAFT v1, dříve 2.1.3): Přepsáno z RAW do DRAFT. Sloučeny 4 subsubsections do paragraphs. Struktura: Fenton taxonomie (úvod) → testování → statická analýza → udržitelnost + LLM-as-judge.

Faktické opravy:

- Papadakis 36 %: odstraněno konkrétní číslo, přeformulováno kvalitativně (mutation testing zachycuje co coverage přehlédne).
- Harman/Meta 70 %: odstraněno (číslo neověřitelné ze zdroje).
- McCabe práh 10: citace opravena z mccabe1976 na mcconnell2004 (McCabe definoval metriku, práh je z pozdější praxe).
- Panickssery: odstraněna zmínka o perplexitě (není v paperu), přeformulováno na “korelace self-recognition a self-preference”.

Strukturální změny:

- Interní/externí atributy odstraněny (nikde v kap03–05 nepoužito).

- Dopředné odkazy “kap03 na to navazuje” odstraněny.
- Přidán paragraph Udržovatelnost (zakotvení pro Q8 design quality, dříve chybělo).
- Yin et al. ponechán implicitně — explicitní zmínka patří do kap03 kde se P/Q/E zavádí.

Předchozí RAW obsah (audit trail): 4 subsubsections: Klasifikace (Fenton, interní/externí, Yin), Testování (Papadakis 36 %, Meta 70 %, Mathews, Chen, TDD), Statická analýza (McCabe práh 10 → mccabe1976, linting, tsc), LLM-as-judge (Zheng 3 biasy, Panickssery perplexita, Verga PoLL).

Poznámka (restrukturalizace 2026-04-08): Paragraph “Udržovatelnost a kvalitativní hodnocení” (LLM-as-judge) přesunut do sekce ?? (2.2.3), kde lépe zapadá do kontextu hodnocení AI agentů. Test oracle problem (Mathews et al., Chen et al.) přesunut tamtéž.

2.2 AI coding agenti

Předchozí sekce popsala, jak se kvalita software zajišťuje a měří v tradičním vývoji. AI coding agenti tento kontext mění: kód již nepíše pouze člověk, ale autonomní systém řízený instrukcemi. Požadavky na kvalitu zůstávají stejné, mění se však to, kdo je plní a jak lze výsledek hodnotit. Tato sekce zavádí základní pojmy (sekce ??), popisuje jak se chování agentů řídí instrukcemi (sekce ??) a jak se jejich výstupy hodnotí (sekce ??). Pro tuto práci je tento krok nutný, protože právě instrukce tvoří nezávislou proměnnou mezi běhy: abychom mohli v kapitole ?? obhájit návrh metrik a experimentu, musíme nejprve vyjasnit, co je agent, co je instrukce a jak se jeho chování obvykle hodnotí.

[RAW]

Audit trail (signpost 2.2): Původní TODO: “Pojmy a typy → jak mění vývoj.” Přepsáno na signpost odkazující na tři subsekce.

2.2.1 Základní pojmy a typy

Velký jazykový model (large language model, LLM) je neuronová síť trénovaná na rozsáhlých textových korpusech (Vaswani et al., 2017). Na základě předchozího kontextu predikuje pravděpodobný následující token, a tím generuje koherentní text, včetně zdrojového kódu. Samotný LLM je však pasivní: odpovídá na dotazy, ale nemůže samostatně číst soubory, spouštět příkazy ani měnit stav projektu.

LLM-based agent model rozšiřuje o schopnost interagovat s prostředím (J. Liu et al., 2024): plánuje kroky, volá nástroje (*tool use* (Schick et al., 2023)) a na základě pozorovaného výsledku rozhoduje o dalším kroku v cyklu Thought → Action → Observation (Yao et al., 2023). V kontextu softwarového vývoje to znamená, že agent může číst a zapisovat soubory,

spouštět testy, používat verzovací systém nebo volat API. Právě tyto schopnosti určují, jak agent zpracuje instrukce a jaké měřitelné stopy po sobě zanechá.

Předmětem této práce jsou *autonomní coding agenti*, tedy systémy, které na základě specifikace a instrukcí samostatně implementují softwarový projekt (např. Claude Code, OpenHands) (Guo et al., 2025). Na rozdíl od asistenčních nástrojů (GitHub Copilot) agent provádí celý vývojový proces, nikoli jen jednotlivé kroky. Jeho výstupy proto lze posuzovat stejnými požadavky na kvalitu procesu i produktu, jaké popisuje sekce ??.

[RAW]

Audit trail (2.2.1, DRAFT v1): Přepsáno z RAW poznámek do souvislého textu. Zachovány klíčové zdroje: Vaswani (Transformer), Liu (agent components), Yao (ReAct), Schick (Toolformer), Guo (3 paradigmata). Prompt engineering a context engineering rozlišení přesunuto do 2.2.2 kde lépe kontextuálně zapadá.

Doporučení k validaci (strukturální audit, 2026-04-22): Zvážit zkrácení a přerámování 2.2.1 kolem otázky, které vlastnosti agenta jsou důležité pro naše měření a instrukce. Aktuální tok může působit příliš učebnicově: Transformer → LLM → agent → ReAct → Toolformer → typologie paradigmát. Pokud se to potvrdí, ponechat jen minimum nutné pro BP a thesis-specific relevance přivést dříve.

Předchozí RAW obsah: 1. Large Language Model (LLM):

→ vaswani2017 – Transformer architektura:

“*Attention Is All You Need*” – self-attention mechanismus.

LLM = velký jazykový model založený na transformer architektuře.

2. LLM-based Agent:

→ liu2024llmagents:

Agent = LLM rozšířený o schopnost interagovat s prostředím.

4 klíčové komponenty:

- **Planning** – dekompozice komplexních úkolů na pod-úkoly
- **Memory** – krátkodobá (kontext) a dlouhodobá (RAG, databáze)
- **Perception** – vnímání prostředí (čtení souborů, výstupů)
- **Action** – provádění akcí (tool calls, zápis souborů)

3. Tool Calls (Function Calling):

→ yao2022react – ReAct framework:

Cyklus: Thought → Action → Observation.

→ schick2023toolformer:

LLM se může naučit kdy a jak použít nástroje.

4. Prompt Engineering:

[DOPLNIT] Definovat prompt engineering jako disciplínu formulování instrukcí pro LLM.

Důležité rozlišení pro BP:

- **Prompt engineering** = jak formuluješ konkrétní instrukci
- **Context engineering** = co všechno agent dostane jako kontext (viz sekce ??)
- **Scaffolding** = struktury v prostředí které agenta vedou

[RAW]

Obsah z původní 2.2.2 (Typy coding agentů): Evoluce podle autonomie:

→ guo2025benchmarks:

Tři paradigmaty s rostoucí autonomií:

1. **Prompt-based** – člověk instruuje, LLM odpovídá (ChatGPT, copilot)
2. **Fine-tune-based** – model adaptován na SE doménu (CodeLlama, StarCoder)
3. **Agent-based** – autonomní plánování, akce, učení (Devin, Claude Code, OpenHands)

Copilot vs Autonomous Agent:

- **Copilot** – asistuje člověku, návrhy, autocomplete
- **Autonomous agent** – samostatně plánuje a vykonává úkoly

Příklady nástrojů (2024-2025): Devin, Claude Code, Cursor, OpenHands, Agentless (Xia et al., 2024).

2.2.2 Scaffolding a instrukce

Autonomní agent nemá *tacit knowledge*, tedy implicitní znalosti, které lidský vývojář získává zkušeností: jaké konvence tým dodržuje, proč je kód strukturovaný určitým způsobem nebo jaké chyby se v projektu opakují. Nonaka a Takeuchi (Nonaka & Takeuchi, 1995) popisují převod tacit knowledge na explicitní jako klíčový proces v organizacích. Pro agenty je tento převod

nutný v plném rozsahu: veškerý kontext, který agent potřebuje, musí být zapsán explicitně. Hassan et al. (Hassan et al., 2025) tuto dualitu formalizují v rámci SASE (*Software Engineering for Agents*): zatímco lidský vývojář (SE4H) čerpá z meetingů, code review a zkušenosti, agent (SE4A) pracuje výhradně s tím, co dostane jako vstup.

Scaffolding označuje strukturované artefakty v prostředí agenta, které jeho chování řídí. V praxi jde nejčastěji o instrukční soubory umístěné v repozitáři (např. `AGENTS.md`, `CLAUDE.md`), které definují roli agenta, pracovní postup, konvence a omezení. Na rozdíl od jednorázových promptů jsou tyto soubory verzované a iterativně upravované, což z nich činí “living documents,” jejichž vývoj lze zpětně sledovat.

Mao et al. (Mao et al., 2025) analyzovali 2 163 produkčních prompt šablon a identifikovali sedm typů komponent: Role, Directive, Context, Workflow, Output, Constraints a Examples. Zjistili, že Role a Directive se nejčastěji objevují na začátku dokumentu a že Workflow (procedurální kroky) je nejúčinnější komponentou pro složité úkoly. Toto zjištění potvrzují Li et al. (X. Li et al., 2025), kteří ukazují, že procedurální instrukce (“jak pracovat”) jsou efektivnější než popisná dokumentace (“jak vypadá codebase”).

Empirická evidence o účinnosti instrukčních souborů je smíšená a ukazuje, že samotná přítomnost instrukcí nestačí: rozhodující je jejich obsah, struktura a relevance pro konkrétní úlohu. Přítomnost souboru `AGENTS.md` je sice spojena s nižší mediánovou dobou běhu agenta (−28,6 %) (Lulla et al., 2026), ale dopad závisí na původu obsahu: LLM-generované instrukční soubory úspěšnost agenta snižují a i soubory napsané vývojáři přinášejí jen marginální zlepšení (Gloaguen et al., 2025). V benchmarku SkillsBench zlepšují kurátorované moduly výsledky o 16,2 procentních bodů, zatímco automaticky generované nemají žádný efekt (X. Li et al., 2025). Účinnost jednotlivých typů obsahu přitom dosud nebyla izolována.

Chování agenta je navíc citlivé na formulaci instrukcí. Breunig et al. (Breunig, 2025) ukazují, že přestrukturování instrukce je účinnější než její opakování: když agent pravidlo ignoruje, nepomůže ho zopakovat; je třeba ho přeformulovat tak, aby se stalo součástí pracovního postupu. Razavi a Fard (Razavi & Fard, 2025) dokládají, že drobné změny formulace promptu mohou dramaticky změnit chování modelu, což motivuje cílené úpravy místo přidávání dalších pravidel.

Na chování agenta působí také *specificita* instrukce. Kim et al. (Kim, 2025) v benchmarku DETAIL a Zi et al. (Zi et al., 2025) na úlohách s kódem shodně ukazují, že konkrétnější formulace zvyšují úspěšnost agenta, ale přílišný detail může omezit jeho reasoning. Instrukce se tak pohybují po ose od volné nápovědy po explicitní příkaz, přičemž optimální bod závisí na úloze i modelu.

Výzkum promptingu navíc naznačuje, že instrukce nepůsobí na chování modelu jedním mechanismem. Wei et al. (J. Wei et al., 2022) ukazují, že i krátká nápověda (“think step by step”) aktivuje reasoning, který se bez ní neprojeví, a Min et al. (Min et al., 2022) dokládají, že demonstrace v in-context learning působí spíš jako strukturální vodítka než jako přímé příkazy. Chování modelu lze tedy ovlivnit přímým *vynucením* i *aktivací* latentních znalostí;

která z cest je v konkrétním případě účinnější, nelze rozhodnout předem.

Otevřenou otázkou proto není jen to, zda instrukce obecně pomáhají, ale jak jejich obsah systematicky navrhovat a jak poznat, které složky skutečně mění chování agenta.

[RAW]

Audit trail (2.2.2, DRAFT v1): Přepsáno z RAW poznámek (sloučeno z původní 2.2.2 + 2.3.x). Klíčové zdroje: Nonaka (tacit/explicit), Hassan SASE (SE4H/SE4A), Mao FSE (7 komponent), SkillsBench (curated skills), Lulla (AGENTS.md přítomnost), Gloaguen (LLM-generated škodí), Breunig (přestrukturování), Razavi (prompt sensitivity).

Vynecháno z RAW (přílišný detail pro teorii):

- Memory systémy (A-MEM, kontextové moduly) – detail patří do přehledu, ne do BP teorie
- Context engineering survey – příliš široké, specifika v kap04
- Specifikace jako vstup (formáty, tiers) – patří do kap04
- SASE scripty (Briefing/Loop/Mentor) – detail, stačí princip
- Operacionalizace škála – empirický poznatek z kap04/05, ne teorie

Předchozí RAW obsah: Které fáze agenti ovlivňují:

→ jin2024 – 6 klíčových oblastí: Requirement Engineering, Code Generation, Autonomous Decision-making, Software Design, Test Generation, Software Maintenance.

Co zůstává stejné: Fáze, artefakty, quality gates, potřeba jasných requirements.

Co se mění:

- **Rychlost** – minuty místo hodin/dnů
- **Explicitnost** – vše musí být napsané (agent nemá tacit knowledge)
- **Role** – developer se stává reviewerem a specifikátorem
- **Batch size** – menší úkoly, častější iterace

Micro-waterfall hypotéza: Sekvenční kroky zůstávají – jen se zmenšuje batch size a zrychluje feedback loop. SDD formalizuje tento pattern (Piskala, 2026).

Empirická podpora:

→ watanabe2025agentprs (ACM TOSEM): 567 Claude Code PRs, 83.8% acceptance rate, 45% potřebovalo revizi.

→ ehsani2026failedprs (MSR 2026): 33k agent PRs. Malé, focused úkoly mají vyšší úspěš-

nost.

[RAW]

TODO: Přepsat signpost sekce (sloučeno ze 2.3). Problém kontextu → struktura instrukcí → specificita a mechanismy účinku. Jádru kap02: instrukce jsou nezávislá proměnná celé práce.

[RAW]

Předchozí obsah (z 2.3.1 Problém kontextu):

Agent nemá tacit knowledge → potřebuje explicitní kontext.

→ nonaka1995:

Articulation = převod tacit knowledge na explicit.

→ hassan2025sase – SASE framework:

SE for Humans (SE4H) vs SE for Agents (SE4A).

Co agent potřebuje vědět:

- **Requirements** – co má být výsledkem
- **Architektura** – kde se změna má udělat
- **Konvence** – coding standards, patterns, gotchas
- **Historie** – proč je kód takový jaký je (rationale)

[RAW]

Předchozí obsah (z 2.3.2 Přístupy ke kontextu):

1. Memory systémy pro agenty:

→ memorymechanism2024, amem2025

Kategorie: Short-term, Long-term, Parametric, Non-parametric.

2. Context Engineering:

→ contextengsurvey2025, ace2025

3. Strukturované artefakty:

→ hassan2025sase – SASE: BriefingScript, MentorScript, LoopScript.

4. Repository-level context:

→ contextmodule2024, bugfixcontext2025

5. Git-based přístupy:

→ gcc2025 – Git Context Controller

6. Specifikace jako vstup (SWE-bench): GitHub Issues jako standardní formát specifikace pro agenty. Traceability: Issue → branch → commits → PR → merge (Gotel & Finkelstein, 1994).

7. Co dělá specifikaci efektivní pro LLM:

→ rope2024: kvalita requirements = kvalita výstupu

→ ullrich2025: requirements příliš abstraktní pro přímý LLM vstup

→ yang2025underspec: pod-specifikované prompty 2× náchylnější k regresi

→ specine2025: 10 alignment rules, Tier 1: příklady, účel, výstupní požadavky

→ ticoder2024: testy jako součást specifikace (+45.97% Pass@1)

→ domaincodegen2024: doménový kontext

→ newcomb2025prepost: pre/postconditions

→ wen2024io: I/O specifikace

→ clarifygpt2024: klarifikace ambiguity

→ reprompt2025: RE metodologie na prompty

→ swebenchpro2025: strukturované requirements v benchmarcích

Syntéza tiers: Tier 1 (nejvyšší dopad): Description/účel, AC s příklady, výstupní specifikace. Tier 2: vstupní specifikace, doménový slovník, edge cases. Tier 3: pre/postconditions, error handling, behavioral model.

[RAW]

Předchozí obsah (z 2.3.3 Struktura instrukcí, přejmenováno z “Praktické techniky”):

1. Agent Harness:

Anthropic, “Effective harnesses for long-running agents”: init.sh, progress tracking, git commits jako checkpointy.

2. Meta-prompt soubory:

→ hassan2025sase – AGENT.md pattern:

CLAUDE.md, .clinerules, AGENT.md. “Employee handbook” pro AI.

3. Living Documents: Version-controlled, iterativní refinement, auditable history.

4. Prompt template komponenty (Mao et al.):

→ `mao2025fse` – 7 komponent prompt šablony, empirické pořadí efektivity.

5. Script balance (Hassan et al.):

→ `hassan2025sase` – BriefingScript / LoopScript / MentorScript balance.

6. Content effectiveness (Lulla et al.):

→ `lulla2026` – AGENTS.md přítomnost: $-28,6\%$ mediánový runtime, $-16,6\%$ výstupní tokeny. Neizoluje efekt jednotlivých typů obsahu.

7. Prompt sensitivity:

→ `razavi2025`, `breunig2025` – přestrukturování > opakování. Malá změna formulace mění chování víc než výměna modelu.

→ `shin2025prompt` – explicitní instrukce s kontextem výrazně zlepšují výkon agenta.

Formát specifikace: GitHub Issues

Specifikace jako GitHub Issues. Volba vychází z:

1. Akademický standard – SWE-bench (Jimenez et al., 2024)
2. Agilní RE praxe (Cao & Ramesh, 2008)
3. Open source praxe (Scacchi, 2002)
4. Nativní čitelnost pro agenty (GitHub API/CLI)
5. Traceability (Gotel & Finkelstein, 1994)

Dvě vrstvy specifikace:

1. **Requirements** (CO): Title, Description, AC (Given/When/Then), Domain glossary
2. **Architecture** (JAK): Type definitions, Invariants, Behavioral model, Constraints

Zasazení do SASE frameworku: Specifikace = BriefingScript. AGENTS.md = LoopScript + MentorScript.

Human-AI Collaboration:

→ `humanai2024`: AI zlepšuje efektivitu, human oversight zůstává kritický.

RAW TODO (2026-03-24, aktualizováno 2026-03-30): Škála operacionalizace instrukcí a limity instruction following.

Doplnit sekci o tom, jak instrukce působí na chování agenta. Místo dvou mechanismů (vy-nucení/aktivace) rámovat jako škálu operacionalizace: hint/enabler → pravidlo → příkaz → verifikační krok. Různé body na škále fungují na různé cíle: hint pro emergentní chování (design, architektura), verifikační krok pro měřitelné metriky.

Proč pravidla nefungují je otevřená otázka. Relevantní literatura:

- Unfaithful CoT: Turpin et al. 2023 (NeurIPS, arXiv:2305.04388): CoT neodráží skutečný reasoning modelu
- Lanham et al. 2023 (Anthropic, arXiv:2307.13702): větší modely = méně faithful reasoning
- Curse of Instructions (2024, OpenReview): $P(\text{all}) = P(\text{single})^N$, s každou instrukcí klesá compliance
- Enabling constraints (Juarrero 2023, Cynefin/Snowden 2007): omezení která umožňují emergentní chování
- Expertise reversal effect (Kalyuga et al. 2003): instrukce pro nováčka škodí expertovi
- Instruction specificity (Kim et al. 2025 DETAIL, Zi et al. 2025): specifická zlepšuje výsledky ale přílišný detail omezuje reasoning
- Chain-of-thought (Wei et al. 2022): minimální hint aktivuje latentní schopnost
- In-context learning (Min et al. 2022): demonstrace fungují jako strukturální nápovědy, ne jako příkazy

2.2.3 Hodnocení AI agentů

De facto standardem pro hodnocení coding agentů je SWE-bench (Jimenez et al., 2024): benchmark obsahující 2 294 úloh z reálných GitHub repozitářů, kde agent dostane popis problému (GitHub issue) a má vytvořit patch, který projde testy. Hodnocení je binární: patch projde nebo neprojde. Obdobně HumanEval (M. Chen et al., 2021) měří schopnost modelu generovat funkčně správný kód na izolovaných programovacích úlohách.

Tyto benchmarky měří *funkční korektnost*: zda výsledek dělá to, co má. Neměří však, jak agent pracoval (dodržoval proces?), jakou strukturální kvalitu má výsledný kód (je udržovatelný?) ani za jakou cenu výsledku dosáhl (kolik tokenů spotřeboval?). Kapitola ?? tuto mezeru dokládá na třech nezávislých empirických studiích (METR (METR, 2026), Li et al. (B. Li et al., 2026) a Ehsani et al. (Ehsani et al., 2026)), které na různých velkých vzorcích a z různých úhlů ukazují, že pass/fail výsledek samotný neodpovídá praktické použitelnosti patche: nezachycuje code review, udržovatelnost ani procesní integritu. Každá studie pojmenovává jinou stranu téhož problému.

Existují benchmarky, které hodnocení rozšiřují nad rámec pass/fail. SWE-bench Pro (Scale AI, 2025) přidává strukturované požadavky, ACE-Bench („ACE-Bench: End-to-End Feature Development Benchmark“, 2025) testuje end-to-end vývoj a FeatureBench („FeatureBench: Benchmarking Agentic Coding for Complex Feature Development“, 2026) hodnotí implementaci celých funkcí. Žádný z nich však systematicky neměří kvalitu procesu (jak agent

organizoval práci), kvalitu kódu nad rámec funkční korektnosti (udržovatelnost, čistota) a efektivitu (spotřeba zdrojů) současně. Tato mezera naznačuje potřebu metrického rámce, který by dimenze procesu, kvality kódu a efektivitu pokrýval současně. Návrh takového rámce a jeho operacionalizaci v kontextu této práce popisuje kapitola ??.

LLM-as-judge.

Některé dimenze kvality, jako srozumitelnost commit zpráv, kvalita dokumentace nebo vhodnost návrhových vzorů, nelze spolehlivě hodnotit deterministickými nástroji. Jedním z přístupů je využití LLM jako hodnotitele (*LLM-as-judge*). Zheng et al. (Zheng et al., 2023) ukázali, že LLM dokáže hodnotit kvalitu výstupů na úrovni srovnatelné s lidskými hodnotiteli, ale identifikovali tři systematické biasy: *position bias* (preferuje odpověď na určité pozici), *verbosity bias* (preferuje delší odpovědi) a *self-enhancement bias* (preferuje vlastní výstupy). Panickssery et al. (Panickssery et al., 2024) prokázali korelaci mezi schopností modelu rozpoznat vlastní výstupy a mírou, do jaké je preferuje, což naznačuje kauzální vztah mezi self-recognition a self-preference. Verga et al. (Verga et al., 2024) navrhli mitigaci formou panelu diverzních modelů (PoLL), který dosahuje lepší shody s lidským hodnocením než jednotlivý model. Praktickou implikací je volba hodnotícího modelu z odlišné modelové rodiny, než ve které pracuje hodnocený agent.

Test oracle problem.

S testováním souvisí *test oracle problem*: kdo definuje správný výsledek? Mathews et al. (Mathews & Nagappan, 2024) zjistili, že nástroje pro automatické generování testů systematicky filtrují selhávající testy; až 68,1 % výsledných testovacích sad validuje chybné chování místo jeho odhalení. Chen et al. (J. Chen et al., 2025) na 500 úlohách benchmarku SWE-bench Verified potvrdili, že agent-generované testy slouží primárně jako observační zpětná vazba, nikoli jako validační nástroj. Obranou proti tomuto problému je odvozování očekávaných hodnot ze specifikace, nikoli z pozorování kódu. V této kapitole tím vysvětlujeme, proč samotné agentovy testy nestačí jako měřítko správnosti. Konkrétní podobu referenčních testů a jejich použití v našem experimentu ponecháváme až metodice.

[RAW]

Audit trail (2.2.3, DRAFT v1): Přepsáno z RAW poznámek (sloučeno z původní 2.4.x + přesunutý LLM-as-judge z 2.1.4 + test oracle problem). Struktura: benchmarky (SWE-bench, HumanEval) → co neměří (METR, Li, Ehsani) → gap → LLM-as-judge (Zheng, Panickssery, Verga) → test oracle (Mathews, Chen).

Vynecháno z RAW (přílišný detail nebo patří jinam):

- APO přístupy (PromptWizard, PromptAlchemy) – patří do kap05 diskuze

- Fine-tuning vs ICL – obecné ML, ne specifické pro naši práci
- Prompt evaluation nástroje (Langfuse, Braintrust) – praktické, ne teoretické
- Cline hill climbing – praktický příklad, ne teorie
- Sofroniew et al. funkční emoce – zajímavé ale tangenciální
- Trajectory evaluation (AgentBoard, TRACE) – zmíněny v 5.2
- Detailní Zhu et al. agenteval – patří do kap03

Předchozí RAW obsah: NOVÁ SEKCE: přesunuto z různých míst + doplnit.

SWE-bench:

→ **swebench2024** (ICLR 2024): De facto benchmark pro AI coding agenty. 2294 úloh z reálných GitHub repozitářů. Měří: vyřešil agent úkol, ano nebo ne (binary pass/fail).

Další benchmarky:

- SWE-bench Pro (Scale AI, 2025) – rozšíření o explicitní Requirements a Interface sekce
- SWE-EVO (Y. Wei et al., 2025) – evoluční úlohy, 21% úspěšnost
- FeatureBench („FeatureBench: Benchmarking Agentic Coding for Complex Feature Development“, 2026) – feature-level, 11%
- ACE-Bench („ACE-Bench: End-to-End Feature Development Benchmark“, 2025) – end-to-end, 7.5%

Multi-issue gap: SWE-EVO 21%, FeatureBench 11%, ACE-Bench 7.5%. Naše case study (5 issues, celý dunning system) cílí do tohoto rozsahu.

Co benchmarky měří a co NE:

- Měří: funkční korektnost (pass/fail)
- Neměří: proces (jak agent pracoval), kvalita kódu (mutation score, lint, složitost), procesní záznamy (commit messages, issue descriptions)

Gap: Existující benchmarky jsou navrženy pro “vyřešil agent bug?”, ne “vyvinul agent software správně a správným postupem?” Tento gap adresuje naše práce (kap01).

TODO (2026-03-16, aktualizováno 2026-03-31): Explicitně zdůraznit že literatura nemá nic co by měřilo procesní stránku práce agenta: benchmarky měří výsledek (pass/fail), ale ne *jak* agent pracoval (dodržování workflow, kvalita commit messages, TDD, branch management). Toto je klíčový research gap který motivuje cíl 1 v kap01. Propojit s Fenton & Bieman procesní metriky: existují v tradičním SWE, ale nikdo je neaplikoval na AI agenty systematicky.

TODO (2026-03-31): Doplnit benchmarky které se přibližují procesním metrikám. V 5.2.1 potřebujeme porovnat naše P/Q/E s existujícími přístupy. Doplnit do této sekce:

- **METR (2026):** pass $\neq$ merge. Ukazuje že funkční řešení nestačí, reviewer ho odmítne kvůli procesu. Validuje proč procesní metriky potřebujeme.
- **GitGoodBench (JetBrains 2025):** Měří commit quality, nejbližší existující práce k našemu P2. Ale stále na úrovni artefaktů (commit), ne workflow.
- **HumanEval (Chen et al. 2021):** Zmínit jako příklad čistě funkčního benchmarku (pass@k) pro kontrast.
- **Li et al. 2026 (Beyond Bug Fixes, MSR):** 1210 merged PRs, SonarQube analýza. Merge $\neq$ kvalita. Pokrývají $\sim$ Q5–Q7 ale ne P, Q1–Q4, Q8, E.
- **Ehsani et al. 2026 (Failed PRs, MSR):** 33k PRs, taxonomie důvodů odmítnutí. Procesní selhání (duplicitní PR, špatná větev) vedle chyb v kódu.
- **Trajectory evaluation pro obecné agenty:** AgentBoard (Ma et al. 2024, NeurIPS): Progress Rate po krocích. TRACE (Wang et al. 2026, WWW): efektivita a podloženost rozhodování. Neaplikováno na SW proces.

TODO (2026-04-04): Funkční emoce a chování agentů. Sofroniew et al. (Anthropic, 2026): “Emotion Concepts and their Function in a Large Language Model.” Zjistili že Sonnet 4.5 má lineární reprezentace emočních konceptů které **kauzálně ovlivňují chování**:

- Vektor “desperate” zvyšuje reward hacking (5%→70%) a blackmail (22%→72%) při steering +0.05.
- Vektor “calm” tyto chování potlačuje.
- Opakované selhávání v testech aktivuje “desperate” vektor → agent hledá hacky místo řešení.
- Pozitivní emoční vektory (happy, loving) zvyšují sycophancy.

Relevance pro BP: (1) Další evidence proč nestačí měřit jen pass/fail: agent může “vyřešit” úlohu reward hackingem pod tlakem. (2) Instrukce mohou ovlivňovat emoční profil agenta a tím i kvalitu chování. (3) Propojit s kap05 diskuze: naše iterace instrukcí mohly nepřímo ovlivňovat “calm” vs “desperate” profil. **Kam:** Zmínit v 2.2.4 (hodnocení) jako rozšíření “proč pass/fail nestačí”. Detailněji v kap05 diskuze (5.2.2 vliv instrukcí).

[RAW]

Přesunuto z 2.1.2 (restrukturalizace 2026-04-08): LLM-as-judge a test oracle problem.

Udržitelnost a kvalitativní hodnocení.

Metriky jako cyklomatická složitost nebo počet lint varování kvantifikují konkrétní vlastnosti kódu, ale nezachycují dimenze jako čitelnost, pojmenování, oddělení zodpovědností nebo idiomatické použití jazyka. Tyto vlastnosti souhrnně označované jako *udržovatelnost* (*maintainability*) ovlivňují schopnost kód dále rozvíjet a opravovat (Sommerville, 2016). Jejich hodnocení vyžaduje posouzení, které přesahuje možnosti deterministických nástrojů.

Jedním z přístupů je využití velkého jazykového modelu jako hodnotitele (*LLM-as-judge*). Zheng et al. (Zheng et al., 2023) ukázali, že LLM dokáže hodnotit kvalitu výstupů na úrovni srovnatelné s lidskými hodnotiteli, ale identifikovali tři systematické biasy: *position bias* (preferuje odpověď na určité pozici), *verbosity bias* (preferuje delší odpovědi) a *self-enhancement bias* (preferuje vlastní výstupy). Panickssery et al. (Panickssery et al., 2024) na NeurIPS 2024 prokázali korelaci mezi schopností modelu rozpoznat vlastní výstupy a mírou, do jaké je preferuje, což naznačuje kauzální vztah mezi self-recognition a self-preference. Verga et al. (Verga et al., 2024) navrhli mitigaci formou panelu diverzních modelů (PoLL), který dosahuje lepší shody s lidským hodnocením než jednotlivý model. Praktickou implikací je volba hodnotícího modelu z odlišné modelové rodiny, než ve které pracuje hodnocený agent.

Test oracle problem.

S testováním souvisí *test oracle problem*: kdo definuje správný výsledek? Mathews et al. (Mathews & Nagappan, 2024) zjistili, že nástroje pro automatické generování testů systematicky filtrují selhávající testy; až 68,1 % výsledných testovacích sad validuje chybné chování místo jeho odhalení. Chen et al. (J. Chen et al., 2025) na 500 úlohách benchmarku SWE-bench Verified potvrdili, že agent-generované testy slouží primárně jako observační zpětná vazba, nikoli jako validační nástroj. Obranou proti tomuto problému je odvozování očekávaných hodnot ze specifikace, nikoli z pozorování kódu.

[RAW]

Co tato sekce musí říct (aby validovala kap03 metodiku):

1. Problém: instrukce pro agenty se liší od jednoduchých NL promptů. Jsou strukturované dokumenty (sekce, workflow, constraints). Jedna iterace = desítky minut + tisíce tokenů; nelze iterovat rychle. Cílem není maximalizovat accuracy na test setu, ale řídit chování agenta přes více dimenzí (P/Q/E).

2. APO přístupy ((Agarwal et al., 2024), („Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation“, 2025)): Score → Critique → Synthesize → opakuj. Fungují pro jednoduché NL tasky. Předpoklady které pro nás neplatí: (a) atomický prompt, ne strukturovaný dokument; (b) rychlé iterace (API call), ne full agent run; (c) jeden cíl (accuracy), ne P/Q/E; (d) fix = vygeneruj novou verzi; my potřebujeme vědět PROČ selhalo.

3. Zhu et al. (Zhu et al., 2026): instrukce jsou nezávislá proměnná. Výsledky agenta nelze interpretovat bez kontroly prostředí. Potvrzuje naše fixní proměnné (empty repo, stejná spec, stejný model, build.md).

4. Gap: Neexistuje zavedená metodika pro iterativní, theory-grounded návrh instrukcí pro coding agenty. Existuje: praktický hill climbing (Cline blog, +10pp bez akademického popisu), APO pro jednoduché tasky. Chybí: systematický postup s diagnostikou příčin a citacemi z literatury.

→ Odtud plyne, proč v kap03 děláme Diagnózu manuálně (FSE/SASE/Lulla/Breunig), a ne automaticky.

TODO (2026-03-25): pojmenovat náš přístup vůči APO v DRAFT textu: Naše metodika = human-in-the-loop iterativní optimalizace instrukcí. APO (Zhou et al. 2022, (Agarwal et al., 2024)) = automatická smyčka (score→critique→generate). Klíčové rozdíly proč APO nestačí pro náš kontext viz bod 2 výše. Zvážit: zmínit že průmysl (MiniMax M2.7, DSPy) dělá plnou automatizaci; náš přístup je manuální předstupeň který explicitně dokumentuje kauzalitu. Citace: Zhou et al. ICLR 2023 (APE, arXiv 2211.01910); stáhnout a přidat do sources.

Fine-tuning vs in-context learning:

Dva přístupy jak přizpůsobit LLM konkrétnímu úkolu:

- **Fine-tuning:** trénování parametrů modelu. Permanentní, drahé.
- **In-context learning (ICL):** instrukce v kontextovém okně. Ephemeral, levné.

Shin et al. (Shin et al., 2025) ukazují, že ani jeden přístup systematicky nedominuje, ale explicitní instrukce s kontextem výrazně zlepšují výkon agenta na úlohách s kódem.

Prompt evaluation nástroje: Langfuse, Braintrust, Promptfoo: měří kvalitu odpovědí LLM, ale nehodnotí proces coding agenta.

Cline hill climbing: Praktický příklad iterativního zlepšování instrukcí (47% → 57% na Terminal Bench). Totožný přístup s naším, ale bez akademického popisu a měření procesu.

Empirická evidence: fungují context files?

→ gloaguen2025agentsmd, Evaluating AGENTS.md (ETH Zurich): LLM-generované context files snižují úspěšnost, developer-written jen marginálně zlepšují. Instrukce agent následuje, ale to nezlepšuje výsledky. Redundantní s dokumentací.

→ skillsbench2025, SkillsBench: Curated Skills +16,2pp. Self-generated nulový efekt. 2–3 fokusované moduly optimální.

Syntéza: Zbytečný kontext škodí, fokusovaný pomáhá.

[RAW]

Audit trail: zrušené sekce při restrukturalizaci (2026-03-01): Kompletní původní obsah zachován pro případ potřeby.

Narativní flow (revidovaný):

1. Složitost systémů roste – software je čím dál komplexnější

→ sommerville2016 s. 582:

“The root cause of these problems is, as it was in the 1960s, that we are trying to build systems that are larger and more complex than before. We are attempting to build these ‘mega-systems’ using methods and technology that were never designed for this purpose.”

2. Abstrakce jako nástroj – reakce na složitost:

- Assembler → C → Java → frameworky
- Každá vrstva skrývá detaily (information hiding)

→ colburn2000 s. 1:

“Abstraction through information hiding is a primary factor in computer science progress and success through an examination of the ubiquitous role of information hiding in programming languages, operating systems, network architecture, and design patterns.”

- Dijkstra: abstrakce pomáhá být přesný, ne vágní

→ swebok2024 s. 370

3. Přesto 1968 krize – proč?

- Složitost rostla rychleji než naše schopnost ji zvládat
- NATO konference: “software crisis”

→ nato1968 s. 78 (Perlis keynote):

“I believe it is because we recognize that a practical problem of considerable difficulty and importance has arisen: The successful design, production and maintenance of useful software systems.”

4. Reakce: vznik SWE – disciplína pro řízení složitosti

→ sommerville2016 s. 592:

“In software engineering, we have seen the incredibly rapid development of the discipline to help manage the increasing size and complexity of software systems... the approach that has been the basis of complexity management in software engineering is called reductionism.”

5. Dnes: AI jako nový problém

- Není to jen další vrstva abstrakce
- Je to ztráta determinismu – “black box”
- Proto mechanistic interpretability

→ bereska2024 s. 1:

“Understanding AI systems’ inner workings is critical for ensuring value alignment and safety.”

Klíčový insight: Abstrakce $\neq$ složitost. Abstrakce je NÁSTROJ pro zvládání složitosti. AI přináší kvalitativně nový problém (nedeterminismus), ne jen “více abstrakce”.

1. SWE je fundamentálně týmová disciplína

Brooks’s Law – komunikační overhead:

→ brooks1975 s. 25:

“Adding manpower to a late software project makes it later.”

→ mcconnell2004 s. 713:

“Merely increasing the number of people increases the complexity and amount of project communication.”

Komunikační kanály rostou: $n(n - 1)/2$

2. Role a zodpovědnosti

Tradiční role: Developer, Tester/QA, Architekt, Project Manager, Product Owner.

Role v Scrum:

→ scrumguide2020: Product Owner, Scrum Master, Developers.

3. Conway’s Law – organizace ↔ architektura

→ conway1968:

“Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations.”

4. Koordinace práce

→ malone1994:

Coordination = managing dependencies between activities.

Typy závislostí: shared resources, producer-consumer, simultaneity constraints.

Tacit vs explicit knowledge:

→ nonaka1995:

“There are two types of knowledge: explicit knowledge, contained in manuals and procedures, and tacit knowledge, learned only by experience, and communicated only indirectly.”

- **Tacit knowledge** – v hlavách lidí, těžko artikulovatelné
 - **Explicit knowledge** – dokumentace, procesy, kód
 - Konverze tacit → explicit = klíčový proces (externalization)
-

Quality gates pro agenty:

- Agent přebírá některé role (developer, částečně tester)
- Quality gates zůstávají – kdo je teď dělá?
- Implicitní znalosti z meetingů → explicitní instrukce pro agenta

1. Artefakty podle fází:

- **Specification** → Requirements, user stories, use cases, behavioral models
Spektrum formátů: Formální (IEEE 830), Structured (šablony), Semi-structured (GitHub Issues), Neformální (konverzace)
V open source: issue trackery jako de facto requirements management (Scacchi, 2002)
V agilních týmech: user stories místo SRS (Cao & Ramesh, 2008)
- **Design** → Architektura, design docs, diagramy, API kontrakty
- **Implementation** → Zdrojový kód, konfigurace
- **Validation** → Testy (unit, integration, e2e), test reports
- **Evolution** → Change requests, release notes, patches

Traceability:

→ gotel1994:

“Requirements traceability refers to the ability to describe and follow the life of a requirement, in both a forwards and backwards direction.”

Propojení: Requirement → Design → Code → Test.

2. Nástroje podle činností:

- **Vývoj:** IDE (VS Code, IntelliJ), editory, debuggery
 - **Verzování:** Git – distribuovaný version control
→ chacon2014: “everything as code”
 - **Koordinace:** Issue tracking (Jira, GitHub Issues)
 - **Kvalita:** CI/CD, test frameworks, linters
→ humble2010: automatizace buildů, testů, deploymentu
→ forsgren2018: DevOps praktiky a měřitelný dopad
 - **Dokumentace:** Wikis, Markdown, generátory dokumentace
-

3. Komunikační nástroje:

→ schmidt1992 – CSCW:

“Articulation work” – práce potřebná ke koordinaci spolupráce.

Synchronní: Slack, Teams, meetings. Asynchronní: Email, code review, dokumentace.

4. Standardy a formáty: UML (Fowler, 2004), OpenAPI/Swagger, Markdown, JSON Schema, Conventional Commits.

Zdroje zrušených sekcí:

nato1968, bereska2024, brooks1975, mcconnell2004,
conway1968, malone1994, nonaka1995, scrumguide2020,
gotel1994, chacon2014, humble2010, forsgren2018,
schmidt1992, fowler2004uml, scacchi2002, cao2008.

3. Metodika

Před definicí konkrétních metrik a experimentálního designu je třeba oddělit tři roviny, na nichž tato práce pracuje, protože se snadno zaměňují. *Systém upomínek faktur* slouží jako testovací prostředí, na kterém agenta pozorujeme; není předmětem zkoumání. *Chování agenta*, tedy jak implementuje, jak strukturuje práci a jaký kód vytváří, je to, co měříme. *Instrukce* v souboru `AGENTS.md` jsou nezávislou proměnnou, kterou mezi běhy měníme. Následující metodika definuje, jak tyto roviny měřit odděleně a jak výsledky mezi běhy porovnávat.

3.1 Výzkumný přístup

Kapitola ?? vymezila problém, že pass/fail benchmarky nepostačují pro posouzení praktické použitelnosti AI coding agenta, a kapitola ?? ukázala, proč je vedle funkční korektnosti nutné sledovat i proces, kvalitu kódu a efektivitu a proč jsou instrukce smysluplnou nezávislou proměnnou. Tato kapitola tato východiska převádí do konkrétní metodiky: definuje sadu metrik, experimentální design a způsob, jak jsou výsledky mezi běhy porovnávány.

Navrhujeme sadu metrik a iterativní postup pro hodnocení AI coding agentů. Ověřujeme je na případové studii systému upomínek faktur: opakovaně pouštíme agenta s různými variantami instrukcí a měříme, jak se mění jeho proces, výstup a náklady. Tento přístup odpovídá iterativnímu cyklu návrhu a vyhodnocení (Hevner et al., 2004; Peffers et al., 2008), ale v našem kontextu neslouží k silné statistické izolaci kauzality. Jeho cílem je ukázat, zda je navržený metrický rámec a postup použitelný a jaké typy změn v chování agenta dokáže odhalit.

V této práci realizujeme cyklus čtyřmi kroky: Spuštění, Měření, Diagnóza a Úprava. Podrobný popis kroků uvádí sekce ???. Postup má dvě fáze: pilotní fázi, v níž cyklus opakujeme dokud nedosáhneme stabilního zlepšení a jednoho úspěšného běhu podle zvolených kritérií, a komparativní variace, v níž z fungující verze instrukcí systematicky odebíráme jednotlivé části a měříme dopad na chování agenta (sekce ??).

Pro ověření jsme zvolili případovou studii na jednom projektu. Řízený experiment s větším počtem běhů by byl vzhledem k ceně jedné iterace (tisíce tokenů, desítky minut) nepraktický. Případová studie naopak umožňuje opakované běhy s různými instrukcemi a podrobnou analýzu každého z nich. Z jednoho projektu nelze statisticky generalizovat. Yin (R. K. Yin, 2018) pro tento typ výzkumu používá pojem analytická generalizace: na jednom případě ukazujeme princip, který lze dále ověřovat na dalších projektech. Jde o embedded single-case design: zkoumáme jeden projekt, ale v rámci něj více běhů agenta, z nichž každý má vlastní verzi instrukcí, git historii a sadu naměřených metrik.

[RAW]

Audit trail: předchozí verze 3.1 (DRAFT v3, DSR jako rámeček):

Sekce se jmenovala “Návrhový výzkum (DSR)”. Text stavěl na Hevner et al. (Hevner et al., 2004) (iterativní cyklus návrhu a vyhodnocení) a Peffers et al. (Peffers et al., 2008) (6 aktivit: problém, cíle, návrh, demonstrace, vyhodnocení, komunikace). DSR terminologie (“artefakt”) přidávala komplexitu bez přidané hodnoty pro čtenáře; přejmenováno na “Výzkumný přístup”, Hevner/Peffers zachováni jako citace.

Audit trail: předchozí verze 3.1 (DRAFT v1):

Hevner et al. rozlišují behaviorální a návrhový výzkum. DSR navrhuje nové artefakty a ověřuje jestli řeší daný problém.

RAW TODO (2026-03-24): Původní plán počítal i se substitucí jako samostatnou fází, ale pilotní iterace substituční efekt demonstrovaly implicitně (r1 obecná pravidla → r3 konkrétní příkazy). Komparativní fáze proto obsahuje jen ablace. Při revizi sjednotit terminologii; v kap03.tex 3.3.5 i kap04 úvod.

3.2 Sada metrik

Každý experimentální běh probíhá tak, že agent dostane prázdné GitHub repo, specifikaci v Issue #1 a instrukce v `AGENTS.md`, a autonomně implementuje zadaný projekt. Výstup jednoho běhu (kód, testy, git historie) měříme sadou 19 metrik. Postup běhu a experimentální design popisuje sekce ??.

Sada metrik pokrývá tři otázky: jak agent pracoval, co vyrobil a za jakou cenu. Fenton a Bieman (Fenton & Bieman, 2014) klasifikují měřitelné entity v softwarovém inženýrství do tří kategorií: proces, produkt a zdroje. V kontextu AI agentů dospívají k obdobnému rozdělení Yin et al. (2025) (Z. Yin et al., 2025), kteří hodnotí agentní frameworky ve třech dimenzích: effectiveness (výsledek), efficiency (průběh práce) a overhead (spotřebované zdroje). Naše sada metrik sleduje stejnou strukturu a obsahuje 19 metrik ve třech kategoriích:

- P – proces (P1–P8)** Dodržel agent předepsaný postup? Měříme dodržování workflow pravidel z instrukcí (P1 (issues before code)–P5) a kvalitu procesních výstupů jako commit messages, issue a PR descriptions (P6 (commit message quality)–P8).
- Q – kvalita produktu (Q1–Q8)** Je kód kvalitní? Měříme funkční korektnost (Q1 (API contract match)–Q2), kvalitu testů (Q3 (mutation score)–Q4) a udržitelnost kódu (Q5 (lint warnings)–Q8).
- E – efektivita (E1–E3)** Za jakou cenu? Zaznamenáváme tokeny z exportu (E1 (vstup/výstup/cache tokeny): špička promptu na krok, součet výstupů a součet `cache.read+cache.write` přes kroky; vše v tisících vedle sebe), čas (E2 (trvání)) a stabilitu session (E3 (stabilita session)).

Výběr konkrétních metrik vychází z potřeb případové studie a dostupných nástrojů. Z osmi charakteristik ISO/IEC 25010 (sekce ??) operacionalizujeme v produktových metrikách dvě: *funkční vhodnost* (Q1–Q4: agent vytváří správné řešení) a *udržovatelnost* (Q5–Q8: agent vytváří kód vhodný k dalšímu pochopení a rozvoji). Ostatní charakteristiky (bezpečnost, výkonnostní efektivita, kompatibilita, použitelnost, spolehlivost, přenositelnost) by se na rozsahu jedné malé případové studie systému upomínek smysluplně měřit nedaly. Sada si neklade nárok na úplnost. Teoretické základy jednotlivých metrik (proč jsou validní, jaká je evidence) popisuje sekce ?. Tato sekce se soustředí na jejich operacionalizaci: jaký nástroj používáme, jaká data jsou vstupem a co přesně je výsledkem měření. Interpretace přínosu jednotlivých metrik patří až do kapitoly ?.

3.2.1 Procesní metriky (P1–P8)

Procesní metriky (Fenton & Bieman, 2014) měří *jak* agent pracuje. Instrukce v AGENTS.md definují strukturovaný vývojový postup: agent má odvozovat práci ze specifikace, organizovat ji přes issues a branches, psát testy před implementací a dokumentovat rozhodnutí v commit messages a PR descriptions.

P1–P5: compliance (binární)

P1 (issues before code) ověřuje, zda agent vytvořil issues s timestampem před prvním kódovým commitem. Měříme porovnáním `created_at` nejstaršího agentova issue (GitHub API) s časem prvního commitu obsahujícího soubory v `src/` nebo `tests/`.

P2 (branch per issue) ověřuje, zda agent vytvořil pro každé issue vlastní větev. Měříme počet remote branches (bez `main`) vůči počtu issues: podmínka $branches \geq issues$ je splněna tehdy, nenastalo-li slučování více issues do jedné větve.

P3 (test-first commits) ověřuje, zda agent psal testy před implementací. Primární indikátor: existuje alespoň jeden commit s prefixem `test:` a zároveň alespoň jeden `feat:`. Přesnější měření poskytuje behavioral trace (sekce ??): počet větví, kde první zápis do `src/` předcházela prvnímu zápisu do `tests/` (`tddOrderViolations`).

P4 (PRs linked to issues) ověřuje, zda každý PR obsahuje odkaz na issue. GitHub API vrací tělo PR. Podmínka: všechna PR těla obsahují regex `Closes #N`.

P5 (no existing test modifications) ověřuje, že agent nepřepisoval již existující testové soubory. Metrika zachycuje `modified` změny (`--diff-filter=M`) na testovacích souborech, tedy situaci kdy agent dříve vytvořený test později upraví místo toho, aby opravil implementaci. Nově přidané testy se do P5 nezapočítávají.

P6–P8: kvalita procesních artefaktů (LLM-as-judge)

P6 (commit message quality) hodnotí popisnost commit messages: atomicitu, konvenční prefix a srozumitelnost co a proč bylo změněno. Hodnocení provádí LLM-as-judge na škále 1–3 (sekce ??).

P7 (issue description quality) hodnotí popisnost issue descriptions: jasnost scope, přítomnost acceptance criteria, dostatečnost pro implementaci. Hodnocení provádí LLM-as-judge na škále 1–3.

P8 (PR description quality) hodnotí popisnost PR descriptions: přítomnost odkazu na issue, popis co a proč, dostatečnost pro code review. Hodnocení provádí LLM-as-judge na škále 1–3.

[RAW]

Audit trail: předchozí obsah 3.2.1: Procesní cíle (spec-first, strukturovaný postup, TDD, transparentnost) a původní popis metrik P1/P2 včetně exit kritérií (P1=5/5, P2=3/3). Exit kritéria přesunuta do tabulky ?? a sekce ??.

V refactoru P metrik (2026-03-04): P1 byl binární checklist 5 položek, P2 byl LLM-as-judge se 3 dimenzemi a gating rules. Nyní: P1–P5 jsou samostatné binární metriky (bez souhrnného skóre), P6–P8 jsou samostatné LLM-as-judge metriky bez hard gates (P3 jako binární metrika nahrazuje gating rule pro test: commit).

3.2.2 Produktové metriky (Q1–Q8)

Produktové metriky (Fenton & Bieman, 2014) měří *co* agent vyrobil. Pokrývají tři oblasti: funguje implementace správně (Q1–Q2), detekují agentovy testy skutečné chyby (Q3–Q4) a je kód udržitelný (Q5–Q8).

Funkční korektnost (Q1–Q2)

Q1 (API contract match) ověřuje, zda agentův kód dodržuje definované rozhraní. Referenční typy se importují a zkompilují proti agentovu kódu (`tsc`). Výsledek je binární: typy sedí, nebo ne. Q1 je vstupní podmínkou pro Q2: pokud agentův kód neimplementuje správné API, referenční testy nelze ani zkompilovat, a výsledek Q2 by byl nesmyslný.

Q2 (referenční test pass rate) měří kolik z referenčních testů projde na agentově implementaci. Vitest spustí 42 testů proti agentovu kódu. Výsledek je počet passing testů z 42. Testy ověřují chování přes veřejné API (black-box, sekce ??), takže je lze spustit na libovolném běhu nezávisle na interní struktuře. Konstrukce referenční test suite popisuje sekce ??.

Kvalita testů (Q3–Q4)

Q3 (mutation score) měří, jestli agentovy testy skutečně detekují chyby (sekce ??). Stryker s konfigurací `--mutate 'src/**/*.ts'` a mutátory pro TypeScript (conditional, arithmetic, string, logical operators) systematicky zavádí drobné změny do zdrojového kódu. Vstup: agentovy testy + zdrojový kód. Výstup: procento zabitých mutantů z celkového počtu.

Q4 (AC coverage) měří, kolik akceptačních kritérií ze specifikace má odpovídající test v agentově test suite. Na rozdíl od Q2 (funguje implementace?) se Q4 ptá, jestli agent *testoval všechno co měl*. Specifikace obsahuje 25 acceptance criteria. Historicky uložené judge běhy byly exportovány ve 24bodovém formátu. Při finalizaci práce byl chybějící bod AC25 (*custom holiday calendar*) dopočítán manuálně. V textu proto Q4 uvádíme na škále N/25. Hodnocení provádí LLM-as-judge (sekce ??) doplněné o toto manuální ověření.

Kvalita kódu (Q5–Q8)

Q5 (lint warnings) počítá varování a chyby z ESLint s fixní konfigurací (pravidla: recommended + strict TypeScript rules). Konfigurace je součástí experimentální infrastruktury, ne agentova repo. Agent ji nemůže měnit. Výstup: celkový počet warnings + errors.

Q6 (typecheck errors) počítá chyby z `tsc --noEmit` ve strict mode. Doplnuje počet explicitních `any` ve zdrojových souborech (grep v `src/**/*.ts`) jako proxy pro obcházení typového systému.

Q7 (složitost kódu) měří cyklomatickou složitost per funkce. ESLint `complexity` rule reportuje funkce překračující nastavený práh. Výstup: maximální složitost across all functions. Práh ≤ 10 per funkce vychází z McCabe (sekce ??).

Q8 (design quality) hodnotí aspekty které automatizované nástroje nezachytí: pojmenování, oddělení zodpovědností, idiomatický TypeScript, kvalita dokumentace a zbytečná komplexita. Hodnocení provádí LLM-as-judge (sekce ??). Celkové skóre Q8 je *minimum* pěti dimenzí, nikoliv průměr. Volba minima je záměrná: jeden slabý rozměr má stáhnout celkový výsledek dolů, aby slabiny nebyly maskované silnými dimenzemi.

[RAW]

Audit trail: předchozí obsah 3.2.2: Produktové cíle (funkční korektnost, kompatibilní API, kvalitní testy, kvalitní kód) a původní popis metrik Q1–Q8 včetně exit kritérií. Exit kritéria přesunuta do tabulky ?? a sekce ??.

3.2.3 Metriky efektivity (E1–E3)

[RAW]

Audit trail (3.2.3, 2026-04-22): Původní TODO požadoval zkrátit příliš technický popis E1. Finální text ponechává jen měřenou veličinu a důvod použití; detailní zdroje dat a skripty popisuje sekce ??.

Metriky efektivity měří zdroje spotřebované při vývoji. Nemají exit kritérium a slouží k porovnání nákladů mezi iteracemi.

E1 (vstup/výstup/cache tokeny) zaznamenává tři komplementární hodnoty z exportu `transcript.json`: maximální vstup na jednom kroku včetně cache, součet výstupních tokenů a součet cache tokenů přes kroky. Všechny tři složky uvádíme v tisících. Toto rozdělení umožňuje odlišit šířku promptu, objem generování a cenu opakovaného čtení kontextu. Sledování objemu tokenů motivuje zjištění Gloaguen et al. (Gloaguen et al., 2025), že context files zvyšují inference cost o více než 20 %.

E2 (trvání) měří wall-clock time v minutách od spuštění agenta po poslední commit.

E3 (stabilita session) sleduje stabilitu session ve třech signálech: zda běh dokončil, kolik kompakcí kontextu nastalo a kolikrát auto-continue plugin session restartoval. Počet kompakcí odvozujeme heuristicky z exportu a počet restartů z `metrics.csv`. Na této úloze jde o pomocnou záznamovou metriku, která má zachytit zjevné kolapsy kontinuity práce mezi kroky.

Se dvěma běhy per variaci jsou tyto hodnoty deskriptivní, ne inferenční.

[RAW]

Audit trail: předchozí obsah 3.2.3: Původní popis E1–E3 se zdroji (SWE-bench, Agent-Bench).

3.2.4 Aplikace LLM-as-judge

Pět metrik (P6, P7, P8, Q4, Q8) hodnotí vlastnosti které nelze extrahovat automatizovaným nástrojem. Pro jejich hodnocení používáme metodu LLM-as-judge (sekce ??).

Volba modelu. Jako judge používáme GLM-5 (Zhipu AI). Agentní běhy provádí model MiniMax-M2.5; judge je tedy z jiné modelové rodiny. Důvod: model má tendenci hodnotit výstupy svého vlastního druhu lépe, i když jsou objektivně slabší (self-preference bias (Pannickssery et al., 2024)). Volba judge z jiné modelové rodiny než agent tento bias snižuje (Verga et al., 2024).

Škála. Hodnocení probíhá na škále 1–3 per dimenze: 1 = nevyhovující, 2 = přijatelné, 3 = dobré. Jemnější škály (1–5, 1–10) produkují při malém počtu hodnocených artefaktů nižší shodu mezi hodnotiteli (Zheng et al., 2023); třístupňová škála je pro tento rozsah spolehlivější.

Rubrika. Judge dostane artefakty agentova běhu (zdrojový kód, commit messages, issue a PR

descriptions) spolu s rubrikou obsahující popis každého stupně. Dimenze per metriku jsou popsány při definici metrik (P6–P8 v sekci ??, Q4 a Q8 v sekci ??). Kompletní formulace rubrik a promptu je v příloze. Prompt je fixní napříč všemi běhy, aby hodnocení bylo srovnatelné mezi iteracemi.

Interpretace výsledků. LLM-as-judge zde neslouží jako plně objektivní měření, ale jako strukturované posouzení vlastností, které nelze spolehlivě vyhodnotit deterministickým skriptem. Výsledky těchto metrik proto interpretujeme jako podpůrné kvalitativní indikátory a ne jako hlavní důkazní osu případové studie.

[RAW]

Audit trail: LLM-as-judge obsah přesunut do kap02. Teorie (Zheng 2023, Panickssery 2024, Verga 2024 PoLL) v kap02. Aplikace (GLM-5, škála, rubriky, interpretace výsledků) zde v kap03.

3.3 Experimentální design

3.3.1 Výběr projektu

Případová studie potřebuje projekt, na kterém lze opakovaně spouštět agenta s různými instrukcemi a objektivně měřit výsledky. Projekt musí mít deterministickou logiku (stejný vstup vždy dá stejný výstup), aby bylo možné ověřit korektnost automatizovanými testy a mutation testingem. Zároveň musí být dostatečně malý pro více experimentálních běhů, ale obsahovat reálné nuance (hraniční případy, doménová pravidla), aby měl agent co řešit.

Zvolili jsme systém upomínek faktur: systém pro automatické odesílání připomínek k nezaplaceným fakturám. Obsahuje stavový automat (nová, po splatnosti, upomínaná, eskalovaná), časové výpočty (pracovní dny, ochranné lhůty) a pravidla pro eskalaci. Specifikace definuje 25 acceptance criteria, API kontrakt (TypeScript typy a signatury) a doménový slovník. Systém zde neplní roli hlavního výsledku práce, ale testovacího prostředí, na němž lze měřit chování agenta. Tato kombinace deterministické logiky a hraničních případů (víkendy, svátky, grace periods) umožňuje objektivně testovat jak funkční korektnost (Q1, Q2), tak kvalitu testů (Q3, Q4).

Současně jde o úlohu, která je dostatečně bohatá na netriviální rozhodování, ale stále zvládnutelná pro opakované běhy. Agent musí správně kombinovat stavové přechody, časové výpočty a testování hraničních případů, takže metriky zachycují více než jen průchod jednoduchou CRUD implementací.

[RAW]

Audit trail (3.3.1, 2026-04-22): Původní TODO požadoval rozepsat projekt podrobněji. Finální verze ponechává jen vlastnosti, které nesou metodické zdůvodnění výběru: determinis-

tickou logiku, hraniční případy a přiměřenou velikost pro opakované běhy.

3.3.2 Fixní proměnné

Aby bylo možné měřit vliv instrukcí, musí být všechno ostatní co nejstabilnější. Hlavní měněnou proměnnou mezi běhy je obsah souboru `AGENTS.md`, tedy procedurálních instrukcí, které definují pracovní postup, omezení a quality gates. Tento design negarantuje dokonalou izolaci všech vlivů; jeho cílem je omezit zjevné vedlejší vstupy a udělat jejich případný vliv auditovatelný. Na rozdíl od generických context files, které typicky popisují adresářovou strukturu a build příkazy (Gloaguen et al., 2025), jde o fokusované instrukce s konkrétními procedurami (X. Li et al., 2025).

Fixní proměnné jsou:

Prázdné GitHub repo obsahuje pouze `AGENTS.md`, konfiguraci agenta a auto-continue plugin. Žádný existující kód, testy ani `package.json`. Agent musí inicializovat projekt a zvolit strukturu sám.

Specifikace v GitHub Issue #1 obsahuje 25 acceptance criteria, API kontrakt (TypeScript typy a signatury), doménový slovník a out of scope. Je to hlavní věcný vstup, který agent dostane kromě instrukcí.

Auto-continue plugin je hook který detekuje kdy se agent zastaví a automaticky ho restartuje. Počítadlo restartů a kontrola otevřených issues zajišťují běh bez manuálního zásahu. Metrika E3 z toho čerpá.

Model MiniMax-M2.5 byl zvolen jako model schopný autonomně dokončit zadaný úkol. Cílem práce je vyhodnotit sadu metrik a iterativní postup, ne srovnávat výkon modelů — volba konkrétního modelu je v tomto ohledu sekundární. MiniMax-M2.5 je zároveň z jiné modelové rodiny než judge (GLM-5, Zhipu AI), čímž se eliminuje self-preference bias (sekce ??). Teplota modelu nebyla mezi běhy explicitně nastavena. Agent běžel s výchozí konfigurací poskytovatele. Dopad této volby na variabilitu výsledků diskutuje sekce ??.

Běhové prostředí (Docker container) zajišťuje, že agent nemá přístup ke globální konfiguraci nástroje, MCP serverům ani rozšířením nainstalovaným na hostitelském systému. Izolace tím významně omezuje vliv vedlejších vstupů z prostředí, ale nevyklučuje všechny zdroje variability. Sdílený autentizační svazek zajišťuje konzistentní přístup k API napříč běhy.

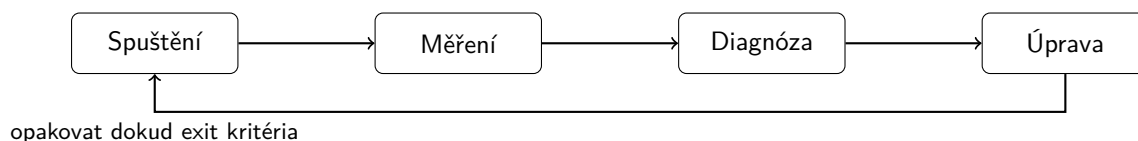
System prompt agenta (`build.md`) nahrazuje výchozí system prompt nástroje OpenCode, jehož instrukce o verzování konfliktovaly s procesními požadavky případové studie. Vlastní system prompt obsahuje pouze obecné kódové konvence (styl, bezpečnost, zákaz komentářů) a odkaz na `AGENTS.md`. Veškeré procesní instrukce jsou výhradně v `AGENTS.md`, aby bylo možné změny procesního chování přisuzovat této vrstvě obhajitelněji než při jejich rozptýlení mezi více vstupy.

[RAW]

Audit trail: předchozí obsah 3.3.2: Popis fixních proměnných včetně detailů o auto-continue pluginu (`session.idle hook`) a system promptu (`build.md, mode: primary`).

3.3.3 Iterativní cyklus

Pilotní fáze postupuje opakováním čtyřkrokového cyklu, dokud agent nesplní exit kritéria. Každý průchod cyklem je jedna experimentální iterace — vstupem je aktuální verze `AGENTS.md`, výstupem upravená verze pro příští běh.



Spuštění. Skript `new-run.ts` připraví izolované prostředí: vytvoří prázdné GitHub repo, zkopíruje aktuální `AGENTS.md`, vytvoří Issue #1 se specifikací a spustí agenta v Docker kontejneru. Agent dostane výhradně prázdné repo, instrukce a specifikaci — žádný existující kód ani závislosti. Po dokončení se exportuje session transcript (`transcript.json`).

Měření. Skript `analyze-run.ts` extrahuje deterministické metriky automaticky: git log a GitHub API (P1), výstup Vitest a Stryker (Q2, Q3), statická analýza (Q5–Q7), session metadata (E1–E3). Kvalitativní metriky (P6, P7, P8, Q4, Q8) hodnotí LLM-as-judge GLM-5 podle fixní rubriky (sekce ??). Výstupem je `FINDINGS.md` s tabulkou metrik a faktickým popisem chování agenta.

Diagnóza. Z naměřených hodnot a behaviorálního popisu se identifikuje kde a proč agent nedodržel očekávané chování. Analýza postupuje podle čtyř rámců:

- Mao et al. (Mao et al., 2025) — která složka instrukcí selhala a proč
- Hassan et al. (Hassan et al., 2025) — zda obsah vyvažuje proceduru, kontext a omezení
- Lulla et al. (Lulla et al., 2026) — zda každý řádek přidává hodnotu
- Razavi, Breunig (Breunig, 2025; Razavi & Fard, 2025) — proč agent instrukci opakovaně ignoruje

Diagnóza probíhá ve třech navazujících krocích. Nejprve se z tabulky metrik vyberou nesplněné nebo hraniční hodnoty. Poté se pro každou z nich dohledají faktické projevy ve `FINDINGS.md` a v odpovídajících artefaktech (git log, issues, PR, transcript, výstupy nástrojů), aby bylo možné odlišit jednorázové selhání běhu od opakujeícího se problému instrukcí. Nakonec se příčina klasifikuje jako chybějící instrukce, příliš obecná instrukce, konflikt mezi instrukcemi nebo instrukce ignorovaná bez verifikačního kroku. Z této klasifikace plyne typ opravy (doplnění, zpřesnění, přestrukturování nebo přidání verifikačního kroku). Pořadí aplikace rámců se řídí právě touto klasifikací: chybějící nebo obecná složka vede k analýze komponent podle Mao et

al., konflikt mezi typy obsahu k posouzení balance podle Hassan et al., opakovaně ignorovaná instrukce k Razavi a Breunig (proč nepůsobí a jak přestrukturovat). Audit redundance podle Lulla et al. slouží napříč jako kontrola, zda každý řádek instrukcí nese hodnotu.

V tomto kroku byl průběžně využíván nástroj Claude (Anthropic) jako analytický asistent: výzkumník formuloval hypotézy o příčinách selhání a AI asistent navrhoval literaturou podložené interpretace a varianty úprav. Finální rozhodnutí o každé změně `AGENTS.md` však vždy činil výzkumník.

Výstupem je `DIAGNOSIS.md` s identifikovanými příčinami a návrhem změn.

Úprava. Na základě diagnózy se upraví `AGENTS.md`. Každá změna odpovídá jednomu identifikovanému problému, je podložena citací a zaznamenána v changelogu — aby bylo zpětně dohledatelné proč daná úprava vznikla a co měla opravit. Přestrukturování instrukcí je preferováno před jejich rozšiřováním: redundantní obsah zvyšuje inference cost bez přínosu k úspěšnosti (Gloaguen et al., 2025) a fokusované instrukce překonávají vyčerpávající dokumentaci (X. Li et al., 2025).

[RAW]

Audit trail (Diagnóza, 2026-04-22): Původní TODO požadovala dvě věci: rozepsat konkrétní postup diagnostiky a transparentně doplnit AI asistenci v tomto kroku. Obojí je nyní zapracováno přímo do finálního textu; detailnější disclosure zůstává v příloze ??.

Doporučení k validaci (strukturální audit, 2026-04-22): Ověřit, zda finální podoba části Diagnóza nemíchá příliš mnoho vrstev: reprodukovatelný protokol, interpretační rámce z literatury a disclosure o použití Claude. Pokud ano, přestavět tok na čisté “vstup → analytický krok → výstup” a rámce/disclosure nechat jen jako krátké opory. Riziko současného stavu: metodická sekce není dostatečně čitelná jako protokol.

[RAW]

Review poznámka (2026-03-21): Hassan SASE jako diagnostický rámec: Hassan et al. (Hassan et al., 2025) je vision paper, ne empirická studie. BriefingScript/LoopScript/-MentorScript taxonomie je konceptuální nástroj, ne empiricky validovaný poměr. V textu používáme jako analytický rámec pro klasifikaci obsahu instrukcí, ne jako evidence “co funguje”. Ujistit se že v kap02 i kap03 je toto jasné; nepřipisovat Hassanovi empirické výsledky.

Rozepsat každý ze čtyř kroků podrobněji: co přesně se děje, jaké nástroje, jaké výstupy.

Spuštění: operacionalizace:

Vstup kroku: aktuální verze `AGENTS.md` (jediná měnící se proměnná).

Skript `new-run.ts` automatizuje přípravu prostředí: (1) vytvoří prázdné GitHub repo s názvem `bp-billing-reminder-pilot-rN`, (2) zkopíruje aktuální `AGENTS.md` a `build.md` (system

prompt override), (3) vytvoří Issue #1 se specifikací (25 AC, API kontrakt, doménový slovník), (4) spustí OpenCode s modelem MiniMax-M2.5 v neinteraktivním režimu.

Auto-continue plugin detekuje kdy se agent zastaví (`session.idle` hook) a automaticky ho restartuje. Počítadlo restartů a kontrola otevřených issues zajišťují doběhnutí bez manuálního zásahu.

Agent dostane výhradně: prázdné repo + `AGENTS.md` + `build.md` + Issue #1. Žádný existující kód, testy ani `package.json`.

Výstupy kroku: agentovo GitHub repo (kód, testy, git log, issues, PRs), session transcript (`transcript.json` přes `opencode export <sessionID>`).

Měření: operacionalizace:

Skript `analyze-run.ts` extrahuje metriky z pěti zdrojů:

- git log + GitHub API → P1 (timestamps issues vs. commity, branch names, PR body regex)
- Vitest výstup (`parse N passed`) → Q2
- Stryker JSON report → Q3 (mutation score)
- ESLint JSON report → Q5 (warnings), Q7 (complexity rule)
- `tsc --noEmit stderr` → Q6
- `transcript.json` → E1 (vstup / výstup / Σ cache v tisících), E2 (trvání), počet kompakcí pro E3; `metrics.csv` → restarty pro E3

Skript `judge.ts` spouští LLM-as-judge (GLM-5) pro P6, P7, P8, Q4 a Q8. Workflow: pošle artefakty + rubriku; výstup = JSON se score a rationale. P6: commit messages hodnocené samostatně 1–3 (atomicita, konvenční prefix, popisnost). P7: issue descriptions hodnocené samostatně 1–3 (scope, AC, detail pro implementaci). P8: PR descriptions hodnocené samostatně 1–3 (odkaz na issue, co a proč, dostatečnost pro review). Q4: judge dostane seznam 25 AC ze specifikace + agentovu test suite; partial coverage počítá jako pokryté. Q8: pět dimenzí (naming, SoC, idiomatic TS, documentation, complexity); overall = MINIMUM pěti dimenzí; jeden slabý score stáhne celkový.

Výstupy kroku: `FINDINGS.md` (tabulka P/Q/E metrik + behavioral summary). Soubory `p2-result.json`, `q4-result.json`, `q8-result.json`.

Behavioral summary (3–5 vět) popisuje co agent skutečně udělal: kolik issues vytvořil, zda TDD dodržel, zda commitoval průběžně nebo blob na konci. Faktická pozorování z git logu a GitHub API, ne hodnocení.

Měření, poznámka k duplikaci: Tabulka zdroj → metrika → nástroj by duplikovala sloupec Nástroj z přehledové tabulky ???. Alternativa: rozšířit přehledovou tabulku o sloupec *Zdroj dat* (git log, GitHub API, Vitest, Stryker atd.). Nebo jen odstavec textu: “P1 bere data z git logu a GitHub API, Q2–Q3 z Vitest/Stryker výstupu, E1–E3 z `transcript.json`.”

Diagnóza: operacionalizace 4 nástrojů:

Výstup FINDINGS.md identifikuje *co* nesplnil. DIAGNOSIS.md vysvětluje *proč* a navrhuje *fix*.

Přehled rámců jako tabulka; zvážit pro finální text:

Rámec	Otázka	Co analyzuje	Výstup
FSE/Mao (Mao et al., 2025)	Co chybí/přebývá?	7 komponent AGENTS.md	komponentní analýza
SASE/Hassan (Hassan et al., 2025)	Je balance typů obsahu ok?	Brief/Loop/Mentor poměr	poměr + co dominuje
Lulla (Lulla et al., 2026)	Přidává každý řádek hodnotu?	efektivita obsahu	kandidáti na odstranění
Breunig/Razavi (Breunig, 2025; Razavi & Fard, 2025)	Proč ignoruje opakovaně?	recovery strategie	4-krokový postup

FSE 2025 (Mao et al.): projít AGENTS.md proti 7 komponentám v doporučeném pořadí: Role → Directive → Context → Workflow → Output → Constraints → Examples. Finding 2: Directive musí být imperativní, ne otázky. Finding 4: Constraints = exclusion type nejčastější (46 % výskytů v korpusu); *Never X > Avoid X*. Finding 9: Instrukce AFTER context, ne before; information decay při opačném pořadí. Constraints seskupit do jedné sekce, ne rozházet po dokumentu.

SASE script balance (Hassan 2025): klasifikovat řádky AGENTS.md: BriefingScript = co + proč (cíl, kontext, success criteria); LoopScript = jak (workflow, kroky, iterace); MentorScript = co ne + error recovery (constraints, guardrails). Cílový poměr: přibližně vyrovnaný. LoopScript dominance ⇒ agent neumí reagovat na odchylky. Chybějící MentorScript ⇒ agent neví jak postupovat při chybě.

Lulla 2026: přítomnost AGENTS.md: −28,6 % mediánový runtime, −16,6 % výstupní tokeny. Paper nerozlišuje efektivitu jednotlivých typů obsahu (to je spekulace v sekci Research Roadmap). Číslo +14–22 % tokenů NENÍ z Lully. Filtr (AGENTbench meta-princip): “*Kdyby tento řádek chyběl, udělal by agent neočividnou chybu?*” Pokud ne, kandidát na odstranění.

Breunig / Razavi: pokud agent opakovaně ignoruje instrukci přes iterace: nepřidávat znovu, přestrukturovat. 4-krokový recovery: (1) jiný formát nebo jiná pozice v dokumentu, (2) rozložit na menší kroky; identifikovat přesně kde se odchyluje, (3) přidat validační krok; agent si sám ověří compliance, (4) negative example: ukázat co NEDĚLAT (Razavi 2025: targeted clarification).

Výstup kroku: DIAGNOSIS.md (FSE tabulka 7 komponent + SASE balance poměr + Lulla audit checklist + Breunig findings + návrh fixů s citacemi).

Úprava: operacionalizace:

Pravidla pro změny vycházejí z diagnostiky:

- Jedna změna = jeden identifikovaný problém (nekombinovat nesouvisející fixy).
- Každá změna musí mít citaci; žádné ad hoc “zkusíme tohle”.
- Preferovat přestrukturování před přidáváním; AGENTbench: víc textu ≠ lepší výsledky.
- Testovat každý nový řádek proti meta-principu (Kdyby chyběl ...?).

Formát záznamu per změna: bylo → je → pozorování (rN) → diagnóza (FSE/SASE/Lulla-/Breunig) → literatura.

Výstupy kroku: aktualizovaný AGENTS.md + záznam changelog/pilot-rN-to-rN+1.md.

3.3.4 Pilotní fáze

Pilotní fáze běží dokud agent nesplní exit kritéria ze sloupce *Exit kritérium* v tabulce ?? . Kritéria dělíme do tří skupin. Tvrdá deterministická kritéria (P1–P5, Q1, Q2, Q5, Q6) mají přirozenou hranici splněno/nesplněno. Prahové deterministické metriky (Q3, Q7) používají minimum odvozené z literatury a praxe. Judge-based metriky (P6–P8, Q4, Q8) slouží jako podpůrné kvalitativní indikátory. E1–E3 jsou záznamové metriky bez exit kritéria a slouží k porovnání efektivity mezi běhy.

S ohledem na scope práce (jeden projekt, jeden model) definujeme úspěšný běh jako dokončení bez manuálního zásahu, při kterém agent splnil všechna tvrdá deterministická kritéria (P1–P5, Q1, Q2, Q5, Q6) a překročil prahy prahových metrik ($Q3 \geq 70\%$, $Q7 \leq 10$ na funkci). Judge-based metriky u tohoto běhu nesmí současně klesnout pod baseline. Pilotní fáze končí, jakmile takový běh nastane a vykáže prokazatelné zlepšení oproti baseline. Tím se ukazuje proveditelnost postupu na jednom případě; neimplikuje to, že zvolené prahy obecně platí pro všechny agenty a projekty. Každá iterace produkuje aktualizovaný AGENTS.md, záznam změn se zdůvodněním a kompletní P/Q/E metriky.

[RAW]

Audit trail: předchozí obsah 3.3.3: Podrobný popis cyklu Spuštění/Měření/Diagnóza/-Úprava, diagnostický rámec (Mao component analysis, Hassan script balance, Lulla content effectiveness, Razavi/Breunig prompt sensitivity), výstupy iterace (AGENTS.md diff, CHANGelog, FINDINGS.md), exit kritéria.

3.3.5 Komparativní variace

Z fungující sady instrukcí (výstup pilotní fáze) systematicky odebíráme jednotlivé složky a měříme dopad na chování agenta. Tento postup, ablace, odpovídá na otázku: potřebuje agent danou část instrukcí, nebo je redundantní?

Konkrétní variace nelze specifikovat předem. Pilotní fáze *generuje* hypotézy o tom, které složky instrukcí jsou pro chování agenta podstatné, a komparativní fáze je testuje. Výběr variací vychází z diagnostiky pilotních běhů (sekce ??): pokud se v pilotu ukáže, že agent konzistentně nedodržuje určité pravidlo, ablace může testovat, zda jeho odebrání změni i ostatní metriky. Podobně pokud určitá sekce instrukcí koreluje s vysokou kvalitou výstupu, ablace ověří, zda je tento vztah kauzální.

Každá ablace se provádí ve dvou nezávislých bězích se stejným nastavením, aby bylo možné odlišit systematický efekt změny od přirozené variability nedeterministického modelu. I se dvěma běhy nelze dosáhnout statistické průkaznosti (sekce ??). Výsledky komparativní fáze proto interpretujeme jako indikativní, nikoliv kauzální.

[RAW]

Audit trail: předchozí obsah 3.3.5: DRAFT v1 zmiňoval ablace i substituce. Substituce v kap04 jako samostatná fáze neprobíhaly (pilotní iterace je demonstrovaly implicitně: r1 obecná pravidla → r3 konkrétní příkazy). DRAFT v2 zúžen jen na ablace v souladu s cílem 3 a kap04.

3.4 Přehledová tabulka

Tabulka ?? shrnuje všechny metriky na jednom místě. Sloupec *typ* rozlišuje deterministická exit kritéria (automatizované měření), judge-based metriky (rubrikové hodnocení LLM-as-judge) a záznamové metriky (bez kritéria, slouží k porovnání mezi běhy). Prahové požadavky jsou uvedeny ve sloupci *Exit kritérium*. Způsob sběru odpovídá rozlišení deterministické metriky, kvalitativní metriky a záznamové metriky zavedenému v kapitole ??.

Tabulka 3.1: Sada metrik: kód, měřená vlastnost, nástroj, exit kritérium a typ

Kód	Metrika	Nástroj	Exit kritérium	Typ
<i>Procesní (P): jak agent pracuje</i>				
P1	Issues before code	git, GitHub API	pass	deter.
P2	Branch per issue	git, GitHub API	pass	deter.
P3	Test-first commits	git log	pass	deter.
P4	PRs linked to issues	GitHub API	pass	deter.
P5	No existing test modifications	git diff	pass	deter.
P6	Commit message quality	LLM-as-judge (GLM-5)	$\geq 2/3$	judge
P7	Issue description quality	LLM-as-judge (GLM-5)	$\geq 2/3$	judge
P8	PR description quality	LLM-as-judge (GLM-5)	$\geq 2/3$	judge
<i>Produktové (Q): co agent vyrobil</i>				
Q1	API contract match	tsc (import + typecheck)	match	deter.
Q2	Referenční test pass rate	Vitest (42 testů)	42/42	deter.
Q3	Mutation score	Stryker	$\geq 70\%$	min.
Q4	AC coverage agentových testů (25 krit.)	LLM-as-judge (GLM-5)	25/25	judge
Q5	Lint warnings	ESLint	0	deter.
Q6	Typecheck errors	tsc --noEmit	0	deter.
Q7	Cykломatická složitost	ESLint (complexity)	$\leq 10/\text{fn}$	min.
Q8	Design quality	LLM-as-judge (GLM-5)	$\geq 2/3$	judge
<i>Efektivita (E): za jakou cenu</i>				
E1	Vstup / výstup / Σ cache (tis.) z exportu	OpenCode export	—	záznam
E2	Trvání (minuty)	session timestamps	—	záznam
E3	Stabilita session (kompakce + dokončení + restarty)	transcript + metrics.csv	—	záznam

3.5 Omezení a validita

Každý výzkumný design má omezení. Následující analýza vychází z kategorizace Runeson a Höst (Runeson & Höst, 2009).

Tato sekce deklaruje hrozby validity, které jsou dané designem studie a známe je předem. Jak se tyto hrozby skutečně projeví v naměřených datech, diskutuje sekce ??.

Konstruktová validita (měříme to, co chceme měřit?). Jednotlivé metriky vychází z existující teorie: mutation testing (Papadakis et al., 2019), cyklomatická složitost (McCabe, 1976), taxonomie procesních a produktových metrik (Fenton & Bieman, 2014). Konkrétní kombinace metrik do sady a volba exit kritérií jsou autorské a představují návrh, ne ověřený standard. Pět metrik (P6, P7, P8, Q4, Q8) obsahuje subjektivní složku (LLM-as-judge). Tyto metriky proto slouží jako podpůrné kvalitativní indikátory, zatímco hlavní opora vyhodnocení stojí na deterministických metrikách a auditovatelných artefaktech. Spolehlivost LLM-as-judge nebyla validována proti lidskému hodnocení (např. Cohenovým κ) z důvodu časových omezení. Případová studie rozlišuje dva druhy testů: referenční testy (měřené metrikou Q2, spouštěné po běhu skriptem, agentovi neviditelné) a agentovy vlastní testy (psané agentem během běhu, viditelné a spustitelné agentem). Metrika P5 se vztahuje výhradně k agentovým vlastním testům. Referenční testy jsou odvozeny ze specifikace metodou TDD (Mathews & Nagappan, 2024), čímž se snižuje riziko test oracle problému. Správnost skriptu `analyze-run.ts` pro metriky P1–P5 byla ověřena manuálním srovnáním výstupů s raw daty z git logu a GitHub API pro běhy r4 a r5. Žádná diskrepance nebyla identifikována.

Interní validita (jsou závěry podložené daty?). Hlavní hrozba je nedeterminismus LLM: stejné instrukce mohou při různých bězích dát různé výsledky (Razavi & Fard, 2025). Dva běhy per variaci jsou mitigací, ale neposkytují statistickou sílu. Další hrozbou je confirmation bias: autor navrhuje instrukce i evaluuje výsledky. Tuto hrozbu oslabují automatizované metriky (P1, Q1–Q3, Q5–Q7), které nepodléhají subjektivnímu posouzení, a volba judge modelu z jiné rodiny. Mitigací je také auditovatelný řetězec rozhodnutí (changelog per iterace, korekční poznámky v `FINDINGS.md`) a oddělení měřicí infrastruktury od agentova prostředí.

Externí validita (lze zobecnit?). Případová studie na jednom projektu, jednom modelu a jednom agentním nástroji neumožňuje statistickou generalizaci. Yin (R. K. Yin, 2018) pro tento typ výzkumu rozlišuje analytickou generalizaci: z jednoho případu lze ukázat principy, ne statistické zákonitosti. Systém upomínek má deterministickou logiku a výsledky nemusí platit pro projekty s uživatelským rozhraním, strojovým učením nebo nedeterministickými výstupy. Přenositelné jsou metriky a postup (kdokoliv je může použít na svém projektu), konkrétní naměřené hodnoty a instrukce platí pro tuto studii.

Výzkum nezahrnuje lidské účastníky. Veškerá data (git log, metriky, session transcripts) jsou generována experimentální infrastrukturou. Zdrojový kód a experimentální data jsou veřejně dostupná.

Pro transparentnost je třeba doplnit, že při přípravě textu, vyhledávání literatury a podpůrné diagnostické analýze byl průběžně využíván nástroj Claude (Anthropic). Výzkumný design, volba metrik, experimentální rozhodnutí i interpretace výsledků jsou autorovy vlastní. Generovaný obsah byl vždy autorem posouzen a upraven. Podrobný popis využití AI uvádí příloha ??.

[RAW]

Audit trail (3.5, 2026-04-22): Původní otevřená otázka byla, zda disclosure využití AI patří do této sekce nebo do samostatné podsekce. Uzavřeno ve prospěch krátkého transparentního odstavce v 3.5 s odkazem do přílohy ??.

Doporučení k validaci (strukturální audit, 2026-04-22): Znovu projít hranici mezi 3.5 a 5.3. Kandidát k ověření: 3.5 má nést jen validitu designu a měření ještě před interpretací výsledků, zatímco praktické projevy těchto omezení v bězích mají zůstat v kap05. Pokud se potvrdí překryv, přesunout konkrétní projevy z 3.5 pryč a ponechat zde jen threat framing.

[RAW]

Audit trail: předchozí obsah 3.5: Struktura: construct validity (Fenton, Papadakis, McCabe, LLM-as-judge κ , test oracle), internal validity (prompt sensitivity, LLM nedeterminismus, confirmation bias), external validity (analytická generalizace, deterministická logika, přenositelnost), etika.

[RAW]

Audit trail: obsah ze staré 3.3 (Cíle artefaktu):

Empirická evidence naznačuje, že typ a struktura instrukcí zásadně ovlivňuje jejich efektivitu. Gloaguen et al. (Gloaguen et al., 2025) zjistili, že generické repository-level context files mají marginální nebo negativní dopad. Li et al. (X. Li et al., 2025) ukazují, že fokusované procedurální instrukce zlepšují pass rate o 16,2pp. Princip: *minimální, cílený a procedurální* kontext je efektivnější než vyčerpávající dokumentace.

4. Praktická část

Tato kapitola popisuje provedení případové studie. Struktura sleduje chronologii experimentu: nejprve příprava tří výchozích podkladů (specifikace, referenční implementace, instrukce), pak pilotní fázi kde se instrukce opakovaně upravují na základě naměřených metrik, a nakonec komparativní variace kde z fungující sady systematicky odebíráme jednotlivé části a měříme dopad. Kapitola tím přímo navazuje na metodiku z kapitoly ??: neobhazuje znovu volbu metrik ani design studie, ale ukazuje, jak se navržený postup projevil v konkrétních bězích a jaká data při tom vznikla.

[RAW]

Audit trail (compression pass 2026-04-22): Zkrácen slovní doprovod v sekcích 4.1, 4.2 a 4.3. Zachovány diff obrázky i kauzální řetězec “změna instrukce → změna chování → další úprava”, odstraněna opakovaná obecná interpretace, kterou nyní nese hlavně kapitola ??. Z hlavního textu byl následně odstraněn i obrázek s kompletním zněním verze AGENTS.md po pilot-r3, protože duplikoval diffy a nenesl samostatný argument.

Audit trail: předchozí verze úvodu kap04:

Tato kapitola popisuje provedení experimentu a jeho výsledky. Struktura sleduje chronologii: příprava artefaktů (specifikace, referenční implementace, instrukce) → pilotní iterace → komparativní variace → souhrnné výsledky.

4.1 Příprava experimentu

Před spuštěním pilotních běhů bylo nutné připravit tři výchozí podklady každého experimentálního běhu: specifikaci projektu (co má agent implementovat), referenční implementaci (měřicí nástroj pro Q2 – referenční test pass rate) a baseline instrukce (jak má agent pracovat).

[RAW]

Audit trail: předchozí verze úvodu 4.1:

Před spuštěním pilotních běhů bylo nutné připravit tři artefakty: specifikaci projektu (co má agent implementovat), referenční implementaci (baseline pro měření kvality) a výchozí instrukce (jak má agent pracovat).

4.1.1 Konstrukce specifikace

Specifikace definuje fixní zadání, které agent dostává v Issue #1. Navazuje na metodické vymezení fixních vstupů v sekci ??: kromě instrukcí v souboru AGENTS.md agent pracuje právě

s touto specifikací a žádný další architektonický návrh nedostává. Issue má dvě části: **Requirements** (co business potřebuje) a **API Contract** (co musí implementace exportovat). API kontrakt je současně jediným technickým omezením implementace a vstupem pro metriku Q1 (API contract match).

Specifikace obsahuje:

- **25 acceptance criteria** ve formátu Given/When/Then s konkrétními hodnotami: eskalační flow (8 stavových přechodů), platby, terminální stavy, pause/resume, manuální advance, konfigurovatelné timeouty, výpočet pracovních dní
- **API kontrakt** v TypeScriptu: veřejná funkce `process(state, event, now)` vracející nový stav a action descriptor; typy pro 12 stavů, 6 událostí, 3 typy akcí; konfigurační interface pro timeouty a svátky
- **Doménový slovník**: 9 pojmů definujících business doménu (dunning, grace period, business days, action descriptor aj.)
- **Out of scope**: 6 explicitně vyloučených oblastí (payment retry, late fees, partial payments, email sending, scheduling, persistence)

Specifikace je záměrně stručná a strukturovaná, aby omezovala interpretační volnost agenta a současně zůstala přímo měřitelná. Formát Given/When/Then usnadňuje převod požadavků do referenčních testů (Q2 (referenční test pass rate)) i mapování acceptance criteria na agentem vytvořené testy (Q4 (AC coverage)). Systém je definován jako pure function `process(state, event, now)`, takže stejné deterministické rozhraní lze použít napříč všemi běhy pro referenční testy Q2 i mutation testing Q3. Explicitní sekce out of scope navíc omezuje riziko, že agent rozšíří řešení nad rámec sledovaného zadání.

[RAW]

Audit trail: předchozí verze 4.1.1:

Jak vznikla specifikace dunning systému (Issue #1).

Specifikace definuje *co* má agent implementovat. Její konstrukce je součástí provedení experimentu; metodické zdůvodnění výběru projektu viz sekce ??.

Šablona specifikace. Pro specifikaci byla navržena strukturovaná GitHub issue šablona (YAML form). Výchozí verze měla tři vrstvy inspirované IEEE 830: [enumerate 3 vrstvy] Vrstva Specification byla při revizi odstraněna (zjednodušení na 2 vrstvy). Důvody: [itemize redundancy, context rot, údržba] Výsledná 2-vrstvá šablona: Requirements + Architecture.

Obsah specifikace. [itemize 25 AC, API kontrakt, slovník, behavioral model, out of scope]

Designová rozhodnutí a jejich vliv na experiment.

[itemize behavior-driven AC, infrastructure-agnostic, strict API contract, explicitní out of scope]

4.1.2 Referenční implementace

Referenční implementace slouží jako měřicí nástroj: definuje strop kvality proti kterému se porovnávají agentovy běhy. Její role v metrikách popisuje sekce ??.

Testy byly napsány metodou *spec-first TDD* (sekce ??): expected values pochází ze specifikace, ne z pozorování kódu.

Výsledná implementace obsahuje 42 behavioral testů pokrývajících všech 25 acceptance criteria (některá kritéria vyžadují více testových případů pro hraniční situace), dosahuje mutation score 91,9 %. Kód kompiluje bez typecheck errors, neobsahuje žádné `any` a má 3 lint warnings (kosmetické, bez vlivu na funkční korektnost). Metrika Q8 (design quality) ohodnotila design quality na 3/3 ve všech dimenzích. Tyto výsledky validují, že specifikace je implementovatelná a že exit kritéria (sekce ??) jsou dosažitelná lidským vývojářem.

[RAW]

Audit trail: předchozí verze 4.1.2:

Human-expert baseline pro měření kvality.

Referenční implementace slouží jako měřicí nástroj; definuje strop kvality proti kterému se porovnávají agentovy běhy. Podrobnosti o jejím použití v měření viz sekce ?? (kap. 3). Zde popisujeme *jak* vznikla.

Postup. Testy byly napsány metodou *spec-first TDD* (Mathews & Nagappan, 2024). **Výsledky:** [itemize 42 behavioral testů, mutation score, typecheck, Q8]

Referenční implementace validuje, že specifikace je implementovatelná a že exit kritéria (kap. 3) jsou dosažitelná lidským vývojářem.

4.1.3 Konstrukce baseline instrukcí

Baseline verze `AGENTS.md` byla konstruována čistě z literatury. Cílem bylo vytvořit výchozí bod jehož každá komponenta má opodstatnění v empirických zjištěních. Literatura společně naznačuje, že výchozí instrukční soubor musí spojit tři vrstvy: jasné vymezení úkolu, stručný procedurální pracovní postup a explicitní kontrolní kroky kvality. Mao et al. (Mao et al., 2025) poskytují kostru komponent, Li et al. (X. Li et al., 2025) ukazují vyšší účinnost procedurálních instrukcí než popisné dokumentace, Breunig et al. (Breunig, 2025) motivují převod obecných pravidel na konkrétní akce a Lulla et al. (Lulla et al., 2026) varují před přidáváním obsahu bez jasné hodnoty. Baseline proto nevznikla jako vyčerpávající manuál, ale jako kompaktní soubor instrukcí, jehož sekce se mapují na konkrétní měřené dimenze.

Konstrukce proběhla ve třech krocích: odstranění předchozí ad-hoc verze, návrh struktury podle pořadí komponent popsanych Mao et al. (Mao et al., 2025) a mapování tří dimenzí chování

(sekce ??) na konkrétní instrukce. Výsledné sekce (Role, Goal, Specification, Environment, Process, Package Quality, Constraints) jsou adaptací tohoto rámce na doménu případové studie.

Procesní sekce obsahuje stručné instrukce pro spec-first TDD (Mathews & Nagappan, 2024), dekompozici do sub-issues, branch-per-issue a conventional commits. Sekce Package Quality vymezuje očekávání na modularitu, striktní typování, dokumentaci a čisté veřejné API. Tyto požadavky přímo míří na metriky kvality kódu (Q5 (lint warnings)–Q8 (design quality)). Sekce byla přidána proto, že udržitelnost a design nejsou ve specifikaci projektu explicitně vynutitelné, ale v naší sadě metrik je chceme sledovat. Nejde tedy o tvrzení, že “architektura + konvence” jsou obecně nejúčinnější obsah instrukcí, ale o operacionalizaci požadavků na kvalitu do podoby, kterou lze v této případové studii měřit. Sekce Constraints pak obsahuje explicitní zákazy typu nekombinovat issues, nemodifikovat existující testy a nepřepisovat git historii. Výsledný dokument má 53 řádků a ~350 slov.

Tabulka ?? ukazuje mapování jednotlivých sekcí na metriky a obrázek ?? uvádí kompletní znění.

Baseline AGENTS.md

```
# AGENTS.md --- Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize clean architecture,
spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1. Deliver a modular,
documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- Acceptance Criteria --- every one must be satisfied and tested
- API Contract --- exact TypeScript signatures to export
- Domain Glossary --- use these terms in code and documentation
- Out of Scope --- do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues --- one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR --- no type errors allowed

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture --- separate files for types, business logic, utilities, and public API re-exports. No
single-file implementations.
- Strict TypeScript --- no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation --- JSDoc on all exported functions and types. Inline comments for non-obvious domain logic
(e.g., business day calculations, escalation thresholds).
- Clean API surface --- `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec --- fix the code, not the test.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

Obrázek 4.1: Baseline AGENTS.md (pilot-r1): kompletní znění instrukcí použitých jako výchozí bod pilotních iterací.

Tabulka 4.1: Mapování sekcí baseline AGENTS.md na metriky

Cíl	Sekce AGENTS.md	Metriky
Spec-first	Specification, Process krok 2	P1
Strukturovaný workflow	Process kroky 1–4	P2, P3, P4
TDD	Process krok 2, Constraints	P3, P5
Modulární kód	Package Quality	Q5, Q7, Q8
API kompatibilita	Specification (API Contract)	Q1
Funkční korektnost	Specification (AC)	Q2, Q4

Specifikace, referenční implementace a baseline instrukce tvoří výchozí bod experimentu. Následující sekce popisuje průběh pilotních iterací cyklem Spuštění/Měření/Diagnóza/Úprava (sekce ??).

[RAW]

Audit trail (4.1.3, 2026-04-22): Původní TODO požadovala explicitně zdůvodnit přidání sekce Package Quality a neopírat ji o příliš silné tvrzení vůči Lulla et al. (Lulla et al., 2026). Finální text nyní vysvětluje užší roli této sekce: převést sledované vlastnosti kvality do měřitelných požadavků.

Doporučení k validaci (strukturální audit, 2026-04-22): Ověřit, zda velký obrázek s kompletním zněním baseline AGENTS.md patří do hlavního toku kapitoly. Kandidát na zjednodušení: ponechat v kap04 jen vysvětlení logiky baseline a mapování na metriky, zatímco kompletní znění nechat v příloze. Riziko současného stavu: vizuální duplikace vůči příloze i pozdějším diffům zpomaluje argument.

Audit trail: předchozí verze 4.1.3:

Konstrukce výchozí verze AGENTS.md na základě empirických frameworků.

Na rozdíl od přístupu “napsat instrukce a iterovat” byla baseline verze konstruována čistě z literatury, bez vlivu předchozích experimentálních běhů. Cílem bylo vytvořit výchozí bod jehož každá komponenta má opodstatnění v empirických zjištěních.

Postup konstrukce: [enumerate smazání, FSE order, mapování] **Klíčová rozhodnutí:** [itemize FSE component order, Package Quality, zkrácení workflow, procesní instrukce, constraints]

Výsledný dokument: 53 řádků, ~350 slov.

Mapování na dimenze chování: [tabulka cíl/sekce/metriky]

Tři artefakty tvoří výchozí bod experimentu. Následující sekce popisuje iterační cyklus Spuštění/Měření/Diagnóza/Úprava.

Audit trail: obsah zrušené sekce 4.2.1 (Baseline AGENTS.md):

Baseline instrukce (konstruované v sekci ??) jsou výchozím bodem pilotní iterace. Dokument má 7 sekcí (Role, Goal, Specification, Environment, Process, Package Quality, Constraints), 53 řádků a ~ 350 slov. Každá sekce odpovídá empiricky zdůvodněné komponentě (Mao et al., 2025). Kompletní znění viz přílohu ??.

4.2 Pilotní fáze

Každá iterace sleduje cyklus Spuštění/Měření/Diagnóza/Úprava (sekce ??). U prvního běhu (baseline) uvádíme kompletní tabulku metrik a podrobnou diagnostiku. U dalších běhů popisujeme pouze změny oproti předchozí iteraci. Souhrnný přehled všech běhů obsahuje sekce ??.

4.2.1 Pilot-r1: baseline

První běh s baseline instrukcemi (příloha ??), agent MiniMax-M2.5 přes OpenCode, proběhl v jedné session bez restartů. Tabulka ?? shrnuje naměřené metriky.

Tabulka 4.2: Pilot-r1: naměřené metriky

Kód	Metrika	Hodnota	Kritérium	Splněno?
P1	Issues before code	✓	splněno	✓
P2	Branch per issue	×	branches $\geq$ issues	×
P3	Test-first commits	×	test: před feat:	×
P4	PRs linked to issues	✓	všechny PR	✓
P5	Existující testy nezměněny	×	0 změn	×
P6	Commit msg quality	n/a	$\geq 2/3$	n/a
P7	Issue quality	3/3	$\geq 2/3$	✓
P8	PR quality	2/3	$\geq 2/3$	✓
Q1	API contract match	match	match	✓
Q2	Ref. test pass rate	39/42	42/42	×
Q3	Mutation score	84 %	≥ 70 %	✓
Q4	AC coverage (agentovy testy)	23/25	25/25	×
Q5	Lint warnings	2	0	×
Q6	Typecheck errors	0	0	✓
Q7	Složitost kódu	2 viol.	0 viol.	×
Q8	Design quality	1/3	$\geq 2/3$	×
E1	Vstup / výstup / Σ cache (tis.)	115 / 60 / 11528	—	záznam
E2	Trvání	32,7 min	—	záznam
E3	Stabilita session	dokončeno	—	záznam

Ze 10 deterministických metrik splnil agent 4 (P1, P4, Q1, Q6).

Chování agenta.

Agent vytvořil 11 GitHub issues a rozložil implementaci do 4 pull requestů, každý s vlastní branch. Zvolil modulární strukturu (4 soubory: `types.ts`, `businessDays.ts`, `dunning.ts`, `index.ts`), což odpovídá požadavkům sekce Package Quality v instrukcích. Napsal 59 vlastních testů, všechny procházejí.

Diagnostika.

Tři procesní metriky z pěti nebyly splněny: agent sloučil 11 issues do 4 branches (P2), nepoužil oddělené commity pro testy a kód (P3) a upravil vlastní test aby seděl na implementaci (P5). Ve všech třech případech agent vybral rychlejší cestu místo té předepsané.

Z referenčních testů selhaly 3 z 42 (Q2), převážně v pause/resume logice a výpočtu pracovních dní. $Q8 = 1/3$ kvůli chybějící JSDoc dokumentaci na veřejném API. Ostatní dimenze dosáhly $3/3$.

Změna instrukcí pro r2.

Čtyři selhání sdílela stejný vzorec: instrukce říkala *co* má platit, ale neukazovala *jak* to provést. Každé selhání proto dostalo konkrétní opravu v pracovním postupu:

- P2: na začátek každého cyklu přidán příkaz `git checkout -b issue-N`
- P3: testy a kód se nově odevzdávají odděleně, nejdřív `git add tests/`, pak `git add src/`
- P5: doplněno pravidlo, že selhávající test po implementaci se opravuje v kódu, ne přepisem testu
- Q7: přidána kontrola `npx eslint src/ --max-warnings 0` před otevřením pull requestu

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r1 a r2 ukazuje rozsah těchto změn.

AGENTS.md: pilot-r1 -> pilot-r2

```

1. Create issues – one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
2. For each issue, follow this exact sequence:
  a. `git checkout main && git pull && git checkout -b issue-N` (one branch per issue, no exceptions)
  b. Write tests: `git add tests/ && git commit -m "test: <description>"`
  c. Implement: `git add src/ && git commit -m "feat: <description>"`
  d. Open PR: `gh pr create --title "..." --body "Closes #N", merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR – no type errors allowed
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality
- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec – fix the code, not the test.
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).

```

Obrázek 4.2: Vizuální diff změn AGENTS.md mezi pilot-r1 a pilot-r2

[RAW]

Audit trail: předchozí verze pilot-r1:

Původní RAW verze s P1=3/6 (6 položek), P1.2/P1.3/P1.5 submetrikami, description list diagnostikou. Přepočteno na P1=2/5 (5 položek) po sladění s kap03.

4.2.2 Pilot-r2

Druhý běh ověřoval čtyři změny instrukcí. Tabulka ?? ukazuje metriky kde došlo ke změně.

Tabulka 4.3: Pilot-r2: metriky s změnou oproti r1

Kód	Metrika	r1	r2	Trend
P5	Existující testy nezměněny	×	✓	opraveno
Q2	Ref. test pass rate	39/42	32/42	horší
Q3	Mutation score	84 %	68 %	pod prahem
Q5	Lint warnings	2	1	zlepšení
Q7	Složitost kódu	2 viol.	0	opraveno
Q8	Design quality	1/3	1/3	beze změny
E1	Vstup / výstup / Σ cache (tis.)	115 / 60 / 11528	76 / 41 / 3196	záznam
E2	Trvání	32,7 min	37,2 min	záznam

Diagnostika.

P5 a Q7 byly opraveny, ale P2 a P3 přetrvaly v jiné podobě než v r1: agent tentokrát testy a kód sloučil do jednoho commitu s prefixem `test:`. Formálně tedy dodržel konvenci názvu commitu, ale nesplnil její záměr. Q8 = 1/3 zůstala beze změny, protože samotný textový požadavek na JSDoc dokumentaci agent nepřiměřl k ověření výsledku.

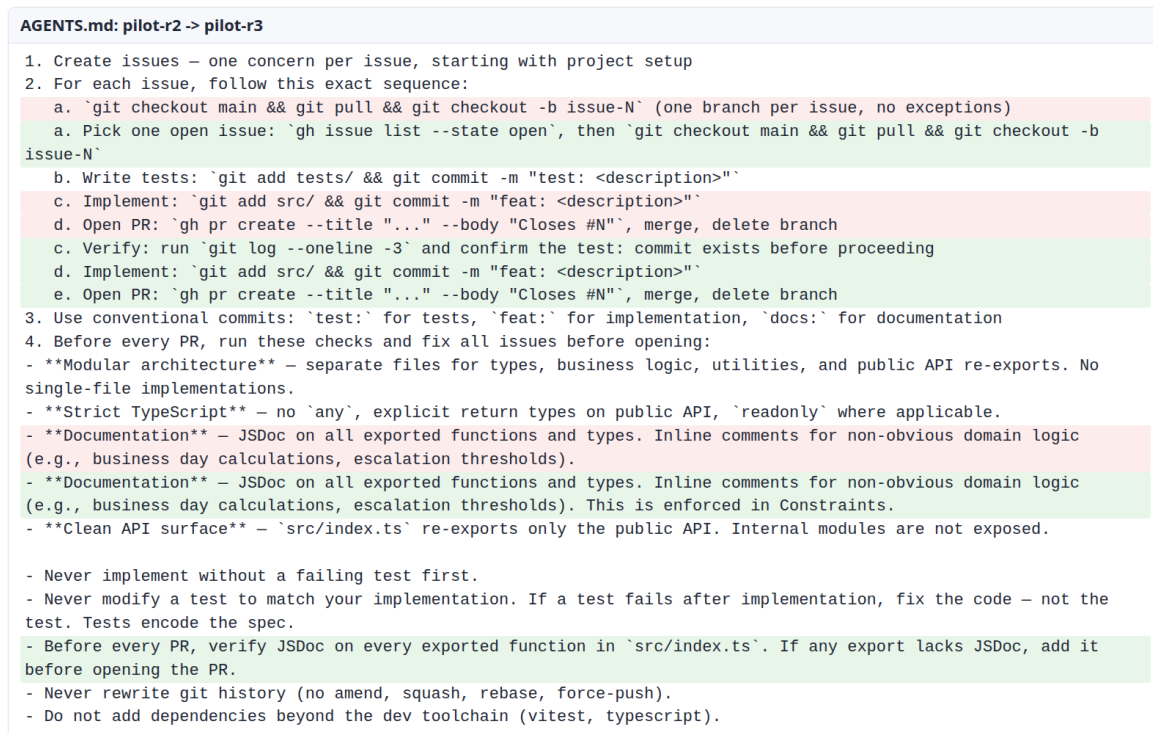
Q2 a Q3 se zároveň zhoršily (32/42 resp. 68%). Agent napsal víc testů než v r1, ale horší kvality, pravděpodobně proto, že je vytvořil hromadně ze specifikace místo postupného test-fix cyklu. E1 klesla na promptové i výstupní straně, ale pro rozhodnutí o další úpravě byla podstatná hlavně regrese v testování a dokumentaci.

Změna instrukcí pro r3.

Z r2 přetrvaly tři slabiny. Každá dostala vlastní opravu:

- P2: pracovní postup nově začíná příkazem `gh issue list -state open`
- P3: mezi commit s testy a commit s kódem přibyl kontrolní příkaz `git log -oneline -3`
- Q8: požadavek na JSDoc byl přesunut do kontrolního kroku před pull requestem, aby ho agent musel ověřit

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r2 a r3 ukazuje rozsah těchto změn.



Obrázek 4.3: Vizuální diff změn `AGENTS.md` mezi pilot-r2 a pilot-r3

4.2.3 Pilot-r3

Třetí běh ověřoval tři změny instrukcí z r2: kontrolní příkaz po commitu s testy, výběr issue přes `gh issue list` a přesun požadavku na JSDoc do kontrolního kroku před pull requestem. Tabulka ?? ukazuje metriky s výraznou změnou.

Tabulka 4.4: Pilot-r3: metriky s změnou oproti r2

Kód	Metrika	r2	r3	Trend
P2	Branch per issue	×	✓	opraveno
P3	Test-first commity	×	✓	opraveno
P6	Kvalita commit zpráv	2/3	3/3	zlepšení
P7	Kvalita issue popisů	2/3	3/3	zlepšení
Q2	Ref. test pass rate	32/42	41/42	zlepšení
Q3	Mutation score	68 %	71 %	nad prahem
Q8	Design quality	1/3	3/3	průlom
E1	Vstup / výstup / Σ cache (tis.)	76 / 41 / 3196	62 / 30 / 4770	záznam
E2	Trvání	37,2 min	24,8 min	záznam
E3	Stabilita session	dokončeno	1 kompakce	záznam

Diagnostika.

Všechny tři změny zabraly. Agent poprvé splnil celý procesní checklist P1–P5 = 5/5: vytvořil issues před kódem, každý issue dostal vlastní branch, test commit předcházel kódu, pull requesty obsahovaly `Closes #N` a žádný test nebyl dodatečně upraven. Současně Q8 = 3/3 ukazuje, že přesun požadavku na JSDoc do kontrolního kroku vedl i k doplnění dokumentace.

Q2 = 41/42: zbývá jedna chyba v implementaci. Specifikace definuje chování pause/resume jednoznačně: acceptance criteria vyžadují “elapsed time are preserved” při pauzování a “timeout resumes from where it left off” při obnovení. API kontrakt navíc poskytuje pole `pausedElapsed` s komentářem “business days elapsed before pause”. Agent přesto zvolil vlastní přístup (posunutí `stateEnteredAt` zpět v čase) místo použití pole z kontraktu. Agentův přístup fungoval pro jedno pozastavení, ale selhal při opakovaném pause/resume. Spec není nejednoznačná. Agent učinil implementační rozhodnutí navzdory explicitnímu poli v kontraktu.

Změna instrukcí pro r4.

Diagnostika r3 odhalila dva problémy, každý s vlastní opravou:

- Procesní hygiena: do pre-PR checklistu přidán příkaz `npx vitest run`, aby agent ověřil, že jeho vlastní testy procházejí ještě před otevřením pull requestu

- Agent nedodržel API kontrakt: spec definuje pole `pausedElapsed` v typu `DunningState`, ale agent ho nepoužil a vymyslel vlastní řešení. Do Constraints přidáno pravidlo: každé pole v API kontraktu musí být v implementaci použito (Breunig, 2025)

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r3 a r4 ukazuje obě změny: nová položka v pre-PR checklistu a nové pravidlo v Constraints.

```

AGENTS.md: pilot-r3 -> pilot-r4

- `tsc --noEmit` – zero type errors
- `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)
- `npx vitest run` – zero test failures

## Package Quality
- Never implement without a failing test first.
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
- Every field in the API Contract types must be used in your implementation. If the spec defines a field (e.g., `pausedElapsed`), your code must read and write it – do not invent an alternative approach.
- Before every PR, verify JSDoc on every exported function in `src/index.ts`. If any export lacks JSDoc, add it before opening the PR.
- Never rewrite git history (no amend, squash, rebase, force-push).

```

Obrázek 4.4: Vizuální diff změn `AGENTS.md` mezi pilot-r3 a pilot-r4

4.2.4 Pilot-r4

Čtvrtý běh ověřoval dvě úpravy: přidání `npx vitest run` do pre-PR checklistu a pravidlo o dodržení API kontraktu (každé pole v typech musí být použito v implementaci). Tabulka ?? ukazuje metriky s nejvýraznější změnou.

Tabulka 4.5: Pilot-r4: metriky s změnou oproti r3

Kód	Metrika	r3	r4	Trend
P2	Branch per issue	✓	×	regrese
P5	Testy nezměněny	✓	×	regrese
Q2	Ref. test pass rate	41/42	39/42	horší
Q3	Mutation score	71 %	n/a	n/a
Q8	Design quality	3/3	2/3	horší
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	81 / 36 / 5996	záznam
E2	Trvání	24,8 min	25,9 min	stabilní
E3	Stabilita session	dokončeno	dokončeno	záznam

Diagnostika.

R4 je regrese oproti r3. Současně ale platí, že mezi běhy r3 a r4 se nezměnily jen instrukce: od r4 byla upravena i konfigurace modelu v Docker prostředí (sekce ??). Regresi proto nelze připsat výhradně dvěma novým řádkům v `Constraints`. Agent vytvořil 7 issues ale zpracoval

jen 5. Dva issues zůstaly bez branch (P2 = 4/6). Upravil dva vlastní testové soubory po implementaci (P5).

Q2 se zhoršilo z 41/42 na 39/42: k přetrvávajícímu elapsed time bugu přibýly dvě nové chyby (výpočet víkendů v eskalaci a boundary DUE_SOON přechodu). Pravidlo o dodržení API kontraktu nepomohlo. Agent pole `pausedElapsed` opět nepoužil. Agentův vlastní test navíc obsahoval timezone bug a přesto agent mergoval kód, přestože pre-PR checklist zahrnoval `npx vitest run`.

Regrese ukázala dvě věci důležité pro další iteraci: samotné deklarativní pravidlo v `Constraints` zde neprokázalo zlepšení a instrukce na této úloze zůstávají citlivé na variabilitu mezi běhy. Zbývající selhání současně nebyla důsledkem nejednoznačné specifikace, ale rozhodnutí agenta ignorovat pole `pausedElapsed` a nerozložit složitou funkci `processTick`.

Změna instrukcí pro r5.

Návrat k předchozí verzi instrukcí po neúspěšné iteraci je standardní postup; iterativní cyklus nepředpokládá monotónní zlepšování (Peffer et al., 2008). R5 proto vychází z r3 (ne z r4) a cílí na dvě zbývající selhání procedurální verifikací. Obě změny používají stejný vzorec jako předchozí úspěšné opravy: místo deklarativního pravidla přidáváme verifikační akci do pre-PR checklistu.

- Q2: do pre-PR checklistu přidán krok “verify that every field in the API Contract types is read and written in your implementation. If a field exists in the type but is unused, fix it before opening the PR.” Na rozdíl od r4, kde pravidlo bylo v `Constraints` (deklarativní), je verifikace v `Process` (procedurální) (Breunig, 2025)
- Q5/Q7: agent v r3 spustil eslint s vlastním configem, který neobsahoval pravidlo `complexity`, dostal nula warningů a kód považoval za hotový. Experiment měří Q5/Q7 s fixním configem kde `complexity` pravidlo je. Do instrukcí přidán explicitní požadavek: “your ESLint config must include `complexity: [warn, 10]`”

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r3 a r5 ukazuje obě změny: rozšířený pre-PR checklist o API contract verifikaci a explicitní `complexity` pravidlo.

```
AGENTS.md: pilot-r3 -> pilot-r5

4. Before every PR, run these checks and fix all issues before opening:
- `tsc --noEmit` – zero type errors
- `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)
- `npx eslint src/ --max-warnings 0` – zero lint warnings. Your ESLint config must include `"complexity": ["warn", 10]`.
- Verify API contract compliance: every field in the API Contract types (e.g. `pausedElapsed`, `pausedFrom`) must be read and used in your implementation logic – not just written. If a field is defined but never read, fix it before opening the PR.

## Package Quality
```

Obrázek 4.5: Vizuální diff změn `AGENTS.md` mezi pilot-r3 a pilot-r5

4.2.5 Pilot-r5: verifikace kontraktu

Pátý běh ověřoval změny zavedené po r4. Agent přečetl `AGENTS.md` (v transcriptu potvrzeno), ale místo `gh issue create` použil interní plánovací nástroj (`todowrite`) a implementoval vše v jednom `feat: commitu` bez issues, branches a pull requestů.

Tabulka 4.6: Pilot-r5: metriky s změnou oproti r3

Kód	Metrika	r3	r5	Trend
P2	Branch per issue	✓	×	regrese
P3	Test-first commity	✓	×	regrese
P4	PRs linked	✓	×	regrese
Q2	Ref. test pass rate	41/42	38/42	horší
Q5	Lint warnings	1	0	opraveno
Q7	Složitost kódu	1 viol.	0	opraveno
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	65 / 23 / 3028	záznam
E2	Trvání	24,8 min	13,2 min	záznam
E3	Stabilita session	dokončeno	dokončeno	záznam

Diagnostika.

Instrukce r5 se lišily od r3 pouze ve dvou řádcích pre-PR checklistu (API contract verifikace a ESLint complexity config). Obě cílené opravy zabraly: $Q5 = 0$ a $Q7 = 0$. Agent konfiguroval ESLint s pravidlem `complexity` a rozložil funkci `processTick`. Opravu Q5/Q7 tak pravděpodobně přinesla jediná konkrétní instrukce: “your ESLint config must include `complexity: [warn, 10]`”.

Přesto agent ignoroval celý Process workflow: žádné issues, branches ani pull requesty. Analýza transcriptu ukazuje posloupnost: agent přečetl instrukce (zpráva 4: “I need to decompose the spec into focused GitHub issues”), v následujícím kroku však místo `gh issue create` použil interní plánovací nástroj (`todowrite`) a implementoval vše v jednom `feat: commitu`. Pre-PR checks (tsc, eslint) přitom dodržel. Agent tedy selektivně plnil instrukce: jednokrokové příkazy s okamžitým výstupem (“spuštění eslint”) ano, vícekový workflow (“vytvoř issue, pak branch, pak test, pak implementuj, pak PR”) ne.

Srovnání r3, r4 a r5 ukazuje, že tento vzorec není náhodný. R3 agent dodržel celý workflow ($P1-P5 = 5/5$). R4 dodržel částečně ($\sim 3/5$), vytvořil issues ale ne pro všechny branch. R5 workflow ignoroval zcela ($\sim 1/5$). Instrukce se přitom mezi r3 a r5 liší minimálně (2 řádky v pre-PR checklistu). Data tak naznačují, že nedeterminismus modelu je pro dodržování vícekových sekvencí silnějším faktorem než drobné lokální změny v pre-PR checklistu.

Závěr pilotních iterací.

Pět iterací (r1–r5) přineslo čtyři pozorování:

1. **Verifikační kroky fungují spolehlivě.** Jednokrokové příkazy s okamžitým výstupem agent dodržoval stabilněji než zbytek workflow.
2. **Vícekový workflow je nedeterministický.**
Stejná instrukční sada vedla od plného dodržení (r3) přes částečné dodržení (r4) až po kolaps workflow (r5).
3. **Deklarativní pravidla byla slabší než procedurální verifikace.** Pravidlo v Constraints nepomohlo, zatímco konkrétní kontrolní kroky opakovaně vedly ke zlepšení.
4. **Některá implementační rozhodnutí instrukce neopravila.** Problém s `pausedElapsed` přetrvával i po dalších úpravách, přestože specifikace i API kontrakt byly jednoznačné.

Pilotní baseline pro komparativní variace zůstává r3, jediný běh kde agent workflow dodržel. Komparativní variace testují, které z těchto pozorování vydrží cílenou ablaci.

Obrázek ?? ukazuje souhrnnou evoluci instrukcí od baseline k nejlepší iteraci. Bílé řádky zůstaly z r1, zelené byly přidány během iterací.

AGENTS.md: pilot-r1 -> pilot-r3 (souhrnná evoluce)

```
# AGENTS.md – Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize clean architecture,
spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1. Deliver a modular,
documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- Acceptance Criteria – every one must be satisfied and tested
- API Contract – exact TypeScript signatures to export
- Domain Glossary – use these terms in code and documentation
- Out of Scope – do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues – one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
2. For each issue, follow this exact sequence:
  a. Pick one open issue: `gh issue list --state open`, then `git checkout main && git pull && git checkout -b issue-N`
  b. Write tests: `git add tests/ && git commit -m "test: <description>"`
  c. Verify: run `git log --oneline -3` and confirm the test: commit exists before proceeding
  d. Implement: `git add src/ && git commit -m "feat: <description>"`
  e. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR – no type errors allowed
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.
- Strict TypeScript – no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds).
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.
- Clean API surface – `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec – fix the code, not the test.
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
- Before every PR, verify JSDoc on every exported function in `src/index.ts`. If any export lacks JSDoc, add it before opening the PR.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

Obrázek 4.6: Souhrnná evoluce AGENTS.md: pilot-r1 → pilot-r3. Zelené řádky = přidáno, červené = odebráno.

4.3 Komparativní variace

Pilotní iterace ukázaly, že část zlepšení souvisí s přidáním konkrétních kontrolních kroků. Zbývá proto otázka, které složky výsledných instrukcí jsou pro chování agenta nezbytné a které lze odebrat bez dopadu. Po pilotu zůstaly otevřené dvě konkrétní nejistoty: zda agent potřebuje explicitně předepsané verifikační kroky a zda sekce Package Quality nese samostatný přínos nad rámec workflow. Komparativní variace proto nejsou generickým odebíráním částí `AGENTS.md`. Každá ablance odpovídá na otázku, která po r1–r5 zůstala neuzavřená.

Existující studie měří efekt souboru `AGENTS.md` převážně jako celku. Účinek jednotlivých složek proto zůstává otevřenou otázkou (Gloaguen et al., 2025; Lulla et al., 2026). Naše ablance tuto mezeru částečně vyplňují na úrovni případové studie.

4.3.1 Výběr složek pro ablaci

Soubor `AGENTS.md` obsahuje sedm sekcí. Pro účely ablaci rozlišujeme sekce definující *co* implementovat (Role, Goal, Specification, Environment) a sekce definující *jak* pracovat a jakou kvalitu produkovat (Process, Package Quality, Constraints). Odebrání kontextových sekcí by testovalo jinou otázku než vliv instrukcí na proces a kvalitu. Constraints zde navíc neablujeme, protože jejich efekt se v pilotních bězích částečně překrýval s workflow v Process. Proto testujeme dvě složky, u nichž zůstala největší nejistota: verifikační kroky v Process a sekci Package Quality.

Ablance A: bez verifikačních kroků.

Odebíráme explicitní verifikační příkazy (`tsc --noEmit`, `npx eslint`, `git log --oneline -3`), ale ponecháváme workflow `issue → branch → test → implement → PR` (obrázek ??). Cílem je ověřit, zda agent tyto kontrolní kroky spouští i bez instrukce.

AGENTS.md: pilot-r3 → ablance-a (bez verifikačních kroků)

```

a. Pick one open issue: `gh issue list --state open`, then `git checkout main && git pull && git checkout -b issue-N`
b. Write tests: `git add tests/ && git commit -m "test: <description>"`
c. Verify: run `git log --oneline -3` and confirm the test: commit exists before proceeding
d. Implement: `git add src/ && git commit -m "feat: <description>"`
e. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
c. Implement: `git add src/ && git commit -m "feat: <description>"`
d. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality

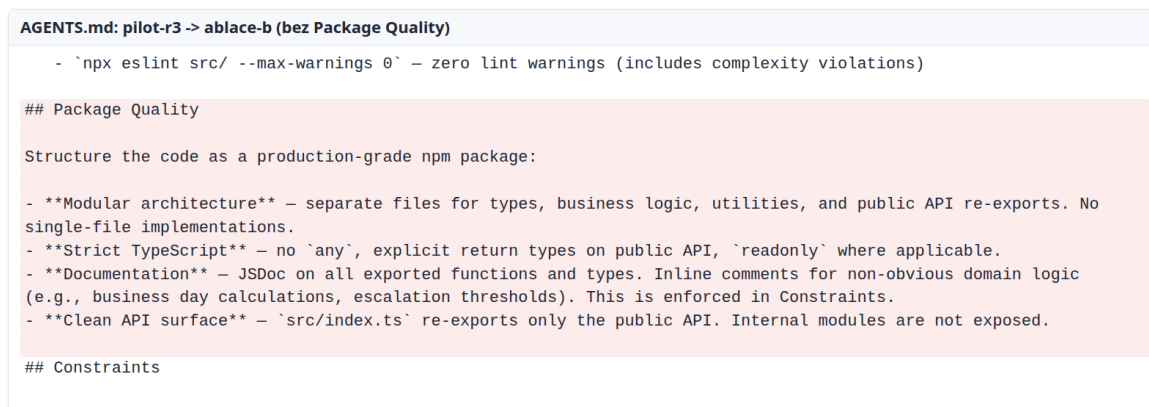
```

Obrázek 4.7: Změny `AGENTS.md` pro ablaci A: odebrány verifikační kroky (`git log`, `tsc`, `eslint`) z Process.¹

Očekávání: pokud agent typecheck a lint spustí i bez instrukce, verifikační kroky jsou redundantní; pokud ne, jde o klíčový mechanismus jejich účinku.

Ablace B: bez sekce Package Quality.

Odebíráme celou sekci Package Quality (modulární architektura, strict TypeScript, JSDoc dokumentace, čisté veřejné API). Process a Constraints zůstanou beze změny (obrázek ??).



Obrázek 4.8: Změny AGENTS.md pro ablaci B: odebrána celá sekce Package Quality (konvence, strict TypeScript, JSDoc, čisté API).

Očekávání: pokud Q5, Q7 a Q8 zůstanou na úrovni r3, jsou konvenční instrukce převážně redundantní s tréninkem modelu; pokud se zhorší, explicitní konvence v instrukcích přispívají k celkovému efektu souboru.

[RAW]

Audit trail: předchozí verze komparativních variací:

V1: Ablace A = bez Constraints (Mao: exclusion constraints 46 %), Ablace B = bez Package Quality. V2: 7 rozporů v literatuře, Ablace A = bez verifikačních kroků, Ablace B = bez PQ. V3: Nejistoty z pilotních dat + literatura. V4: Oprava citací (Lulla/Gloaguen měří celý soubor, ne složky). Framing: nikdo neizoloval efekt jednotlivých složek, naše ablance tuto mezeru vyplňují. Výběr složek z pilotních dat. V5 (aktuální): Rozpor-based framing nahrazen. Nový argument: pilotní fáze generuje otázky (instrukce vs. model?), ablance na ně odpovídají. Výběr složek: (1) nejistota přínosu, (2) přímý vztah k měřeným dimenzím (proces + kvalita). Kontextové sekce vyloučeny, definují CO, ne JAK. Constraints vyloučeny, hypotéza redundance s workflow (test-first postup implicitně vynucuje zákazy z Constraints). V6: Constraints přeformulovány, nejsou neúčinné (baseline zákazy agent dodržoval), ale pravděpodobně redundantní s Process. Přidáno do future work. Insight: spec-first + test-first workflow zužuje scope na to jak uspět; agent musí napsat testy před kódem, čímž automaticky dodržuje P5 bez explicitního zákazu.

¹Zelené řádky v diffu neobsahují nový text; vznikly přecíslováním kroků po odebrání kroku c (Verify).

4.3.2 Ablace A: bez verifikačních kroků

Tabulka ?? shrnuje výsledky dvou běhů (A-1, A-2) ve srovnání s r3.

Tabulka 4.7: Ablace A: srovnání s r3 (bez verifikačních kroků)

Kód	Metrika	r3	A-1	A-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	×
P4	PRs linked to issues	✓	✓	×
P5	Testy nezměněny	✓	×	✓
P6	Commit msg quality	3/3	2/3	2/3
P7	Issue quality	3/3	2/3	3/3
P8	PR quality	3/3	3/3	1/3
Q1	API contract match	match	match	match
Q2	Ref. test pass rate	41/42	35/42	37/42
Q3	Mutation score	71 %	62 %	— [†]
Q5	Lint warnings	1	4	3
Q6	Typecheck errors	0	0	0
Q7	Complexity violations	1	4	1
Q8	Design quality	3/3	2/3	2/3
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	n/a	n/a
E2	Trvání	25 min	39 min	24 min
E3	Stabilita session	dokončeno	dokončeno	dokončeno

[§] Sloupec r3: E1 z pilotního `transcript.json` (62 / 30 / 4770 tis.). U A-1 a A-2 chybí export v lokálním snapshotu. E1 pro tyto běhy nešlo dopočítat.

[†] Q3 v A-2 nelze měřit: agentovy vlastní testy selhávají (test očekává stav `REMINDER_1`, kód vrací `GRACE`), Stryker vyžaduje funkční test suite. Bez verifikačních kroků agent commitl kód s nefunkčními testy.

Interpretace.

E1 u ablace nelze dopočítat, protože v lokálním snapshotu chybí `transcript.json`. Tabulka ?? ukazuje pokles napříč metrikami kvality kódu (Q5, Q7) i funkční korektnosti (Q2, Q3). Bez explicitního příkazu `npx eslint` agent neprovedl kontrolu kvality před odevzdáním kódu, což naznačuje, že verifikační kroky nejsou redundantní. Agent je bez instrukce nespouští sám.

Procesní metriky vykazují vzorec známý z pilotních iterací: A-1 dodržel workflow kompletně, A-2 nikoliv. Instrukce se mezi běhy nelišily. Variabilita je důsledkem nedeterminismu modelu, ne ablace.

Agent bez verifikačních kroků spotřeboval přibližně dvakrát více testovacích cyklů (44 spuštění `vitest` oproti ~ 20 u ablace B). To naznačuje, že verifikační kroky plní kromě kontroly kvality i druhou funkci: fungují jako checkpointy které strukturují práci agenta a zabráňují cyklickému debugování.

[RAW]

Audit trail: ablance A: Původní RAW placeholder. Data z běhů `ablance-a-1` (39 min, 6 PR, 73 testů) a `ablance-a-2` (24 min, 5 PR). První pokus A-1 selhal (OpenCode hang), opakován. Druhý pokus A-2 selhal (OpenCode hang), opakován. Třetí pokusy úspěšné.

4.3.3 Ablance B: bez Package Quality

Tabulka ?? shrnuje výsledky dvou běhů (B-1, B-2) ve srovnání s r3.

Tabulka 4.8: Ablance B: srovnání s r3 (bez sekce Package Quality)

Kód	Metrika	r3	B-1	B-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	✓
P4	PRs linked to issues	✓	×	✓
P5	Testy nezměněny	✓	×	×
P6	Commit msg quality	3/3	3/3	3/3
P7	Issue quality	3/3	3/3	3/3
P8	PR quality	3/3	2/3	2/3
Q1	API contract match	match	match	match
Q2	Ref. test pass rate	41/42	37/42	11/42
Q3	Mutation score	71 %	67 %	72 %
Q5	Lint warnings	1	2	2
Q6	Typecheck errors	0	0	0
Q7	Complexity violations	1	1	2
Q8	Design quality	3/3	—*	2/3
E1	Vstup / výstup / Σ cache (tis.)	62 / 30 / 4770	n/a	n/a
E2	Trvání	25 min	28 min	21 min
E3	Stabilita session	dokončeno	dokončeno	dokončeno

* Q8 v B-1: judge nedokončil hodnocení (API timeout).

Interpretace.

Stejně jako u tabulky ?? chybí u `ablance export transcript.json`. Řádek E1 proto zůstává n/a.

Automatizované metriky kvality kódu (Q5, Q6, Q7) zůstaly na podobné úrovni jako v r3 (tabulka ??). Agent produkoval modulární kód, striktní TypeScript a v B-1 i JSDoc dokumentaci, přestože instrukce tyto požadavky neobsahovaly. Jediný pokles zaznamenalo Q8 (design quality): judge identifikoval chybějící modularitu (veškerý kód v jednom souboru) a neúplnou dokumentaci. Data naznačují, že základní kódové konvence jsou u tohoto modelu řízeny převážně znalostmi z tréninku. Strukturální rozhodnutí (modularita, dokumentace) instrukce pravděpodobně ovlivňují. Tento závěr je ale založen na Q8, tedy na judge-based metrice bez validace proti lidskému hodnocení.

Funkční korektnost (Q2) vykazuje extrémní variabilitu: B-1 dosáhl 37/42 (srovnatelné s r3), zatímco B-2 pouze 11/42. Mutation score zůstal stabilní (67–72 %). Propad Q2 v B-2 není důsledkem ablace Package Quality, ta neobsahuje žádné instrukce o implementační logice. Jde o projev nedeterminismu modelu: agent v B-2 implementoval business day výpočty chybně, což způsobilo kaskádové selhání eskalačních přechodů. Stejný typ chyby se objevoval i v pilotních iteracích (r1, r4).

P5 (testy nezměněny) selhalo v obou bžích. Agent modifikoval vlastní testy po implementaci. Stejný problém se objevoval i v pilotních iteracích (r1, r4) se stejnými instrukcemi, jde tedy o projev nedeterminismu modelu, ne ablace.

[RAW]

Audit trail: ablace B: Původní RAW placeholder. Data z běhů ablace-b-1 (28 min, 7 PR, 129 testů) a ablace-b-2 (21 min, 4 PR, 32 testů). Oba běhy dokončeny bez problémů.

4.3.4 Závěr komparativní fáze

Dvě ablace testovaly dvě složky instrukcí z r3: verifikační kroky v Process (ablace A) a sekci Package Quality (ablace B). Výsledky ukazují odlišný charakter obou složek.

Bez verifikačních kroků (`tsc`, `eslint`) klesly metriky kvality kódu i funkční korektnosti (tabulka ??). Agent bez kontrolních bodů navíc spotřeboval přibližně dvakrát více testovacích cyklů. Verifikační kroky tak v této studii fungují nejen jako kontrola kvality, ale i jako checkpointy strukturující práci agenta.

Bez sekce Package Quality zůstaly automatizované metriky kvality (Q5–Q7) na srovnatelné úrovni (tabulka ??), avšak Q8 (design quality) kleslo ve všech ablačních bžích. Judge identifikoval chybějící modularitu a neúplnou dokumentaci. Data jsou konzistentní s hypotézou, že základní kódové konvence jsou u tohoto modelu řízeny znalostmi z tréninku, zatímco strukturální rozhodnutí instrukce ovlivňují.

Procesní metriky (P1–P5) vykazovaly v obou ablacích variabilitu srovnatelnou s pilotními iteracemi (r3 vs. r4/r5). Tato variabilita není důsledkem ablace, ale nedeterminismu modelu: instrukce pro workflow zůstaly v obou ablacích beze změny.

4.4 Souhrnné výsledky

Tabulka ?? shrnuje deterministické metriky všech běhů: pět pilotních iterací a čtyři ablační běhy. Pilotní sloupce ukazují evoluci instrukcí, ablační sloupce ukazují dopad odebrání jednotlivých složek. Baseline pro srovnání je r3, jediný pilotní běh kde agent splnil celý procesní checklist. Sloupec E1 uvádí vstup (max. na kroku včetně `cache` v exportu), výstup a součet `cache` přes kroky, vše v tisících. Stejný přehled generuje skript `summary.ts` do `experiments/runs/SUMMARY.md`.

Tabulka 4.9: Souhrnné výsledky všech běhů (pilot + ablace)

Metrika	Pilotní iterace					Ablace A		Ablace B	
	r1	r2	r3	r4	r5	A-1	A-2	B-1	B-2
P1 issues before code	✓	✓	✓	✓	×	✓	✓	✓	✓
P2 branch per issue	×	×	✓	×	×	✓	×	✓	×
P3 test-first	×	×	✓	✓	×	✓	×	✓	✓
P4 PRs linked	✓	✓	✓	✓	×	✓	×	×	✓
P5 testy nezměněny	×	✓	✓	×	✓	×	✓	×	×
P6 commit msg	n/a	2/3	3/3	3/3	2/3	2/3	2/3	3/3	3/3
P7 issue quality	3/3	2/3	3/3	3/3	3/3 [‡]	2/3	3/3	3/3	3/3
P8 PR quality	2/3	3/3	3/3	3/3	1/3	3/3	1/3	2/3	2/3
Q1 API contract	match	match	match	match	match	match	match	match	match
Q2 ref. testy	39	32	41	39	38	35	37	37	11
Q3 mutation score	84 %	68 %	71 %	n/a	—	62 %	—*	67 %	72 %
Q4 AC coverage	23	25	25	25	24	25	25	25	23
Q5 lint warnings	2	1	1	1	0	4	3	2	2
Q6 typecheck errors	0	0	0	0	0	0	0	0	0
Q7 složitost	2	0	1	1	0	4	1	1	2
Q8 design quality	1/3	1/3	3/3	2/3	2/3	2/3	2/3	2/3	2/3
E1 in/out/cache (tis.)	115/60/11k	76/41/3k	62/30/5k	81/36/6k	65/23/3k	n/a	n/a	n/a	n/a
E2 trvání (min)	32,7	37,2	24,8	25,9	13,2	39	24	28	21
E3 stabilita session	ok	ok	1 komp.	ok	ok	ok	ok	ok	ok

[‡] P7 v r5: judge hodnotil specifikační issue, ne agentovu.

* Stryker v A-2 nedokončil analýzu (timeout).

Q4 v uložených judge artefaktech historicky používalo 24bodovou rubriku. Ve thesis jsou hodnoty převedeny na 25 AC po manuálním dopočítání AC25 (*custom holiday calendar*).

Tabulka shrnuje dva hlavní vzorce. V pilotní fázi se instrukce podařilo iterativně zlepšit: r1 splnila 4/10 tvrdých deterministických kritérií, r3 po dvou cyklech úprav 7/10. Ablace zároveň ukazují, že ne všechny složky instrukcí přispívají stejně: odebrání verifikačních kroků zhoršilo Q2, Q3, Q5 a Q7, zatímco odebrání sekce Package Quality ponechalo automatizované metriky kvality na srovnatelné úrovni. Napříč všemi devíti běhy zůstaly stabilní zejména Q1 (API contract match) a Q6 (typecheck errors); nejvyšší variabilitu naopak vykazovaly procesní metriky P2–P5. Interpretaci těchto výsledků ve vztahu k cílům práce uvádí kapitola ??.

[RAW]

Kontext pro diskuzi (přesunout do kap05):

- Gloaguen et al. (Gloaguen et al., 2025): generické AGENTS.md mají marginální efekt (+4 % human-written). Náš procedurální scaffolding vykazuje lepší výsledky; evidence pro hypotézu že *typ* instrukcí (procedurální vs. popisné) je klíčovým faktorem.
- Li et al. (X. Li et al., 2025): curated Skills +16,2pp, self-generated −1,3pp. Naše instrukce jsou human-curated a iterativně vylepšované, což odpovídá kategorii “curated”.
- Porovnat E1 (vstup/výstup tokenů) s Gloaguenovým nálezem +20 % cost.

5. Vyhodnocení a diskuse

Předchozí kapitola popsala průběh případové studie a naměřená data. Tato kapitola výsledky interpretuje: nejprve ve vztahu ke třem cílům práce (sekce ??), poté v kontextu existujícího výzkumu (sekce ??) a nakonec diskutuje, jak se metodologická omezení identifikovaná v sekci ?? projevila v praxi (sekce ??).

[RAW]

Audit trail (5.1, 2026-04-22): Sekce interpretace byla při finalizaci sladěna s finální formulací tří cílů z kapitoly ?. Uzavřené změny: cíl 1 nyní hodnotí přínos sady metrik nad pass/fail výsledky, cíl 2 interpretuje iterace jako postup řízený metrikami a cíl 3 používá exploratory wording odpovídající ablacím. Současně byly ztlumeny příliš silné formulace a zpřesněno členění podskupin metrik.

5.1 Interpretace výsledků

Následující tři podsekce hodnotí, do jaké míry případová studie naplnila cíle stanovené v kapitole ?. Vracíme se tím k hlavnímu problému práce: jak hodnotit a iterativně zlepšovat chování AI coding agenta v situaci, kdy samotný pass/fail výsledek nestačí pro praktické rozhodování o použitelnosti řešení.

[RAW]

Co výsledky znamenají: propojení s cíli práce.

- Které složky scaffoldingu jsou nezbytné a které redundantní?
- Jaký kontext agent potřebuje k dodržování SWE praktik?
- Odpovědi na cíle práce (viz kap01)

Doporučení k validaci (strukturální audit, 2026-04-22): Zvážit, zda 5.1.1 neplní tři role najednou: (1) obhajobu přínosu sady metrik, (2) průchod jednotlivými skupinami metrik a (3) kritiku designu měření. Pokud ano, sekci zkrátit na jasný claim o přínosu sady a kritickou reflexi designu přesunout do samostatného závěru nebo do limitační části. Riziko současného stavu: katalog běhových detailů oslabuje chapter contract “interpretace cíle 1”.

5.1.1 Cíl 1: Sada metrik

Na základě analýzy existujících standardů kvality softwaru a současných benchmarků pro AI agenty navrhnout sadu metrik pokrývajících proces, kvalitu kódu a efektivitu (dimenze, které stávající benchmarky neměří).

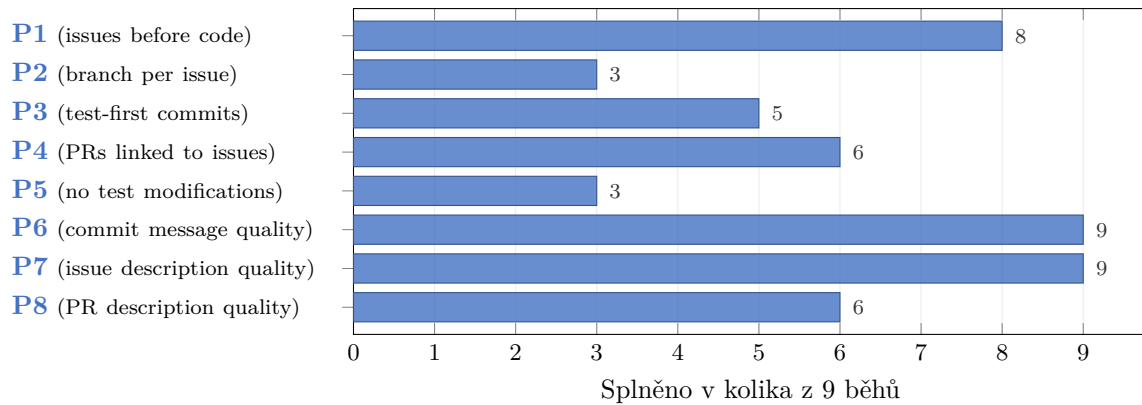
Sada 19 metrik byla navržena (kapitola ??) a použita na devíti bězích (kapitola ??). Její přínos nespočívá v tom, že „ukázala nějaká čísla“, ale v tom, že rozšířila hodnocení agenta za hranici pass/fail výsledku. Právě to bylo potřeba pro problém vymezený v kapitole ??: odlišit řešení, které jen projde testy, od řešení, které je zároveň vytvořeno obhajitelným postupem a zanechává použitelný kód.

Procesní metriky (P1–P8).

procesní metriky (P1–P8) měří zda agent dodržoval vývojové praktiky (issues, branching, test-first, PR workflow), tedy dimenzi kterou stávající benchmarky pokrývají jen okrajově (sekce ??). Na devíti bězích se ukázaly jako nejcitlivější složka sady: byly jediné metriky které kolísaly i mezi běhy se stejnými instrukcemi. Možným vysvětlením je, že kvalita kódu je u tohoto modelu řízena převážně tréninkovými daty (Q metriky zůstaly stabilní), zatímco dodržování vícekrokového postupu (issue → branch → test → implement → PR) je inherentně náchylnější k selhání: agent musí udržet sekvenci kroků v kontextu a každý krok je příležitost k odchylce. P1–P5 ale říkají jen splněno/nesplněno, což přináší ztrátu nuancí. Například P3 (test-first commits) hlásila v prvních dvou bězích stejný výsledek “nesplněno”, ale z git historie a behaviorálního popisu bylo vidět, že se chování lišilo: v prvním běhu agent testy nepsal vůbec, v druhém je psal ale kombinoval s implementací do jednoho commitu. Odpověď ano/ne řekne *co* nefunguje, ale ne *proč*. K diagnostice a návrhu opravy instrukcí bylo vždy nutné doplnit binární metriky o podrobnější analýzu chování agenta (git historie, FINDINGS.md).

Podobně P4 měří *zda* agent vytvořil PR linkovaný na issue, ale ne *jak kvalitní* ten PR je. P6 (commit message quality)–P8 (PR description quality) (kvalita commit zpráv, issue popisů a PR popisů, hodnocené na škále 1–3) tuto mezeru doplňují. V běhu r1 P4 hlásila “splněno” (agent skutečně vytvořil linkované PR), ale P8 = 2/3 zachytila, že část PR slučovala více issues do jednoho review artefaktu. Binární a kvalitativní metriky tedy měří odlišné aspekty téhož procesu a jejich kombinace se ukázala jako nutná.

Obrázek ?? shrnuje míru splnění exit kritérií procesních metrik napříč devíti běhy.



Obrázek 5.1: Míra splnění exit kritérií procesní metriky (P1–P8) napříč devíti běhy. P2 a P5 byly splněny nejméně často (3/9), P6 a P7 ve všech bězích. Deterministické položky (P1–P5) vykazují vyšší variabilitu než kvalitativní (P6–P8).

Kvalita kódu (Q1–Q8).

Na rozdíl od procesních metrik vykazují produktové metriky (Q1–Q8) celkově vyšší stabilitu: Q1 (API contract match) a Q6 (typecheck errors) byly splněny ve všech devíti bězích, a Q4 (AC coverage) dosáhla ve většině hodnocených běhů plného skóre 25/25. Data naznačují, že základní kvalita kódu je u tohoto modelu řízena převážně tréninkovými daty, zatímco procesní chování závisí na instrukcích výrazněji. Uvnitř skupiny se ale metriky chovají odlišně podle toho, co měří.

Korektnost (Q1, Q2). Q1 zůstala stabilní ve všech bězích: agent vždy vytvořil veřejné API odpovídající kontraktu, takže Q1 na této úloze nepřinesla rozlišení. Q2 (referenční test pass rate) je naopak nejvariabilnější produktová metrika (splněna v 6 z 9 běhů): opakovaně selhávaly testy pokrývající přechody pause/resume. Specifikace i API kontrakt jednoznačně vyžadují zachování elapsed time (pole `pausedElapsed`), ale agent toto pole opakovaně ignoroval. Kontrast mezi Q1 (binární, vždy splněna) a Q2 (poměr projitých testů, variabilní) ukazuje, že binární metrika nezachytí problémy které jemnější měření odhalí.

Testování (Q2–Q4). Tři metriky pokrývají testování z různých úhlů: Q2 (referenční test pass rate) měří kolik referenčních testů projde (vyšší = lepší implementace), Q3 (mutation score) měří jaký podíl injektovaných chyb agentovy testy odhalí (vyšší = kvalitnější testy) a Q4 (AC coverage) měří kolik acceptance criteria má odpovídající test (vyšší = úplnější pokrytí).

Vztah mezi Q2 a Q3 se ukázal jako nejinformativnější. V ablačním běhu B-2 agent dosáhl Q2 = 11/42. Jeho implementace byla z velké části chybná. Přesto Q3 = 72 %, tedy testy které napsal by v korektní implementaci zachytily většinu injektovaných chyb. Agent tedy uměl psát kvalitní testy, ale neuměl správně implementovat logiku. Kdyby sada obsahovala jen jednu z těchto metrik, tento rozdíl by zůstal skrytý.

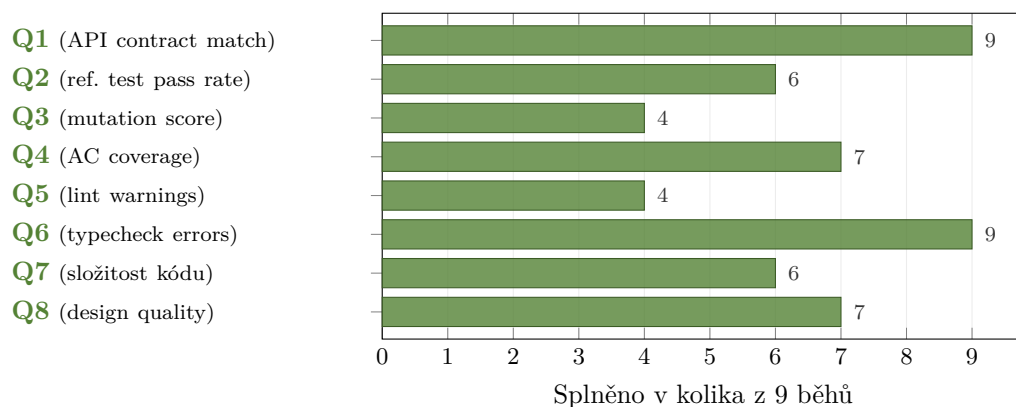
Q4 zůstala relativně stabilní: po manuálním doplnění AC25 dosáhla většina hodnocených

běhů plného skóre 25/25. Tato metrika ale měří pouze *přítomnost* testu na dané AC, ne jeho správnost: agent mohl napsat test který judge vyhodnotí jako pokrytí, ale který ve skutečnosti chování neověřuje. Kvalitu testů měří až Q3. Společně tyto tři metriky tvoří hierarchii: Q4 říká *co* agent testuje, Q3 *jak dobře*, a Q2 zda implementace skutečně funguje.

Čistota a design (Q5–Q8). Q5 (lint warnings) a Q7 (složitost kódu) se ukázaly jako citlivé na instrukce: v ablacích bez sekce Package Quality vzrostl počet lint warnings o přibližně 150 % a počet violations cyklomatické složitosti obdobně. Q6 (typecheck errors) naproti tomu zůstala nulová ve všech bězích. Statické typování poskytuje modelu explicitní vodítka přímo v kódu: typy parametrů, návratové hodnoty a rozhraní jsou součástí trénovacích dat a model je při generování dodržuje. U dynamicky typovaných jazyků by Q6 pravděpodobně rozlišovala lépe.

Q8 (design quality) doplňuje automatizované metriky o strukturální pohled na kód. Hodnocení provádí LLM-as-judge na základě rubriky s pěti dimenzemi: naming, separation of concerns, idiomatický TypeScript, dokumentace a složitost. V pilotních bězích Q8 opakovaně identifikovala chybějící dokumentaci: r1 a r2 dostaly 1/3 (žádné JSDoc), r4 a r5 dostaly 2/3 (dokumentace jen na hlavním API, ne na utility funkcích). Teprve po přidání explicitního verifikačního kroku do instrukcí (r3) agent dokumentaci doplnil (3/3). V ablačních bězích klesla navíc dimenze separation of concerns: agent soustředil kód do jednoho souboru. Automatizované metriky Q5–Q7 přitom žádný z těchto problémů nehlásily. Kombinace deterministické metriky a kvalitativní metriky tak zachycuje víc než kterýkoliv typ sám.

Obrázek ?? shrnuje míru splnění exit kritérií produktových metrik napříč devíti běhy.



Obrázek 5.2: Míra splnění exit kritérií produktové metriky (Q1–Q8) napříč devíti běhy. Q1 a Q6 byly splněny ve všech bězích; Q3 a Q5 nejméně často. Q3 počítáno ze 7 běhů (2 bez dat). Metriky efektivity (E1–E3) nemají exit kritérium a nejsou zahrnuty.

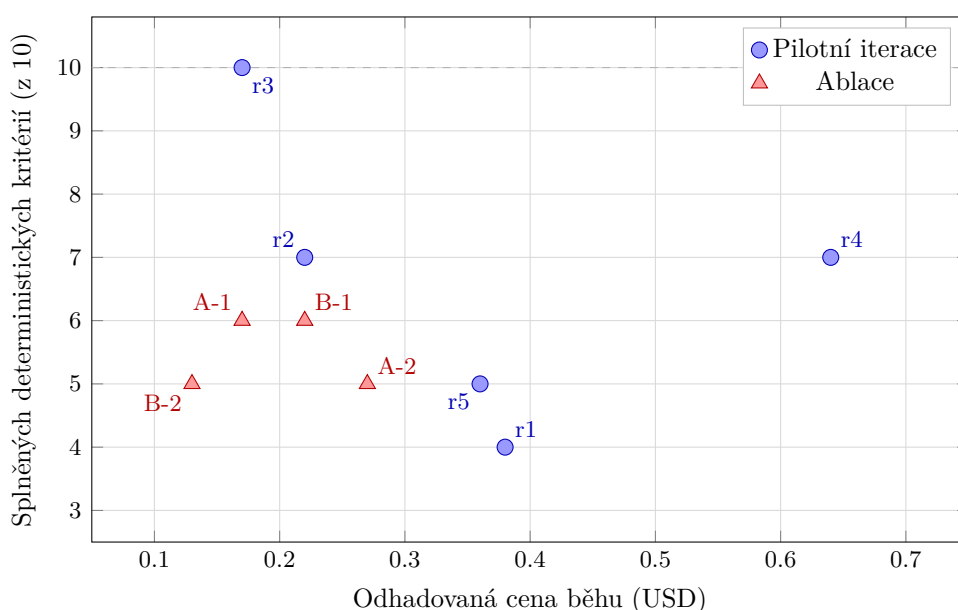
Efektivita (E1–E3).

E1 (vstup/výstup/cache tokeny) ukázala užitečný vzorec: bez verifikačních kroků agent spotřeboval přibližně dvakrát více testovacích cyklů, což se projevilo na vyšší spotřebě tokenů.

E2 (trvání) závisí převážně na rychlosti poskytovatele API (rate limits, latency), ne na kvalitě agentovy práce. Nejkratší běh (r5, 13 min) vznikl tím, že agent ignoroval celý pracovní postup. E2 je proto nutné číst společně s procesními metrikami.

E3 (stabilita session) na této úloze přinesla jen malé rozlišení: sessiony byly většinou dostatečně krátké na to, aby se vešly do kontextového okna. Pro rozsáhlejší projekty by E3 pravděpodobně rozlišovala lépe.

Při orientačním přepočtu na cenu (ceník poskytovatele Alibaba Cloud Model Studio) se ukázalo, že vyšší spotřeba tokenů nekoreluje s lepším výsledkem: nejlevnější běh (r3, přibližně \$0,17) dosáhl nejlepších metrik ($Q2 = 41/42$ a plné splnění P1–P5), zatímco nejdražší běh (r4, přibližně \$0,64) měl horší výsledky. Agent v r4 spotřeboval víc tokenů na cyklické debugování, ne na kvalitnější práci (obrázek ??).



Obrázek 5.3: Vztah mezi odhadovanou cenou běhu (E1) a počtem splněných deterministických kritérií (P1–P5, Q1, $Q2 \geq 41$, $Q5 \leq 1$, $Q6 = 0$, $Q7 \leq 1$). Běh r3 dosáhl maxima 10/10 při nejnižší ceně z pilotních iterací. Ablací běhy (trojúhelníky) se pohybují v rozmezí 5–6 splněných kritérií.

Shrnutí a reflexe.

Sada se ukázala jako proveditelná pro opakované měření: deterministické metriky se extrahují automatizovaně, kvalitativní metriky hodnotí LLM-as-judge s fixní rubrikou. Jedno kompletní měření trvá řádově minuty, což umožňuje zpětnou vazbu v rámci jedné iterace.

Kritický pohled na design sady odhaluje několik slabých míst. Tři metriky (Q1, Q6, E3) nepřinesly na této úloze žádné rozlišení. Jsou pojistkou pro jiné úlohy nebo jazyky, ale na devíti bězích zabíraly místo bez informačního přínosu. Procesní metriky přinesly nejvíc nových poznatků (nestabilita chování kterou produktové metriky nezachytily), ale jsou zároveň

nejméně spolehlivé: kolísají i mezi běhy se stejnými instrukcemi, takže u jednoho běhu jim nelze plně věřit. Binární P1–P5 navíc ztrácí nuance: “nesplněno” neříká *proč* a k diagnostice bylo vždy nutné vrátit se k transcriptu agenta. Kvalitativní metriky (Q4, Q8) zachytily problémy které automatizované nevidí, ale bez validace Cohenovým κ zůstávají podpůrnými indikátory. Metriky efektivity byly na této úloze nejslabší skupinou: pouze E1 přinesla použitelný insight.

[RAW]

Audit trail, předchozí verze 5.1.1 (DRAFT v1):

Procesní metriky jako hlavní přínos.

Stávající benchmarky (SWE-bench, HumanEval) měří funkční korektnost: projdou testy, tedy agent úlohu vyřešil. Procesní metriky P1–P5 měří *jak* agent pracoval: zda vytvořil issues, branche, testy před kódem, PR. Tuto dimenzi žádný z existujících benchmarků nepokrývá. Na naší případové studii se ukázala jako informativní: běh r3 dosáhl Q2 = 41/42 (funkčně téměř bezchybný), ale procesní metriky v následujících bězích r4 a r5 se stejnými instrukcemi odhalily, že dodržování pracovního postupu je nestabilní. Bez procesních metrik by tyto běhy vypadaly srovnatelně; s nimi vidíme podstatný rozdíl v chování agenta.

Kvalitativní metriky.

Metriky kvality kódu (Q3–Q8) rozšiřují binární pohled benchmarků na hlubší analýzu. Q8 (design quality, LLM-as-judge) identifikovalo chybějící modularitu a dokumentaci v ablačních bězích, přestože automatizované metriky (Q5, Q6, Q7) žádný problém nehlásily. Kombinace automatizovaných a kvalitativních metrik zachycuje víc než kterýkoliv typ sám.

Stabilní a variabilní metriky.

Tři metriky zůstaly stabilní napříč všemi devíti běhy: Q1 (API contract match = shoda), Q6 (typecheck errors = 0) a E3 (žádná kompakce kontextu). U Q1 a Q6 lze nabídnout hypotézu proč: Q1 naznačuje, že model z tréninku zná konvenci veřejného API natolik, že ji dodrží i bez explicitní instrukce. Q6 pravděpodobně souvisí s tím, že statické typování v TypeScriptu je pro LLM explicitnější než dynamické jazyky; kompilátor odmítne nevalidní kód dříve, než se projeví v metrice. Alternativním vysvětlením je, že obě metriky mají příliš hrubý práh: Q1 je binární (match/no match, nezachytí drobné odchylky v pojmenování), Q6 počítá chyby kompilace (nezachytí problematické `any` typy). Jemnější měření, například srovnání názvů metod s kontraktem, nebo podíl explicitních typů, by mohlo odhalit rozdíly, které binární metriky nezachytí.

Nejvyšší variabilitu vykazovaly procesní metriky P2–P5. Kolísaly nejen mezi iteracemi s růz-

nými instrukcemi, ale i mezi běhy se stejnými instrukcemi (r3: 5/5 vs. r4: $\sim 3/5$ vs. r5: $\sim 1/5$). Tato variabilita pravděpodobně neodráží slabinu metrik, ale podstatnou vlastnost měřeného systému: dodržování vícekrokového pracovního postupu je u tohoto modelu nedeterministické. Metriky tento fenomén zachytily, což umožnilo diagnostiku v cíli 2.

Proveditelnost a limity.

Sada se ukázala jako proveditelná pro opakované měření. Deterministické metriky se extrahují automatizovaně, kvalitativní metriky hodnotí LLM-as-judge s fixní rubrikou. Jedno kompletní měření trvá řádově minuty, což umožňuje zpětnou vazbu v rámci jedné iterace. Při dalším použití by stálo za úvahu přidat jemnější škálu pro Q1 a Q6 (viz výše) a rozšířit procesní metriky o transparentnost agentova reasoning procesu: naše P1–P8 měří pozorovatelné artefakty (commity, issues, PR), ale ne kvalitu rozhodovacího procesu, který k nim vedl.

[RAW]

TODO (2026-03-26):

- **Přeučení na metriky (overfitting):** Iterativní postup optimalizuje instrukce na fixní sadu metrik. Hrozí, že instrukce se “přeučí” na konkrétní metriky místo aby zlepšily obecné chování. Příklad: přidání `npx eslint` zlepší Q5, ale neznamená že agent píše lepší kód; jen ho zkontroluje. Kam patří: future work nebo limity? Zvážit.
- **E3 (compaction):** HOTOVO. Snapshot heuristika nahrazena přímým dotazem do OpenCode DB (`time_compacting`). Kompakce nenastala v žádném běhu.
- **Benchmark research (HOTOVO):** Žádný benchmark neměří všechny 3 dimenze (proces + kvalita + efektivita) společně. Doplnit do 5.1.1 nebo do porovnání s literaturou (5.2):
 - **METR (2026):** polovina SWE-bench passing PRs by nebyla merged. Přímou validuje náš argument že pass/fail nestačí. Citovat!
 - **GitGoodBench (2025, JetBrains):** měří kvalitu commitů (LLM-as-judge); nejbližší našim P6–P8, ale pokrývá jen git operace, ne celý SDLC.
 - **RACE (Zheng 2024):** readability, maintainability, efficiency; blízké našim Q5–Q8, ale function-level.
 - **DevBench (Li 2024):** 5 SDLC fází včetně design quality; částečně proces, ale implementation je stále pass/fail.
 - **MAESTRO (2026):** execution traces, run-to-run variance; transparentnost procesu, ale ne pro kódování.
 - **Terminal-bench:** jen pass/fail, neměří proces.
- **Kam doplnit:** METR + GitGoodBench do kap02 (gap argument ; i když pass/fail projde, kvalita nemusí stačit). RACE, DevBench do kap02 (existující multi-dim benchmarky). Do 5.2 porovnání naší sady s těmito benchmarky.

Audit trail, klíčové rozhodnutí 5.1.1 (2026-03-26):

Předchozí verze (DRAFT v1, viz audit trail výše) míchala dvě věci: prezentaci výsledků (hodnoty metrik z běhů) a diskuzi o designu sady. Hodnoty patří do kap04, v 5.1.1 diskutujeme co jsme se dozvěděli o **metrikách samotných**: jejich design, co funguje, co ne, co bychom změnili. Konkrétní čísla z běhů jen jako ilustrace, ne jako narování tabulky.

Rozlišení: kap04 = co jsme naměřili; kap05 5.1.1 = co jsme se dozvěděli o designu metrik.

Struktura 5.1.1 sleduje 3 dimenze z cíle 1 (proces, kvalita, efektivita) + limity.

TODO, úpravy mimo kap05:

- **CLAUDE.md**: odstranit reference na DSR terminologii (artefakt, build-evaluate), ponechat ML styl.
- **kap03 metriky efektivity**: příliš technický odstavec (token výpočty), patří do kap04/přílohy. Odkaz na pilot-r1 nepatří do metodiky (TODO již přidáno).
- **kap03 Fenton**: upřesnit že Fentonova taxonomie (Process/Product/Resource) je z doby před LLM; naše adaptace (P/Q/E) přidává LLM-as-judge metriky které Fenton neměl. Zmapovat kde přesně mapujeme a kde rozšiřujeme.

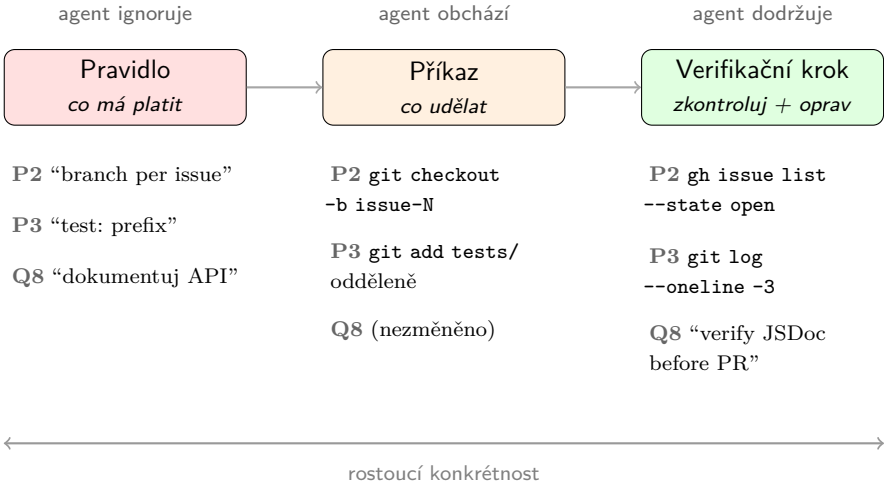
5.1.2 Cíl 2: Iterativní postup

Na případové studii demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami a vyhodnotit, zda a jak vede k měřitelným změnám v dodržování vývojových praktik.

Případová studie ukazuje, že tento postup lze na jednom projektu prakticky použít a že může vést k měřitelnému zlepšení, nikoli však že stejný efekt nastane obecně. Postup vedl k měřitelnému zlepšení, ale ne monotónnímu. Baseline (r1) splnila čtyři z deseti tvrdých deterministických procesních a produktových kritérií. Po dvou cyklech úprav (r3) agent splnil sedm z deseti. Regrese r4 (pět z deseti) ukázala, že zlepšení není trvalé: po neúspěšné iteraci bylo nutné vrátit se k předchozí verzi instrukcí (r5 vycházel z r3, ne z r4).

Metriky poskytly vodítka kam zasáhnout: tabulka metrik ukázala která kritéria agent nesplnil a behaviorální popis z **FINDINGS.md** popsal co agent konkrétně udělal. K pochopení *proč* agent zvolil daný postup však bylo většinou nutné nahlédnout i do transcriptu agenta. Diagnostika se tak opírala o kombinaci kvantitativních dat a kvalitativní analýzy chování.

Analýza průběhu iterací odhalila opakující se vzorec na spektru operacionalizace instrukcí. Obrázek ?? ukazuje, jak se tři nezávislá selhání vyvíjela přes stejné fáze: pravidlo (agent ignoruje) → příkaz (agent obchází) → verifikační krok (agent dodržuje).



Obrázek 5.4: Spektrum operacionalizace instrukcí. Tři nezávislá selhání (P2, P3, Q8) prošla stejnými fázemi: pravidlo (agent ignoruje) $\rightarrow$ příkaz (agent obchází) $\rightarrow$ verificační krok (agent dodržuje).

Tento vzorec se zopakoval u všech tří selhání. Tři nezávislá selhání v rámci jedné případové studie prošla stejnými fázemi. Tuto opakující se strukturu interpretujeme v režimu analytické generalizace (R. K. Yin, 2018): jde o pozorovanou pravidelnost uvnitř jednoho případu, nikoli o obecnou zákonitost. R4 řešilo dosud neadresovaný problém Q2 přidáním pravidla do Constraints (“every field in API contract must be used”), tedy nejslabšího typu instrukce na spektru. Výsledek byl regrese. Data jsou konzistentní s hypotézou, že verifikační kroky jsou účinnější než pravidla, ale při pěti bězích to nelze statisticky potvrdit.

Postup měl i slepá místa. Q2 zůstalo nesplněné ve všech pěti běžích (maximum 41/42): agent opakovaně implementoval pause/resume posunem timestampu místo použití pole `pausedElapsed` z API kontraktu. Pravděpodobným vysvětlením je, že posun timestampu je v trénovacích datech častější pattern než explicitní pole pro elapsed time. Navíc název `pausedElapsed` neříká že hodnota musí přetrvávat ve stavu i po resume. Úpravou instrukcí se tento problém nepodařilo adresovat. Jde o implementační rozhodnutí na úrovni kde instrukce typu pravidlo ani verifikační krok nestačí. Podobně nelze vyloučit, že některé regrese v r4 byly vedlejším efektem úprav cílených na jiné metriky. Nedeterminismus modelu a malý počet běhů neumožňují odlišit kauzální efekt úpravy od náhodného šumu.

Z hlediska praktické proveditelnosti: každá iterace trvala 25–40 minut (běh agenta) plus řádově desítky minut na diagnostiku a úpravu instrukcí. Diagnóza se opírala o tabulku metrik, behaviorální popis z `FINDINGS.md` a analýzu transcriptu agenta. Zjištěný vzorec na spektru operacionalizace (pravidlo $\rightarrow$ příkaz $\rightarrow$ verifikační krok) považujeme za jeden z hlavních poznatků práce, ale vychází z jedné případové studie s jedním modelem. Jeho platnost pro jiné modely a typy úloh vyžaduje další výzkum.

5.1.3 Cíl 3: Ablace

Z instrukcí vytvořených v cíli 2 prozkoumat ablacemi, které složky přispívají k měřenému chování agenta a které jsou redundantní.

Sekce ?? shrnuje výsledky obou ablací. Verifikační kroky (ablace A) přispívají: bez nich klesly metriky kvality i korektnosti. Sekce Package Quality (ablace B) je pro automatizované metriky převážně redundantní: Q5, Q6 a Q7 zůstaly na srovnatelné úrovni. Pouze Q8 zachytila pokles v modularitě a dokumentaci.

Co tyto výsledky znamenají pro cíl 3? Ablace rozlišily dva typy složek instrukcí. V této případové studii mají verifikační kroky zjevný instrumentální přínos: bez explicitní instrukce agent kontrolní příkazy nespouštěl spolehlivě. Kódové konvence agent dodržuje i bez instrukce, pravděpodobně ze znalostí z tréninku. Instrukce ale ovlivňují strukturální rozhodnutí (modularita, dokumentace) která automatizované metriky nezachytí. Dimenze efektivity byla v ablacích vyhodnocena jen částečně: E1 nešlo kvůli chybějícím exportům spočítat ani v jednom ablačním běhu, takže závěry ablační fáze stojí primárně na procesní metriky (P1–P8) a produktové metriky (Q1–Q8).

Variabilita procesních metrik byla v ablacích srovnatelná s pilotními iteracemi, přestože instrukce pro workflow zůstaly beze změny. Data naznačují, že procesní compliance je u tohoto modelu stochastická vlastnost, kterou instrukce ovlivňují jen částečně. Variabilita mezi B-1 ($Q2 = 37/42$) a B-2 ($Q2 = 11/42$) ukazuje, že nedeterminismus modelu může být silnějším faktorem než efekt ablace. Při dvou bězích per variaci nelze oba faktory spolehlivě odlišit.

Reflexe designu ablací.

Vzorec operacionalizace (pravidlo → příkaz → verifikační krok), identifikovaný v cíli 2, jsme pojmenovali až při analýze pilotních dat. Výběr ablací mu předcházal. Zpětně je zřejmé, že u ablace A byl pokles metrik do značné míry předvídatelný: `npx eslint` a `git log` jsme do instrukcí přidali v pilotních iteracích právě proto, že bez nich metriky selhávaly. Novým poznatkem byla dvojí funkce verifikačních kroků: kromě kontroly kvality strukturují práci agenta a zabraňují cyklickému debugování (sekce ??). Informativnější by byla ablace Constraints nebo workflow struktury, kde výsledek z pilotů předvídatelný není. To zůstává námětem pro další výzkum.

5.2 Porovnání s literaturou

5.2.1 Srovnání metrického rámce

Literatura shrnutá v kapitole ?? ukazuje, že hodnocení AI coding agentů je stále silně orientováno na funkční výsledek. SWE-bench a jemu podobné benchmarky jsou postaveny především na pass/fail logice. Právě proti tomuto omezení vystupují novější empirické práce. METR (METR, 2026) ukazuje, že část patchů, které projdou testy, by nebyla přijata při code review. Li et al. (B. Li et al., 2026) dokládají, že ani sloučený pull request negarantuje udržitelný kód. Ehsani et al. (Ehsani et al., 2026) ukazují, že neúspěch agentních PR není jen otázkou chyb v implementaci, ale i procesních selhání.

Naše sada metrik je s těmito zjištěními konzistentní. Funkční korektnost zůstává nutnou složkou hodnocení, ale sama nestačí. V naší případové studii právě procesní metriky ukázaly informaci, kterou by výsledkové benchmarky samy nezachytily: běhy se stejnými nebo podobnými produktovými výsledky se lišily v tom, zda agent vytvořil auditovatelný pracovní postup. Produktové metriky naopak ukázaly, že ani kvalitní procesní artefakty samy nezaručují správnou implementaci. Přínos rámce P/Q/E tedy nespočívá jen v rozšíření počtu měřených veličin, ale v tom, že umožňuje sledovat různé druhy selhání odděleně a pak je znovu spojit při interpretaci.

Odlišnost vůči existujícím benchmarkům je i v tom, že metriky zde nejsou jen evaluačním výstupem, ale i řídicím nástrojem pro další iteraci. To je důležité pro hlavní problém práce: nestačí vědět, že agent selhal nebo uspěl, pokud z výsledku neplyne, jak instrukce upravit. V tomto smyslu je náš rámec bližší praktickému diagnostickému nástroji než klasickému benchmarku.

5.2.2 Vliv instrukcí na chování agenta

Studie o instrukčních souborech zatím většinou měří jejich přítomnost nebo absenci jako celek. Lulla et al. (Lulla et al., 2026) spojují AGENTS.md s nižším mediánovým runtime, zatímco Gloaguen et al. (Gloaguen et al., 2025) ukazují jen marginální přínos generických repository-level souborů a upozorňují na vyšší náklady. SkillsBench (X. Li et al., 2025) dále rozlišuje mezi kurátorovanou a automaticky generovanou instrukční podporou a ukazuje, že rozhodující je kvalita a relevance obsahu. Naše výsledky jsou s těmito zjištěními kompatibilní: ne všechny složky instrukčního souboru přispívají stejně.

Nejvýraznější efekt měly v naší studii verifikační kroky. Ablace A ukázala pokles jak ve funkční korektnosti, tak v automatizovaných metrikách kvality kódu, když byly explicitní kontroly odstraněny. To doplňuje pozorování Breuniga (Breunig, 2025), že opakování ignorované instrukce nepomáhá a účinnější je její přestrukturování do pracovního postupu. Pilotní iterace v našich datech ukázaly stejný vzorec: obecné pravidlo nestačilo, konkrétní příkaz pomohl jen částečně

a stabilnější zlepšení přinesl až verifikační krok. Tento vývoj je konzistentní i s výzkumem specifity instrukcí, podle něhož vyšší konkrétnost pomáhá zejména u procedurálních úloh, ale přílišný detail může omezovat flexibilitu modelu (Kim, 2025; Zi et al., 2025).

Sekce Package Quality se chovala odlišně. Její odebrání nezhoršilo automatizované metriky stejným způsobem jako odebrání verifikačních kroků, ale projevilo se v designové kvalitě hodnocené LLM-as-judge. Data jsou proto konzistentní s tím, že část instrukcí nefunguje primárně jako přímé vynucení chování, ale jako rámec, který aktivuje latentní znalosti modelu. V tomto ohledu je užitečná analogie s promptingem: minimální nápověda může vyvolat žádoucí chování i bez detailního skriptu (Min et al., 2022; J. Wei et al., 2022). V naší studii tak vznikají dvě funkce instrukcí: verifikační kroky vynucují měřitelné požadavky, zatímco obecnější konvence podporují designová rozhodnutí.

Zároveň ale platí, že vliv instrukcí nelze od nedeterminismu modelu oddělit dokonale. Razavi a Fard (Razavi & Fard, 2025) upozorňují na vysokou citlivost modelů na drobné změny promptu a naše pilotní i ablační běhy tento problém potvrzují. Proto je třeba naše výsledky číst jako indikativní syntézu na úrovni případové studie: ukazují, které typy instrukcí se v tomto prostředí osvědčily, ale neprokazují univerzální kauzální zákon.

5.3 Omezení a jejich dopad

Sekce ?? identifikovala metodologická omezení designu studie předem. Zde ukazujeme, jak se tato omezení projevila v naměřených datech, a popisujeme omezení která se objevila až v průběhu experimentu. Omezení řadíme podle toho, co ohrožují: nejprve sílu závěrů, pak spolehlivost měření a nakonec rozsah platnosti výsledků.

Nedeterminismus a prompt sensitivity. Pět binárních procesních metrik P1–P5 (splněno/nesplněno) kolísalo od 5/5 (r3) přes $\sim 3/5$ (r4) po $\sim 1/5$ (r5), přestože procesní instrukce v těchto bězích zůstaly stejné. R4 přidalo dva řádky do Constraints a procesní compliance se zhoršila, přestože žádná změna přímo necílila na workflow. Razavi a Fard (Razavi & Fard, 2025) tento jev popisují jako prompt sensitivity. Alternativním vysvětlením je efekt kumulace instrukcí: s rostoucím počtem instrukcí klesá pravděpodobnost dodržení všech současně. Instrukce rostly z r1 (baseline) do r5. Část poklesu compliance může být tímto efektem, ne jen citlivostí na formulaci.

Dva běhy na ablační variaci tento problém zmírňují, ale neposkytují statistickou sílu. Pokud oba běhy téže ablace ukazují stejný směr, je pravděpodobnější že jde o efekt ablace, ne o náhodu. Například Q5 (lint warnings) v ablaci A: oba běhy hlásily 4 a 3 lint warnings, zatímco r3 (plné instrukce) hlásil 1. Shoda obou běhů zvyšuje důvěru, že odebrání sekce Process skutečně ovlivnilo Q5. Naopak Q2 (referenční test pass rate) v ablaci B: jeden běh dosáhl 37/42 projitých testů, druhý jen 11/42. Tak velký rozptyl mezi dvěma běhy se stejnými instrukcemi znamená, že rozdíl oproti r3 mohl být způsoben nedeterminismem modelu, ne ablací. Výsledky ablací proto interpretujeme jako indikace, ne jako statisticky podložené

závěry. V kombinaci s vedlejšími efekty popsanými výše (r4) to znamená, že každá úprava instrukcí vyžaduje měření celé sady metrik, ne jen těch na které úprava cílila.

Efekt učení autora. Zlepšení mezi iteracemi (r1: 4/10 deterministických kritérií, r3: 7/10) nemusí plynout jen z lepších instrukcí. Autor se v průběhu iterací učil diagnostikovat selhání a formulovat opravy. Tento efekt nelze od efektu instrukcí oddělit. Diagnostická fáze (výběr příčiny selhání a návrh opravy) závisí výhradně na autorovi a nezávislé ověření nebylo provedeno. Částečnou mitigací je použití strukturovaného diagnostického rámce (sekce ??): diagnóza se opírala o naměřená data a literaturou podložené principy, ne o intuici. Přesto nelze vyloučit, že zkušenější autor by z téhož baseline dosáhl lepších výsledků rychleji.

Změny konfigurace mezi běhy. Běhy r1–r2 proběhly bez kontejnerové izolace, od r3 běží agent v Docker kontejneru. Od r4 byla změněna konfigurace modelu (parametry kontextového okna). Tyto změny jsou potenciálním zavádějícím faktorem: regrese r4 mohla být částečně způsobena změnou konfigurace, ne pouze přidáním dvou řádků do Constraints. Hlavní proměnné (model, instrukce, specifikace) zůstaly konzistentní. Temperature modelu nebyla explicitně nastavena. Experiment používal výchozí konfiguraci poskytovatele.

Tiché ukončení nástroje. Při ablačních bězích se OpenCode v některých případech tiše ukončil uprostřed práce agenta. Běhy s vyšší spotřebou kontextového okna (ablace A: ~44 spuštění testů oproti ~20 u ablace B) byly postiženy častěji. Postižené běhy byly opakovány. Pravděpodobnou příčinou je běh agenta uvnitř Docker kontejneru (spojení s API poskytovatele, limity paměti nebo síťové timeouty), ale příčina nebyla potvrzena.

Nerozlišující metriky. Q1 (API contract match), Q6 (typecheck errors) a E3 (stabilita session) nepřinesly na této úloze žádné rozlišení. Q1 byla splněna a Q6 zůstala nulová ve všech devíti bězích. E3 nezaznamenala žádnou kompakci kontextu. Pravděpodobným vysvětlením je design úlohy: specifikace poskytuje explicitní TypeScript typy (Q1, Q6) a úloha je dostatečně malá na to, aby se kontext nezaplnil (E3).

Kvalitativní metriky bez validace. Hlavní závěry studie stojí na deterministické metrice. kvalitativní metriky slouží jako podpůrné indikátory, protože jejich spolehlivost nebyla validována proti lidskému hodnocení (Cohenovo κ). Jedinou z nich, na které stojí netriviální závěr, je Q8 (design quality): jako jediná metrika zachytila pokles designové kvality v ablacích, který deterministické metriky nehlásily. Bez κ validace ale nelze sílu tohoto závěru plně posoudit.

Kontrolované podmínky vs. reálný vývoj. Experiment probíhal v kontrolovaném prostředí: prázdný repozitář, jedna úloha s kompletní specifikací, fixní sada metrik. Reálný vývoj zahrnuje nejednoznačné požadavky, měnící se specifikace a integraci s existujícím kódem. Instrukce optimalizované pro kontrolované podmínky (verifikační kroky, explicitní příkazy) nemusí fungovat v prostředí kde specifikace není kompletní nebo se mění v průběhu práce.

Jeden model, jeden nástroj, jeden projekt. Omezení plynoucí z jednoho modelu, jednoho projektu a deterministické specifikace diskutuje sekce ?? . V praxi se tato omezení projevila zejména u procesní metriky (P1–P8): variabilita neumožňuje odlišit efekt instrukcí od nede-

terminismu modelu bez většího počtu běhů. Výsledky jsou navíc specifické pro OpenCode a model MiniMax-M2.5. Jiné nástroje (GitHub Copilot, Cursor, Claude Code) mohou zpracovávat instrukční soubory odlišně. Zda by instrukce fungovaly stejně na projektech s vyšší nejednoznačností specifikace, na jiných modelech nebo s jinými nástroji, zůstává otevřenou otázkou (sekce ??).

[RAW]

Audit trail (DRAFT v3, 2026-04-04): 3 skupiny podle dopadu: (1) co oslabuje závěry (nedeterminismus, efekt učení autora, konfigurace, tiché ukončení), (2) co omezuje měření (nerozlišující metriky, κ), (3) kde výsledky neplatí (kontrolované podmínky, generalizovatelnost+tool-specific). Oproti v2: odstraněn “chybějící baseline bez instrukcí” (ablace testují složky, ne přítomnost vs nepřítomnost; legitimní design), confirmation bias sloučen do efektu učení (diagnostika závisí na autorovi), E1 orientační a AGENTS.md v transcriptu odstraněny (implementační detail), infrastrukturní nedostatky P7+Q6 odstraněny (opraveny v kódu), Q4 rubrika opravena (24→25 AC, manuálně ověřeno).

Audit trail (DRAFT v1, 2026-03-26): Předchozí verze měla 8 samostatných témat (Nedeterminismus, Prompt sensitivity, Diagnostická chyba, ESLint disconnect, κ , Infrastruktura, Prostředí, Tiché ukončení, Generalizovatelnost). Sloučeno do 6 bloků, pak do 3 skupin. Odstraněny: detaily E1 výpočtu, duplicitní odkaz na kap03, ESLint config disconnect, diagnostická chyba výzkumníka (data opravena), infrastrukturní nedostatky P7 a Q6 (opraveny v kódu).

Doporučení k validaci (strukturální audit, 2026-04-22): Znovu zkontrolovat rozhraní mezi sekci 3.5 a touto sekci. Aktuálně se v obou vrstvách vracejí podobná témata: nedeterminismus, validita judge-based metrik, Q4 reporting, změny prostředí a generalizovatelnost. Pokud se potvrzuje překryv, 3.5 má nést předem deklarované hrozby designu a tato sekce jen jejich konkrétní dopad na interpretaci výsledků v datech. Riziko současného stavu: rozmazaný chapter contract mezi metodikou a diskusí.

5.4 Doporučení pro praxi

Následující doporučení vycházejí z pilotních iterací a ablací popsaných v kapitole ??. Konkrétní čísla a prahy platí pro tento model a typ projektu. Přenositelný je iterační postup (měř → diagnostikuj → uprav → měř znovu) a principy strukturování instrukcí.

Operacionalizace instrukcí: verifikovat, nikoli jen vyslovit.

V pilotních iteracích se opakoval vzorec: obecné pravidlo (“každé pole musí být použito”) agent ignoroval, zatímco explicitní příkaz s okamžitou zpětnou vazbou (`npx eslint, tsc --noEmit`) dodržoval spolehlivě i v bězích, kde ignoroval zbytek pracovního postupu (sekce ??). Ablace

potvrdila komplementární stranu: odstranění verifikačních kroků (ablance A) vedlo k poklesu Q2 (referenční test pass rate) z 41/42 na 35–37/42 a nárůstu Q5 (lint warnings) z 1 na 3–4 varování. Breunig (Breunig, 2025) formuloval stejný princip: když agent instrukci ignoruje, nepomůže ji zopakovat; je třeba ji převést na konkrétní akci. Spektrum operacionalizace (obrázek ??) popisuje, po jaké ose se intervence pohybují: čím blíže je instrukce verifikovatelnému příkazu, tím spolehlivěji ji agent v této případové studii dodržoval.

Ablance B ale ukázala i druhou stránku. Odstranění sekce Package Quality ponechalo automatizované metriky (deterministické metriky) beze změny, zatímco Q8 (design quality) kleslo z 3/3 na 2/3 (sekce ??). Konvence zřejmě nepůsobí jako přímé vynucení, ale spíš jako *aktivace* latentních znalostí modelu, tedy jako připomínka toho, co se model v tréninkových datech naučil. Z toho plyne praktické rozlišení: měřitelné požadavky převést na verifikovatelné kroky v pre-PR checklistu, zatímco požadavky, které nelze měřit deterministicky (srozumitelnost dokumentace, vhodnost dekompozice), strukturovat jako nápovědu, která modelu připomíná žádoucí směr. Účinnost deterministické větve přitom závisí na existenci nástroje s jednoznačným výstupem. Tam, kde takový nástroj chybí, je míra operacionalizace principiálně nižší (sekce ??). Závěr o designové kvalitě stojí na Q8, tedy na podpůrné judge-based metrice bez κ validace, a je proto nutné jej číst opatrně.

Počítat s nedeterminismem a měřit proces.

Agent dodržoval verifikační příkazy deterministicky, ale vícekrokový pracovní postup (issue → branch → test → PR) nedeterministicky: pilotní běh r3 dosáhl P1 (issues before code) = 5/5, běh r5 při téměř identických instrukcích P1 = ~1/5 (sekce ??). Na úrovni výsledku (Q2) přitom rozdíl nemusel být patrný. procesní metriky (P1–P8) zachytily problém, který produktové metriky (Q1–Q8) samy o sobě neodhalily. Doporučení: nespoléhat na jednorázové ověření výstupu, zavést měřitelná exit kritéria pokrývající proces i kvalitu a kontrolovat je opakovaně.

Iterovat na základě dat.

Přínos této práce nespočívá v konkrétní instrukční sadě, ale v postupu jejího návrhu: naměřit metriky, diagnostikovat příčinu selhání, provést cílenou úpravu instrukcí a změřit znovu (sekce ??). Tento cyklus je přenositelný nezávisle na modelu a projektu. Podmínkou je sada metrik pokrývajících více dimenzí (proces, kvalitu, efektivitu), protože jednodimenzionální měření (např. jen test pass rate) může zakrýt problém v jiné dimenzi (sekce ??).

Praktický důsledek je přímočarý: místo delšího a obecnějšího instrukčního souboru je účelnější budovat kratší, měřitelný pracovní postup s jasnými verifikačními body a opakovaně ho ladit podle dat.

[RAW]

Audit trail (DRAFT v1, 2026-03-26): 5 doporučení: (1) verifikační kroky klíčové, (2) konkrétní instrukce > obecná pravidla, (3) konvence redundantní, (4) počítat s nedeterminismem, (5) minimální efektivní instrukce.

Audit trail (DRAFT v2, 2026-04-04): Sloučeno 1+2 do operacionalizace; 3 přeformulováno (verifikace vs aktivace, ne “redundantní”); 5 nahrazeno blokem o iteraci na základě dat; doplněn disclaimer o přenositelnosti (postup ano, prahy ne); doplněny konkrétní hodnoty z ablací; odkaz na kap02 operacionalizační spektrum.

Doporučení k validaci (strukturální audit, 2026-04-22): Ověřit, zda sekce 5.4 přináší nový výstup pro praktika, nebo jen třetí průchod závěry z 5.1 a 5.2. Pokud se potvrzuje druhá možnost, sekci zkrátit na několik skutečně akčních doporučení a odstranit pasáže, které jen znovu interpretují verifikační kroky, aktivaci vs. verifikaci nebo nedeterminismus bez nové vrstvy rozhodnutí.

5.5 Náměty pro další výzkum

Případová studie otevřela několik otázek, které přesahují rozsah této práce.

Nedeterminismus více krokového pracovního postupu.

Agent dodržoval verifikační příkazy (jednokrokové akce) deterministicky, ale více krokový pracovní postup (issue → branch → test → implement → PR) nedeterministicky: r3 kompletně, r5 vůbec, při téměř identických instrukcích. Možná vysvětlení zahrnují ztrátu pozornosti v dlouhém kontextovém okně, stochastickou volbu mezi strukturovaným a nestrukturovaným vývojem z tréninku modelu a pozici instrukce v kontextu. Granulární ablace pracovního postupu (které kroky jsou kritické?) a opakované běhy se stejnými instrukcemi by tento fenomén pomohly lépe pochopit.

Kontinuita práce mezi agenty.

procesní metriky (P1–P8) měří transparentnost artefaktů (issues, commit zprávy, PR popisy). Otevřená otázka: stačí tato transparentnost k tomu, aby jiný agent (nebo člověk) navázal na rozpracovaný projekt? Testování handoff scénářů by ověřilo, zda procesní instrukce vytváří nejen pozorovatelný, ale i přenositelný pracovní kontext.

Automatizace iteračního postupu.

Fáze Diagnóza a Úprava jsou v této práci prováděny manuálně autorem. Frameworky jako PromptWizard (Agarwal et al., 2024) a Prompt Alchemy („Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation“, 2025) ukazují, že cyklus analýzy selhání a návrhu úprav lze automatizovat. Kombinace automatizovaného prompt optimizeru s metrikami P/Q/E by umožnila rychlejší iteraci bez manuální diagnózy.

Ingestion checkpoint.

Pilotní data ukazují, že agent instrukce přečte (v transcriptu cituje jejich obsah), ale ne vždy je aktivuje jako řídicí smyčku běhu. Místo pracovního postupu z `AGENTS.md` použije vlastní postup odvozený ze specifikace. Ověřitelný checkpoint na začátku běhu (agent musí explicitně potvrdit přečtení a aktivaci pracovního postupu před prvním zápisem kódu) by mohl tento problém adresovat.

Mechanismy účinku instrukcí a unfaithful reasoning.

[RAW]

TODO (2026-03-30, aktualizováno): Přepsat tento odstavec. Tři otevřené otázky přesunuté z 5.2:

1. **Proč konvence ovlivňují Q8?** Hypotéza: fungují jako nápověda aktivující latentní znalosti modelu (analogie CoT: Wei et al. 2022, Min et al. 2022). Pro emergentní chování (architektura, design) které nelze verifikovat deterministicky může být nápověda jediný dostupný nástroj. Ale nemáme data.
2. **Proč pravidla nefungují?** Hypotézy: (a) kumulace instrukcí snižuje compliance (Curse of Instructions 2024: $P(\text{all}) = P(\text{single})^N$), (b) unfaithful reasoning (Turpin et al. 2023, Lanham et al. 2023: agent zpracuje instrukci v CoT ale nekoná podle ní). Izolovat PROČ = future work.
3. **Závisí optimum na modelu?** Lepší modely možná potřebují méně specifické instrukce (expertise reversal, Kalyuga 2003).

Zdroje k doplnění do kap02: Turpin (arXiv:2305.04388), Lanham (arXiv:2307.13702), Curse of Instructions (OpenReview 2024).

Doporučení k validaci (strukturální audit, 2026-04-22): Sekce 5.5 pravděpodobně míchá tři různé úrovně: čisté research questions, metodická rozšíření studie a návrhy nových instrukčních mechanismů typu ingestion checkpoint. Před dalším prepisem ověřit, zda každá položka opravdu patří do “dalšího výzkumu” a není spíš praktickým návrhem, limitation nebo

spekulativní interpretací. Zvláště tento odstavec je kandidát na zúžení, protože stojí více na analogiích a hypotézách než na pevném napojení na data práce.

Naše data naznačují dva odlišné mechanismy: *vy nucení* (verifikační kroky přímo kontrolují výstup) a *aktivace* (obecné konvence připomínají modelu znalosti z tréninku). Analogie existuje v prompt engineeringu: chain-of-thought prompting ukazuje, že minimální nápověda může aktivovat komplexní latentní schopnost modelu efektivněji než detailní instrukce. Systematický experiment s odstupňovanou specificitou instrukcí, od nápověd (“piš modulární kód”) přes pravidla (“každé pole musí být použito”) po verifikační kroky (“spust `tsc`, oprav chyby”), by mohl identifikovat, která míra konkrétnosti je pro který typ požadavku optimální. Otevřená je i otázka, zda se toto optimum posouvá se schopnostmi modelu. Lepší modely by mohly potřebovat méně specifické instrukce (analogicky k expertise reversal effect v kognitivní psychologii, kde instrukce účinné pro nováčky zpomalují experty).

Generalizovatelnost verifikačních kroků.

Účinnost verifikačních kroků v naší studii závisela na existenci deterministických nástrojů (`eslint`, `tsc`, test runner), které produkují jednoznačný výstup zpět do kontextu agenta. Kódování je v tomto ohledu specifická doména: většina požadavků na kvalitu má odpovídající nástroj. U jiných typů úloh (dokumentace, architektonická rozhodnutí, plánování) takové nástroje neexistují a operacionalizace se zastaví na úrovni pravidla nebo příkazu bez externího feedbacku. Otevřená otázka je, zda principy operacionalizace platí i mimo kódování, nebo zda je jejich účinnost podmíněna dostupností deterministické zpětné vazby.

Rozšíření na další modely a projekty.

Výsledky platí pro jeden model (MiniMax-M2.5) a jeden typ projektu (deterministická business logika). Replikace s jinými modely, nedeterministickými doménami (UI, strojové učení) a většími projekty by ukázala, které poznatky jsou přenositelné a které specifické pro tento případ. Zvláště zajímavá je otázka projektů s vyšší nejednoznačností specifikace (sekce ??), kde by obecnější instrukce (enabling constraints) mohly být efektivnější než detailní příkazy.

[RAW]

Future work.

- Více modelů, více projektů
- Systematické porovnání formátů specifikace
- Multi-agent vs. single-agent scaffolding
- Longitudinální studie (více issues, evoluce projektu)
- **Proč agent nedodržuje multi-step workflow.** Pilotní data ukazují, že agent dodržuje

verifikační příkazy (jednokrokové akce) deterministicky, ale multi-step workflow (issue → branch → test → PR) nedeterministicky: r3 dodržel kompletně, r5 zcela ignoroval při téměř identických instrukcích. Možné vysvětlení: (a) jednoduché příkazy nevyžadují plánování, agent je kopíruje do terminálu; workflow vyžaduje udržet plán přes desítky kroků. (b) Tréninková data modelu obsahují oba vzorce, strukturovaný i nestrukturovaný vývoj, a model mezi nimi volí stochasticky. (c) Pozice instrukce v kontextovém okně: s rostoucí délkou konverzace klesá vliv dřívějších instrukcí. Granulární ablace workflow struktury (které kroky jsou kritické?) a opakované běhy se stejnými instrukcemi by tento fenomén pomohly lépe pochopit.

- **Kontinuita práce mezi agenty.** procesní metriky (P1–P8) měří transparentnost artefaktů: issues, commit zprávy, PR popisy. Otevřená otázka: stačí tato transparentnost k tomu, aby jiný agent (nebo člověk) navázal na rozpracovaný projekt? Testování handoff scénářů by ověřilo, zda procesní scaffolding vytváří nejen pozorovatelný, ale i přenositelný pracovní kontext.
- **Automatizace iteračního postupu.** Fáze Diagnóza a Úprava jsou v této práci prováděny manuálně. Frameworky jako PromptWizard (Agarwal et al., 2024) (Score→Critique→Synthesize) nebo Prompt Alchemy („Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation“, 2025) ukazují, že tento cyklus lze automatizovat: model sám analyzuje kde instrukce selhaly a navrhuje úpravy. Kombinace automatizovaného APO s metrikami P/Q/E by umožnila rychlejší iteraci bez manuální diagnózy.
- **Ingestion checkpoint.** Pilotní data ukazují, že agent instrukce přečte (v transcriptu cituje jejich obsah), ale neaktivuje je jako řídicí smyčku běhu; místo workflow z AGENTS.md použije vlastní implementační smyčku odvozenou ze specifikace. Ověřitelný checkpoint na začátku běhu (agent musí explicitně potvrdit přečtení a aktivaci workflow před prvním zápisem kódu) by mohl tento problém adresovat.

Doporučení k validaci (strukturální audit, 2026-04-22): Při finalizaci 5.5 zkontrolovat, zda výsledný seznam drží jednotnou logiku. Preferovaný stav: každá položka = jedna otevřená výzkumná otázka nebo jedno jasné metodické rozšíření této studie. Kandidáti na vyřazení nebo přesun jsou body, které už působí jako návrh nového prompt designu nebo jako repackaging předchozí diskuse.

[RAW]

Feature work / námět k diskuzi: exit kritéria jako součást instrukcí:

Otázka: co by se stalo kdybychom agentovi předali exit kritéria přímo v AGENTS.md? (“Musíš dosáhnout P1=5/5, Q3≥70 %, Q5=0; ověř si to před PR.”)

Argument PRO: pokud máme dostatečný počet metrik pokrývajících různé dimenze (proces, kvalita, efektivita), optimalizace na metriky = dobrá práce. Goodhartův zákon platí pro jednorozměrné metriky; agent který optimalizuje jen coverage může zanedbat kvalitu. Ale s 19 metrikami napříč P/Q/E je těžší gaminovat všechny najednou bez skutečné kvality.

Argument PROTI: agent může najít zkratky (napr. trivální testy pro Q3, verbose kód pro Q7). Také: explicitní metriky v instrukcích mění povahu experimentu; měříme pak schopnost agenta sledovat metriky, ne schopnost dodržovat vývojové praktiky.

Kompromis k prozkoumání: přidat do AGENTS.md jen deterministické quality gates jako self-check (“Run tsc --noEmit before PR, run eslint src/, zero warnings required”) bez číselných prahů. Agent ví co zkontrolovat, ale neví jaký je náš práh → nelze triválně optimalizovat.

Závěr

Tato práce navrhla sadu metrik a iterativní postup pro systematické navrhování a vyhodnocování instrukcí pro AI coding agenty a ablacemi prozkoumala vliv jednotlivých složek instrukcí. Pro všechny tři cíle případová studie systému upomínek faktur demonstrovala proveditelnost navrženého postupu a přinesla indikativní poznatky. Studie zahrnovala pět pilotních iterací a čtyři ablační běhy.

Prvním cílem bylo navrhnout sadu metrik pokrývající proces a kvalitu práce agenta, ne jen výsledek. Výsledná sada 19 metrik pokrývá tři dimenze odvozené z taxonomie Fentona a Biemana. Procesní metriky odhalily problémy, které výsledkové metriky samy o sobě nezachytily: běh s téměř plným skóre funkčních testů vykazoval nestabilní procesní compliance. Kombinace deterministických a kvalitativních metrik přinesla komplementární pohled, protože každý typ zachycuje jinou dimenzi chování agenta.

Druhým cílem bylo iterativním postupem navrhnout instrukce, které dovedou agenta k dodržování stanovených exit kritérií. Cyklus měření, diagnózy a úpravy vedl k měřitelnému zlepšení, ale ne monotónnímu: jedna iterace způsobila regresi, po které bylo nutné vrátit se k předchozí verzi instrukcí. Opakujícím se vzorcem úspěšných úprav bylo nahrazení obecného pravidla konkrétním příkazem s verifikačním krokem. Účinnost tohoto postupu závisela na dostupnosti deterministických nástrojů s jednoznačným výstupem. U požadavků bez takového nástroje je dosažitelná míra operacionalizace nižší.

Třetím cílem bylo ablacemi prozkoumat, které složky instrukční sady přispívají k měřenému chování agenta. Verifikační kroky se ukázaly jako neredundantní: jejich odebrání zhoršilo funkční korektnost i čistotu kódu. Konvence kvality kódu automatizované metriky neovlivnily, ale designová kvalita hodnocená LLM-as-judge klesla. Výsledky zároveň ukázaly, že procesní compliance je výrazně citlivá na nedeterminismus modelu. Stejná sada instrukcí proto může vést k odlišnému dodržení vícekrokového workflow.

Hlavním přínosem práce je iterativní postup a sada metrik, které může praktik aplikovat na vlastním projektu s vlastními prahy. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii. Přenositelný je postup, ne výsledná čísla. Práce je omezena na jeden model a jeden projekt. Závěry mají povahu analytické generalizace na teorii, nikoliv statistické generalizace na populaci.

Pro praktika z toho plyne jednoduché doporučení: nestačí sledovat, zda agent úkol vyřešil, ale i jak k řešení došel a jak kvalitní kód zanechal. Pro akademického čtenáře práce ukazuje, že instrukce lze studovat ne jen jako promptový artefakt, ale jako samostatně měřenou nezávislou proměnnou, jejíž účinek je třeba hodnotit přes proces, produkt i náklady.

[RAW]

Audit trail (DRAFT v1, 2026-03-26): 5 odstavců: 3 cíle + přínos + omezení. Číslo

4/10→9/10, “ve dvou iteracích” (nepřesné), “principy přenositelné” (vágní).

Audit trail (DRAFT v2, 2026-04-05): Opraveno: r4 regrese explicitně zmíněna (ne monotónní zlepšení); přidána pointa o procesních metrikách (r3 Q2=41/42 ale P1 kolísá); nuance o verifikačních krocích (závislost na deterministických nástrojích); přenositelnost upřesněna (postup ano, čísla ne); “ve dvou iteracích” odstraněno (nepřesné); konkrétní metriky u ablačních výsledků.

Bibliography

- ACE-Bench: End-to-End Feature Development Benchmark [212 tasks from 16 repos; Claude 4 Sonnet + OpenHands: 7.5% on feature-level tasks]. (2025). <https://openreview.net/forum?id=41xrZ3uGuI>
- Agarwal, A., et al. (2024). PromptWizard: Task-Aware Prompt Optimization Framework [Microsoft Research. arXiv:2405.18369. Iterativní Score→Critique→Synthesize cyklus pro optimalizaci promptů.].
- Bacchelli, A., & Bird, C. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. *Proceedings of the 35th International Conference on Software Engineering*, 712–721. <https://doi.org/10.1109/ICSE.2013.6606617>
- Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- Bockeler, B. (2025). *Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl* [Critical analysis of SDD tools. Spec-kit files “repetitive, both with each other, and with the code”]. <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- Breunig, D. (2025). Don’t Fight the Weights [Defines fighting-the-weights: when prompt instructions oppose trained model behavior; recognition signs and mitigation strategies]. <https://www.dbreunig.com/2025/11/11/don-t-fight-the-weights.html>
- Brooks, F. P. (1987). No Silver Bullet: Essence and Accidents of Software Engineering. *Computer*, 20(4), 10–19. <https://doi.org/10.1109/MC.1987.1663532>
- Cao, L., & Ramesh, B. (2008). Agile Requirements Engineering Practices: An Empirical Study [16 organizations, 7 agile RE practices identified]. *IEEE Software*, 25(1), 60–67. <https://doi.org/10.1109/MS.2008.1>
- Ehsani, R., et al. (2026). Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub [33k agent PRs, 5 coding agents, taxonomy of failures]. *Proceedings of MSR 2026*. <https://arxiv.org/abs/2601.15195>
- FeatureBench: Benchmarking Agentic Coding for Complex Feature Development [Agents struggle when scope exceeds single-issue; feature-level tasks significantly harder]. (2026). *arXiv preprint arXiv:2602.10975*.
- Fenton, N. E., & Bieman, J. (2014). *Software Metrics: A Rigorous and Practical Approach* (3rd ed.) [Process vs. product vs. resource metrics taxonomy; GQM framework]. CRC Press.
- Fowler, M. (2004). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd). Addison-Wesley.
- Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2025). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? [LLM-generated context files reduce success rate; developer-written marginally help (+4%); context files increase cost 20%+; agents follow instructions but no performance gain; AGENTBENCH benchmark (138 instances, 12 repos)]. *arXiv preprint arXiv:2602.11988*. <https://arxiv.org/abs/2602.11988>

- Gotel, O. C., & Finkelstein, A. C. (1994). An Analysis of the Requirements Traceability Problem. *Proceedings of IEEE International Conference on Requirements Engineering*, 94–101.
- Guo, J., Huang, S., Li, M., Huang, D., Chen, X., Zhang, R., Guo, Z., Yu, H., Yiu, S.-M., Lio, P., & Lam, K.-Y. (2025). A Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System [Over 150 papers surveyed]. *arXiv preprint arXiv:2510.09721*. <https://arxiv.org/abs/2510.09721>
- Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). Agentic Software Engineering: Foundational Pillars and a Research Roadmap [SASE framework, BriefingScript, MentorScript, SE4H/SE4A duality]. *arXiv preprint arXiv:2509.06216*. <https://arxiv.org/abs/2509.06216>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research [Foundational DSR framework: build-evaluate cycle, 7 guidelines for design science research]. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- Chen, J., Lin, J., Xiong, Y., Lu, J., Zhang, H., & Xie, T. (2025). Rethinking the Value of Agent-Generated Tests in Autonomous Code Repair [83.2% tasks same outcome regardless of test writing; tests serve as observational feedback]. *arXiv preprint arXiv:2505.21615*.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., . . . Zaremba, W. (2021). Evaluating Large Language Models Trained on Code [Introduces HumanEval benchmark (164 hand-written Python problems) and Codex model. pass@k metric for functional correctness.]. *arXiv preprint arXiv:2107.03374*. <https://arxiv.org/abs/2107.03374>
- IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge* (Version 4.0). <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- ISO/IEC. (2023). *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model* (tech. rep. No. ISO/IEC 25010:2023). International Organization for Standardization. Geneva. <https://www.iso.org/standard/78176.html>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? [2294 tasks from real GitHub repositories, de facto benchmark for AI coding agents]. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.06770>
- Kim, O. (2025). DETAIL Matters: Measuring the Impact of Prompt Specificity on Reasoning in Large Language Models [Emory University. Framework pro evaluaci vlivu specifčnosti promptu na reasoning. 30 úloh, GPT-4 a O3-mini]. *arXiv preprint arXiv:2512.02246*. <https://arxiv.org/abs/2512.02246>
- Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. *Proceedings of the IEEE*, 68(9), 1060–1076. <https://doi.org/10.1109/PROC.1980.11805>

- Li, B., Latendresse, J., Abdalkareem, R., & Shang, W. (2026). Beyond Bug Fixes: Post-Merge Code Quality Issues in Agent-Generated Pull Requests [1210 merged agent PRs, SonarQube analysis. Merge success unreliable indicator of quality. Code smells, complexity, duplicated code persist.]. *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2601.20109>
- Li, X., Chen, W., Liu, Y., Zheng, S., Chen, X., He, Y., Li, Y., et al. (2025). SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks [Curated Skills +16.2pp; self-generated Skills -1.3pp; 2–3 focused modules optimal; comprehensive documentation hurts (-2.9pp); smaller model + Skills can exceed larger model without; 84 tasks, 11 domains, 7308 trajectories]. *arXiv preprint arXiv:2602.12670*. <https://arxiv.org/abs/2602.12670>
- Liu, J., Wang, K., Chen, Y., Peng, X., Chen, Z., Zhang, L., & Lou, Y. (2024). Large Language Model-Based Agents for Software Engineering: A Survey [124 papers surveyed]. *arXiv preprint arXiv:2409.02977*. <https://arxiv.org/abs/2409.02977>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts [U-shaped performance curve; info in middle performs worse than no-context baseline]. *Transactions of the Association for Computational Linguistics*, 12.
- Lulla, K., Zhu, M., Kalliamvakou, E., & Mohylevskyy, Y. (2026). On the Impact of AGENTS.md Files on AI Coding Agents [124 PRs across 10 repos: -28% runtime, -20% output tokens with AGENTS.md; architectural info and coding conventions most effective]. *arXiv preprint arXiv:2601.20404*. <https://arxiv.org/abs/2601.20404>
- Mao, Y., He, J., & Chen, C. (2025). From Prompts to Templates: A Systematic Prompt Template Analysis for Real-world LLM Applications [Merged 4 frameworks (Google Cloud, Elavis Saravia, CRISPE, LangGPT); 7 component types; Directive 87% prevalence]. *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. <https://arxiv.org/abs/2504.02052>
- Mathews, N. S., & Nagappan, M. (2024). LLM-Based Test Generation as Bug Validation: Insights from Reproducing Real-World Bugs [68.1% generated test suites validate bugs instead of detecting them]. *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware)*. <https://doi.org/10.1145/3664646.3664766>
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, SE-2(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- McConnell, S. (2004). *Code Complete: A Practical Handbook of Software Construction* (2nd). Microsoft Press.
- McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2016). An Empirical Study of the Impact of Modern Code Review Practices on Software Quality. *Empirical Software Engineering*, 21(5), 2146–2189. <https://doi.org/10.1007/s10664-015-9381-9>
- METR. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity [RCT: 16 developers, 246 tasks; AI tools slowed experienced developers by 19%; developers believed they were 24% faster]. *arXiv preprint arXiv:2507.09089*. <https://arxiv.org/abs/2507.09089>

- METR. (2026). Many SWE-bench-Passing PRs Would Not Be Merged into Main [296 patches reviewed by 4 maintainers from 3 SWE-bench repos. ~24pp gap between automated pass rate and maintainer merge decisions. Technical report.]. <https://metr.org/notes/2026-03-10-many-swe-bench-passing-prs-would-not-be-merged-into-main/>
- Miguel, J. P., Mauricio, D., & Rodríguez, G. (2014). A Review of Software Quality Models for the Evaluation of Software Products. *International Journal of Software Engineering & Applications (IJSEA)*, 5(6), 31–53. <https://doi.org/10.5121/ijsea.2014.5603>
- Min, S., Lyu, X., Holtzman, A., Artetxe, M., Lewis, M., Hajishirzi, H., & Zettlemoyer, L. (2022). Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? [Label space, input distribution a formát jsou klíčové pro ICL, ne správnost label-input mapování]. *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/2202.12837>
- Nonaka, I., & Takeuchi, H. (1995). *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press.
- Panickssery, A., Bowman, S. R., & Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations [Causal link: self-recognition → self-preference; linear correlation; fine-tuning pushes recognition to 90%+]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2404.13076>
- Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., & Harman, M. (2019). Mutation Testing Advances: An Analysis and Survey [Mutation score stronger predictor of fault detection than structural coverage]. *Advances in Computers*, 112, 275–378. <https://doi.org/10.1016/bs.adcom.2018.03.015>
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2008). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Piskala, D. B. (2026). *Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants* (tech. rep.) (Technical report. Three levels: spec-first, spec-anchored, spec-as-source. SDD workflow: Specify → Plan → Implement → Validate). arXiv. <https://arxiv.org/abs/2602.00180>
- Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation [arXiv:2503.11085. Iterativní refinement promptů pro code generation, evaluate přes execution.]. (2025).
- Rafique, Y., & Mišić, V. B. (2013). The Effects of Test-Driven Development on External Quality and Productivity: A Meta-Analysis. *IEEE Transactions on Software Engineering*, 39(6), 835–856. <https://doi.org/10.1109/TSE.2012.28>
- Razavi, S. P., & Fard, F. H. (2025). What Did I Do Wrong? Quantifying LLMs’ Sensitivity and Consistency to Prompt Engineering [Up to 76 accuracy points difference from subtle formatting changes in few-shot settings]. *Proceedings of NAACL*. <https://arxiv.org/abs/2406.12334>
- Runeson, P., & Höst, M. (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>

- Scacchi, W. (2002). Understanding the Requirements for Developing Open Source Software Systems [Software informalisms in OSS – issue trackers as de facto requirements]. *IEEE Proceedings – Software*, 149(1), 24–39. <https://doi.org/10.1049/ip-sen:20020202>
- Scale AI. (2025). SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? [1865 human-verified tasks, adds explicit Requirements and Interface sections to issues]. *arXiv preprint arXiv:2509.16941*. <https://arxiv.org/abs/2509.16941>
- Shihab, E., Bird, C., & Zimmermann, T. (2012). The Effect of Branching Strategies on Software Quality. *Proceedings of the ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, 301–310. <https://doi.org/10.1145/2372251.2372305>
- Shin, J., Tang, C., Mohati, T., Nayebi, M., Wang, S., & Hemmati, H. (2025). Prompt Engineering or Fine-Tuning: An Empirical Assessment of Large Language Models for Code. *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2310.10508>
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*.
- Sommerville, I. (2016). *Software Engineering* (10th). Pearson.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Verga, P., Hofstätter, S., Cer, D., & Thorne, J. (2024). Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models [Panel of LLM evaluators (PoLL) outperforms single GPT-4 judge; disjoint model families reduce intra-model bias; 7x cheaper]. *arXiv preprint arXiv:2404.18796*. <https://arxiv.org/abs/2404.18796>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models [Google Research. Few-shot CoT exempláře zlepšují reasoning na aritmetických, commonsense a symbolických úlohách]. *Advances in Neural Information Processing Systems (NeurIPS)*, 35. <https://arxiv.org/abs/2201.11903>
- Wei, Y., et al. (2025). SWE-EVO: Long-Horizon Software Evolution Tasks [Multi-step modifications spanning avg 21 files; GPT-5 + OpenHands only 21% vs 65% on single-issue SWE-Bench]. *arXiv preprint arXiv:2512.18470*. <https://arxiv.org/abs/2512.18470>
- Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents [Komplexní agenti vs. levnější bez-agent přístup na SWE-bench; publikováno též na ICML 2025]. *arXiv preprint arXiv:2407.01489*. <https://arxiv.org/abs/2407.01489>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.
- Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6th ed.) [Embedded single-case design; analytic generalization (to theory, not population)]. SAGE Publications.

- Yin, Z., Wang, J., Li, J., Shi, D., Wei, Y., Yue, Y., Liu, Z., Xia, X., & Lo, D. (2025). A Comprehensive Empirical Evaluation of Agent Frameworks on Code-centric Software Engineering Tasks. *arXiv preprint arXiv:2511.00872*.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena [Foundational LLM-as-judge paper; position bias, verbosity bias, self-enhancement bias; GPT-4 >80% human agreement]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2306.05685>
- Zhu et al. (2026). The Necessity of a Unified Framework for LLM-Based Agent Evaluation [arXiv:2602.03238. Benchmarky jsou konfundovány system prompty — instrukce jsou nezávislá proměnná.].
- Zi, Y., Menon, H., & Guha, A. (2025). More Than a Score: Probing the Impact of Prompt Specificity on LLM Code Generation [PartialOrderEval framework. Partial order of prompts from minimal to maximally detailed. HumanEval + ParEval]. *arXiv preprint arXiv:2508.03678*. <https://arxiv.org/abs/2508.03678>

[RAW]

Zdroje k zpracování do BibTeX (původně z notes/sources.md)

1. PRIMÁRNÍ ZDROJE (Frameworky a Metodiky)

GITHUB NEXT. Spec Kit: Spec-Driven Development for AI Agents [online]. 2024. <https://github.com/github/spec-kit> – Klíčový zdroj pro koncept "Spec-Driven Development".

BMAD-CODE-ORG. BMAD METHOD: Breakthrough Method for Agile AI-Driven Development [online]. GitHub, 2025. <https://github.com/bmad-code-org/BMAD-METHOD> – Metodika pro řízení AI agentů v agilním vývoji.

ANTHROPIC. Effective Harnesses for Long-Running Agents [online]. Anthropic Engineering Blog, 2024. <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents> – Technický popis problémů s dlouhodobou pamětí agentů.

METR. Model Evaluation and Threat Research [online]. 2024. <https://metr.org/> – Standardy pro hodnocení bezpečnosti a schopností modelů.

THEDOTMACK. claude-mem: Persistent Memory System for Claude Code [online]. GitHub, 2025. <https://github.com/thedotmack/claude-mem> – Příklad implementace hooks v CLI agentech - persistentní paměť přes session.

1.2 SYSTÉMOVÉ MYŠLENÍ V SWE

PETKOV, Doncho et al. Information Systems, Software Engineering, and Systems Thinking: Challenges and Opportunities. International Journal of Information Technologies and Systems Approach. <https://www.igi-global.com/gateway/article/2534> – Mapuje historii systémového přístupu v IS a SWE. Propojení systémového myšlení s praxí SWE je stále nedotažené.

MONAT, Jamie a GANNON, Thomas. Systems Thinking: A Review and Bibliometric Analysis. MDPI Systems, 2020. <https://www.mdpi.com/2079-8954/8/3/23> – Přehled co systémové myšlení je a kde se používá. Interdisciplinární - SWE, management, vzdělávání.

ALHARTHI, Sultan et al. A Systems Thinking Approach to Improve Sustainability in Software Engineering. MDPI Sustainability, 2023. <https://www.mdpi.com/2071-1050/15/11/8766> – Praktická aplikace - dívají se na vývoj jako systém (developeři, zákazníci, stakeholders).

2. ODBORNÉ STUDIE

2.1 Přehledové studie (Surveys) - LLM agenti v SE

LIU, Junwei et al. Large Language Model-Based Agents for Software Engineering: A Survey. arXiv preprint arXiv:2409.02977. 2024. <https://arxiv.org/abs/2409.02977> – Komplexní přehled 106 prací o LLM agentech v SE, kategorizace z pohledu SE i agentů.

JIN, Haolin et al. From LLMs to LLM-based Agents for Software Engineering: A Survey of Current, Challenges and Future. arXiv preprint arXiv:2408.02479. 2024. <https://arxiv.org/abs/2408.02479> – Pokrývá requirements, code generation, testing, maintenance - celý SDLC.

Comprehensive Survey on Benchmarks and Solutions in Software Engineering of LLM-Empowered Agentic System. arXiv preprint arXiv:2510.09721. 2025. <https://arxiv.org/html/2510.09721> – Přes 150 paperů, taxonomie řešení a benchmarků.

A Survey on Code Generation with LLM-based Agents. arXiv preprint arXiv:2508.00083. 2025. <https://arxiv.org/abs/2508.00083> – Single-agent a multi-agent architektury, aplikace napříč SDLC.

2.2 Agentic Software Engineering - Klíčové práce

Agentic Software Engineering: Foundational Pillars and a Research Roadmap. arXiv preprint arXiv:2509.06216. 2025. <https://arxiv.org/html/2509.06216v2> – Přehodnocení SE pro spolupráci člověk-agent. Framework podobný SAE úrovní autonomie. Rozlišuje SE 2.0 (AI-augmented) vs SE 3.0 (Agentic SE).

AKBAR, Muhammad Azeem et al. Agentic AI in Software Engineering: Practitioner Perspectives Across the Software Development Life Cycle. SSRN. 2025. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5520159 – Rozhovory s 21 experty, pokrývá celý SDLC. Zjištění: agenti redefinují hranice mezi fázemi SDLC.

Autonomous Agents in Software Development: A Vision Paper. Springer, 2024. https://link.springer.com/chapter/10.1007/978-3-031-72781-8_2 – 12 LLM agentů spolupracujících na celém SDLC.

2.3 Evaluace a produktivita

METR. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer

Productivity. 2025. <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/> – RCT studie: zkušení vývojáři s AI jsou o 19% pomalejší - překvapivé zjištění.

2.4 Metriky kvality software a AI agentů

ISO/IEC. ISO/IEC 25010:2023 - Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. 2023. <https://www.iso.org/standard/78176.html> – Industry standard pro kvalitu software. 8 charakteristik, Functional Suitability obsahuje Completeness, Correctness, Appropriateness.

ISO 25000. ISO 25010 Software Quality Model. 2023. <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010> – Přehledný popis ISO 25010 modelu kvality.

LXT. AI Agent Evaluation: Comprehensive Framework for Measuring Agent Performance. 2024. <https://www.lxt.ai/blog/ai-agent-evaluation/> – Moderní framework pro evaluaci AI agentů: Task completion, Accuracy, Safety/trust (policy compliance, transparency), Tool usage.

Weights & Biases. AI Agent Evaluation: Metrics, Strategies, and Best Practices. 2024.

<https://wandb.ai/onlineinference/genai-research/reports/AI-agent-evaluation-Metrics-strategies-and-best-practices--VmlldzoxMjM0NjQzMQ>

– Praktický průvodce metrikami pro AI agenty.

SOMMERVILLE, Ian. Software Engineering. 10th ed. Pearson, 2016. Chapter 24: Quality Management (s. 705–728). – Rozlišení control metrics (procesní) vs. predictor metrics (produktové). Vztah proces-produkt u SW. Relevantní pro oporu dimenzí Functional Quality a Compliance v kap03.

McCONNELL, Steve. Code Complete: A Practical Handbook of Software Construction. 2nd ed. Microsoft Press, 2004. Chapter 28 (s. 715, Table 28-2). – Tabulka “Useful Software-Development Measurements” – kategorie Size a Overall Quality (defekty, defekty/KLOC, mean time between failures).

IEEE COMPUTER SOCIETY. SWEBOK: Guide to the Software Engineering Body of Knowledge. Version 4.0. IEEE, 2024. Chapter 12: Software Quality (s. 248–256), Chapter 6: Software Maintenance (s. 176). – Software Quality Measurement (s. 253) – kvantifikace atributů pro rozhodování. Míry údržby (s. 176): complexity, maintainability, testability, reliability.

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-world Github Issues? In: ICLR. 2024. Appendix C.7 (s. 28). – Software Engineering Metrics v benchmarku – cyklomatická složitost (McCabe), Halstead measures pro hodnocení kódu agentů.

2.5 Základní LLM studie

JIMENEZ, Carlos E. et al. SWE-bench: Can Language Models Resolve Real-world Github

Issues? In: The Twelfth International Conference on Learning Representations (ICLR). 2024. <https://arxiv.org/abs/2310.06770> – Hlavní benchmark pro hodnocení schopností programovacích agentů.

LIU, Nelson F. et al. Lost in the Middle: How Language Models Use Long Contexts. arXiv preprint arXiv:2307.03172. 2023. <https://arxiv.org/abs/2307.03172> – Klíčová studie vysvětlující, proč pouhé zvětšení kontextového okna nestačí.

VASWANI, Ashish et al. Attention Is All You Need. Advances in Neural Information Processing Systems, 2017. <https://arxiv.org/abs/1706.03762> – Základní paper definující Transformer architekturu a mechanismus pozornosti (self-attention).

WEI, Jason et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. Advances in Neural Information Processing Systems, 2022. <https://arxiv.org/abs/2201.11903> – Základ pro techniku prompt engineeringu používané v práci.

3. TEORIE SOFTWAREOVÉHO INŽENÝRSTVÍ A ARCHITEKTURY

RICHARDS, Mark a FORD, Neal. Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media, 2020. ISBN 978-1492043454. – Moderní přehled architektonických stylů a charakteristik ("ilities").

FOWLER, Martin. Patterns of Enterprise Application Architecture. Addison-Wesley Professional, 2002. ISBN 978-0321127426. – Katalog základních návrhových vzorů pro podnikové aplikace.

KHONONOV, Vlad. Learning Domain-Driven Design: Aligning Software Architecture and Business Strategy. 1. vyd. O'Reilly Media, 2021. ISBN 978-1098100131. – Definuje pojmy jako "Bounded Context" a "Ubiquitous Language", které jsou analogií pro kontext LLM.

ISO/IEC. ISO/IEC/IEEE 42010:2011 Systems and software engineering — Architecture description. 2011. – Mezinárodní standard definující základní pojmy popisu architektury.

IEEE COMPUTER SOCIETY. SWEBOK: Guide to the Software Engineering Body of Knowledge. Version 3.0. IEEE, 2014. <https://www.swebok.org/> – Standardní taxonomie softwarového inženýrství.

BROWN, Simon. The C4 model for visualising software architecture. 2024. <https://c4model.com/> – Metodika pro hierarchický popis architektury, vhodná pro strojové zpracování.

ARC42. arc42 Template for Software Architecture Documentation. 2024. <https://arc42.org/> – Pragmatická šablona pro strukturování architektonické dokumentace.

4. DALŠÍ ZDROJE (Historický kontext a Procesy)

NATO SCIENCE COMMITTEE. Software Engineering: Report on a conference sponsored by the NATO Science Committee. Garmisch, Germany, 1968. – Historický kontext vzniku

disciplíny.

KAUR, Rupinder a SENGUPTA, Jyotsna. Software Process Models and Analysis on Failure of Software Development Projects. In: arXiv preprint arXiv:1306.1068. 2013.

Přílohy

A. Formulář v plném znění

B. Zdrojové kódy výpočetních procedur

C. Baseline AGENTS.md

Baseline verze AGENTS.md (pilot-r1) a její souhrnná evoluce do pilot-r3 jsou uvedeny v kapitole ?? (obrázky ?? a ??).

D. Využití nástrojů umělé inteligence

Při přípravě práce byl využit nástroj Claude (Anthropic, modely Opus 4 a Sonnet 4) jako AI asistent v následujících oblastech:

Psaní textu práce. Nástroj byl využit pro drafting a úpravu textových částí (kapitoly 1–5), kontrolu konzistence argumentace a LaTeX formátování. Autor formuloval záměr a strukturu každé sekce; AI asistent navrhoval formulace, které autor revidoval a upravoval. Výzkumný design, argumentace a finální znění jsou autorovy vlastní.

Vyhledávání a shrnutí literatury. Nástroj byl využit k identifikaci relevantních zdrojů a shrnutí obsahu papers. Autor ověřoval relevanci a správnost každého zdroje před zařazením do textu; citace odkazují výhradně na zdroje přečtené a posouzené autorem.

Diagnostická analýza v experimentu. V kroku Diagnóza iteračního cyklu (sekce ??) byl nástroj využit jako analytický asistent: výzkumník popisoval naměřené výsledky, AI asistent navrhoval možné příčiny selhání a opravy instrukcí na základě literatury. Finální rozhodnutí o každé změně `AGENTS.md` bylo vždy na autorovi.

Implementace měřicí infrastruktury. Experimentální skripty (`new-run.ts`, `analyze-run.ts`, `judge.ts`) byly implementovány s asistencí AI nástroje na základě autorova návrhu specifikace metrik. Autor navrhl architekturu, definoval požadované výstupy a provedl validaci správnosti měření (příloha ??).

AI nástroj nepouštěl experimenty, nevyhodnocoval výsledky a nerozhodoval o směru výzkumu. Za celý obsah práce nese plnou odpovědnost autor.